

Master Degree in Big Data Analytics
2024-2025

Master Thesis

"Mathematical Analysis of Fairness and Bias in Recommender Systems"

Jorge Moral Fernández

Sandra Benítez Peña

Rosa Elvira Lillo Rodríguez

Madrid, June 2025

AVOID PLAGIARISM

The University uses the **Turnitin Feedback Studio** for the delivery of student work. This program compares the originality of the work delivered by each student with millions of electronic resources and detects those parts of the text that are copied and pasted. Plagiarizing in a TFM is considered a **Serious Misconduct**, and may result in permanent expulsion from the University.



This work is licensed under Creative Commons **Attribution – Non Commercial – Non Derivatives**

SUMMARY

This Master's Thesis explores the dual challenge of achieving both accuracy and fairness in recommender systems, with a particular focus on the issue of popularity bias. While most fairness research concentrates on sensitive demographic attributes such as gender or race, this work shifts the focus towards how popular items tend to dominate recommendations, potentially limiting diversity and disadvantaging niche content. Using the MovieLens 100k dataset, which provides rich user-item interactions and metadata, we comprehensively evaluate a variety of recommendation algorithms, including collaborative filtering and latent factor models, assessing their predictive quality and personalization ability.

In addition, we apply fairness evaluation protocols adapted from prior works to measure the effects of popularity bias on recommendation outcomes. Our findings highlight significant trade-offs between optimizing accuracy and ensuring equitable item exposure, revealing that some models struggle to generalize fairly under protocols designed to mitigate popularity bias.

Keywords: Recommender Systems, Fairness, Popularity Bias, Collaborative Filtering, Latent Factor Models, MovieLens Dataset.

DEDICATION

To my parents, whose unwavering support and endless encouragement have been the foundation of every step I take. Like a steady coach on the tennis court, you have taught me patience, resilience, and the power of perseverance. Your belief in me lights the way even in the darkest moments.

To my tutors, whose guidance, patience, and wisdom have shaped not only this work but also my growth as a learner and thinker. Just as in a challenging tennis match, you've helped me strategize, adapt, and keep my focus on the goal. Thank you for inspiring me to reach beyond what I thought possible.

This journey, much like a great game of tennis, is won point by point with teamwork, determination, and heart.

CONTENTS

1. INTRODUCTION.	1
2. BASICS OF RECOMMENDER SYSTEMS	3
2.1. Objectives of Recommender Systems	3
2.1.1. Prediction-Based Recommendation (Matrix Completion)	4
2.1.2. Ranking-Based Recommendation (Top-k Recommendation)	5
2.2. Reality in Recommender Systems	8
2.3. Methodological Approach.	9
3. COLLABORATIVE FILTERING	10
3.1. Neighborhood-Based Collaborative Filtering	10
3.1.1. User-Based Neighborhood Models	11
3.1.2. Item-Based Neighborhood Models	15
3.1.3. Advanced methodology for Neighborhood Models	16
3.2. Model-Based Collaborative Filtering.	19
3.2.1. Decision Trees and Their Extension to Collaborative Filtering	19
3.2.2. Naive Bayes in Collaborative Filtering.	20
3.3. Latent Factor Models	23
3.3.1. Dimensionality Reduction and Matrix Factorization	23
3.3.2. Unconstrained factorization for Fully-Specified Matrices	24
3.3.3. Singular Value Decomposition in Recommender Systems	26
4. CONTENT-BASED RECOMMENDER SYSTEMS	28
4.1. Preprocessing and Feature Extraction	29
4.2. Learning User Profiles	30
4.2.1. Nearest Neighbor Classification	30
4.2.2. Bayes Classifier	31
4.2.3. Regression-based Models	32
5. THEORETICAL FOUNDATIONS OF FAIRNESS	35
5.1. The Framework of Classification Problems	35

5.2. Statistical Criteria for Non-Discrimination	38
5.2.1. Independence	39
5.2.2. Separation.	41
5.2.3. Sufficiency	43
5.3. Incompatibility of Criteria	45
6. FAIRNESS APPLIED TO RECOMMENDER SYSTEMS	47
6.1. Factors Contributing to Unfairness in Recommender Systems.	47
6.1.1. Data Bias in Recommender Systems	47
6.1.2. Model Bias in Recommender Systems	48
6.1.3. Feedback Bias in Recommender Systems	48
6.2. Expressions of Fairness in Recommender Systems	48
6.2.1. Individual Fairness vs. Group Fairness.	49
6.2.2. Static Fairness vs. Dynamic Fairness.	49
6.2.3. Single-sided Fairness vs. Multi-sided Fairness.	50
6.3. Techniques for Enhancing Fairness in Recommendations	51
6.3.1. Pre-Processing Approaches for Fairness in Recommender Systems.	51
6.3.2. In-processing methods for fairness-aware recommender systems	52
6.3.3. Post-processing methods for fairness-aware recommender systems.	52
7. CASE STUDY: MOVIELENS	54
7.1. Methodology used for this analysis.	54
7.1.1. Collaborative Filtering Evaluation	54
7.1.2. Fairness and Popularity Bias Evaluation	55
7.1.3. Evaluation Metrics	55
7.2. Dataset description & Exploratory Analysis.	56
7.3. Evaluation of Recommender System Performance	58
7.4. Evaluation of Bellogín's U1R & U3R Models.	67
8. DISCUSION AND VISUALIZATION OF RESULTS.	78
8.1. Visualization and Analysis of Collaborative Filtering Results	78
8.2. Visualization and Analysis of U1R & U3R Results.	80
9. CONCLUSIONS	83
BIBLIOGRAFÍA	83

A. PROOFS OF THEOREMS AND PROPOSITIONS	85
B. THESIS PROJECT TIMELINE	98

LIST OF FIGURES

7.1	Exploratory Data Analysis (EDA) of the MovieLens 100k dataset. The plot on the left shows the distribution of ratings, while the plot on the right displays the top 10 most rated movies.	57
7.2	Distribution of ratings per movie in the MovieLens 100k dataset.	58
7.3	Comparison of boxplots for user-based models with Pearson and Cosine similarity across the different defined metrics.	61
7.4	Comparison of boxplots for item-based Cosine and Adjusted Cosine similarity across the previously defined metrics.	64
7.5	Comparison of boxplots for Pearson and Cosine similarity under the U1R protocol.	70
7.6	Comparison of boxplots for Pearson and Cosine similarity under the U3R protocol.	71
7.7	Boxplot comparison of Item-based CF models using Cosine and Adjusted Cosine similarity for U1R	73
7.8	Boxplot comparison of Item-based CF models using Cosine and Adjusted Cosine similarity for U3R	74
8.1	RMSE results of all the classical models considered, evaluated using 5-fold cross-validation.	78
8.2	MAE results of all the classical models considered, evaluated using 5-fold cross-validation.	79
8.3	Average per-fold execution time of each model during cross-validation.	79
8.4	RMSE results for all classical models under the different evaluation setups.	81
8.5	MAE results for all classical models under the different evaluation setups.	81

LIST OF TABLES

5.1	Confusion matrix	36
7.1	Benchmark Models used as reference points to evaluate the performance of different recommendation algorithms throughout the study.	59
7.2	Results of the user-based models with Pearson similarity for different values of k (number of neighbors), showing the impact of neighborhood size on recommendation accuracy.	60
7.3	Results of the user-based models with Cosine similarity for different values of k (number of neighbors), showing the impact of neighborhood size on recommendation accuracy.	60
7.4	Results of the collaborative filtering models using the discounted Pearson similarity for different values of β (discounting factor) and k (number of neighbors considered).	62
7.5	Results of the collaborative filtering item-based model using cosine similarity, evaluated for different values of k (number of neighbors).	63
7.6	Results of the collaborative filtering item-based model using Adjusted cosine similarity, evaluated for different values of k (number of neighbors).	63
7.7	XGBoost per-item model results with cross-validation for different number of estimators, maximum tree depth, and learning rate values.	65
7.8	Performance metrics of the Unconstrained Matrix Factorization model for different values of k (latent factors), α (learning rate) , and λ (regularization parameter)	66
7.9	Evaluation results of the SVD factorization model across different latent factors and iteration steps.	67
7.10	Evaluation results of baseline models under the Bellogín-inspired methodology (U1R).	68
7.11	Evaluation results of baseline models under the new proposed methodology (U3R).	68
7.12	Evaluation results of Memory-based Collaborative Filtering models for different values of k (neighbors) under the U1R protocol.	69
7.13	Evaluation results of Memory-based Collaborative Filtering models for different values of k (neighbors) under the porposed U3R protocol.	69

7.14	Evaluation results of Item-based Collaborative Filtering models with Cosine and Adjusted Cosine similarity under the U1R protocol, for different values of k (number of neighbors).	72
7.15	Evaluation results of Item-based Collaborative Filtering models with Cosine and Adjusted Cosine similarity under the U3R protocol, for different values of k (number of neighbors).	72
7.16	Evaluation results for the Unconstrained Matrix Factorization model using the U1R setup, across different numbers of latent factors (k), learning rates (α), and regularization parameters (λ).	75
7.17	Evaluation results for the Unconstrained Matrix Factorization model using the U3R setup, across different numbers of latent factors (k), learning rates (α), and regularization parameters (λ).	76
7.18	Evaluation results for the SVD factorization model under the U1R setup, considering different values of k (latent factors) and number of training steps.	76
7.19	Evaluation results for the SVD factorization model under the U3R setup, considering different values of k (latent factors) and number of training steps.	77

1. INTRODUCTION

In the current era of data-driven decision making, recommender systems have become essential tools for enhancing user experiences across a wide range of digital platforms. From streaming services and e-commerce to news aggregators and social media, these systems shape the content users consume by predicting their preferences and suggesting relevant items. However, as their influence grows, so does the need to scrutinize not only their accuracy and efficiency but also their fairness and ethical implications.

This Master’s Thesis addresses the dual challenge of accuracy and fairness in recommender systems. While traditional recommendation algorithms focus primarily on optimizing performance metrics such as precision or relevance, recent research has highlighted how such approaches can unintentionally amplify existing biases. These biases often lead to unequal exposure of content and can disproportionately affect certain user groups. Although the standard approach to fairness in recommender systems often focuses on sensitive user attributes such as gender, race, or age, this work adopts a different perspective in its practical component. Instead of analyzing fairness through the lens of these demographic variables, we center our attention on the issue of popularity bias, a phenomenon in which popular items receive disproportionately high exposure regardless of a user’s individual preferences. This can lead to reduced diversity in recommendations, limited visibility for niche items, and ultimately a diminished user experience for those seeking personalized content beyond the mainstream.

To explore this tension between effectiveness and equity, we conduct a comprehensive study based on the MovieLens 100k dataset, which offers a rich set of user-item interactions along with demographic and content-based information. Our analysis is structured in two parts. First, we evaluate the performance of various recommendation algorithms, including collaborative filtering methods and latent factor models, focusing on their predictive quality and personalization capabilities. Second, we assess fairness through the lens of popularity bias by applying evaluation protocols adapted from Ekstrand et al., 2018, originally developed to detect and mitigate the influence of popularity and demographic imbalances in recommender outputs.

Through this dual perspective, the goal of this thesis is to provide a better understanding of the trade-offs that arise when designing recommender systems that are both accurate and fair. By leveraging both theoretical foundations and empirical evaluation, we aim to highlight the limitations of current approaches and propose directions for developing more responsible and balanced recommendation strategies.

This project is structured into several sections, each of which addresses a specific aspect of our study on recommender systems and fairness. In Section 2, we will focus on providing a global overview of recommender systems, exploring their main objectives, commonly used methodologies, and the potential challenges that may arise when apply-

ing them in real-world scenarios. In Section 3, we will examine collaborative filtering models, which rely on user interactions to identify behavioral patterns and make predictions. In Section 4, we will analyze content-based recommender systems, which concentrate on the intrinsic features of each item and on user preferences based on those features. Section 5 will be dedicated to a theoretical introduction to the concept of fairness, presenting key definitions and foundational propositions. In Section 6, we will explore how fairness notions can be integrated into the specific case of recommender systems. Finally, Section 7 will present our practical case study, in which we analyze the MovieLens 100k dataset and implement various algorithms and training methodologies to evaluate which ones achieve the best performance, while also exploring new training methods that take popularity bias into account.

All source code and experimental results can be found in the following GitHub repository: <https://github.com/JorgeMoral1906/TFM-RecommenderSystems-Fairness>.

2. BASICS OF RECOMMENDER SYSTEMS

In the era of digital transformation, the ability to personalize content and services has become a key competitive advantage for businesses across industries. With the vast amount of data generated online, recommender systems play a crucial role in filtering and delivering relevant information to users. These systems are widely used in e-commerce, streaming platforms, and online marketplaces, shaping user experiences by suggesting products, movies, music, or even social connections.

At the core of recommender systems lies the concept of leveraging user interactions, both explicit, such as ratings, and implicit, such as browsing behavior or purchase history, to infer preferences and predict future interests. The more data available about user behavior, the more accurate and effective these recommendations become.

Several approaches have been developed to enhance recommendation accuracy. Collaborative filtering, one of the most widely used techniques, relies on the collective behavior of users to make predictions. Content-based filtering, on the other hand, focuses on the attributes of items that users have previously engaged with to determine new recommendations. Additionally, knowledge-based recommender systems take a different approach, generating suggestions based on explicitly stated user preferences rather than past interactions. Beyond these traditional methods, modern recommender systems increasingly integrate contextual information, such as time, location, and social connections, to refine recommendations. Moreover, the field continues to evolve with advancements in AI, deep learning, and hybrid models that combine multiple techniques for improved performance.

Understanding these different methodologies and their implications is essential for designing effective recommender systems that balance accuracy, fairness, and user satisfaction. This section provides an overview of the key concepts, techniques, and challenges in the field, setting the stage for a deeper exploration of recommendation models and their applications.

All the background of recommender systems covered in this section will be based on the foundational concepts presented in *Recommender Systems: The Textbook* by Aggarwal (2016), which serves as a key reference for understanding the theoretical and practical aspects of these systems.

2.1. Objectives of Recommender Systems

Recommender systems have become essential tools in various digital platforms, from e-commerce to content streaming services, as they help users navigate an overwhelming number of choices. Their primary function is to personalize the user experience by suggesting relevant items based on historical interactions and inferred preferences. However,

beyond simple personalization, recommender systems pursue several key objectives that influence their design and effectiveness.

From a mathematical point of view, recommender systems can be formulated in various ways, depending on the specific task they are designed to address. The two most prevalent formulations are **prediction-based recommendation** and **ranking-based recommendation**.

2.1.1. Prediction-Based Recommendation (Matrix Completion)

In this formulation, the recommendation problem is framed as a **matrix completion task**, where the goal is to estimate missing entries in a partially observed user-item interaction matrix. The **prediction-based recommendation** approach is the primary focus of this work, as it aligns closely with the classical machine learning methodology. It is important to note that this approach represents a methodology rather than a specific model or algorithm, which allows for the development of diverse and complex solutions. However, in this work, we will focus on the general idea behind this approach and its core techniques.

Definition 2.1.1. Let $\mathcal{U} = \{u\}_{u=1}^m$ be the set of users and $\mathcal{I} = \{i\}_{i=1}^n$ the set of items. We define the **rating matrix** $\mathbf{R} \in \mathbb{R}^{m \times n}$, where each entry r_{ui} represents the preference score given by user u to item i . Most entries are unknown, indicating missing preferences. The matrix $\mathbf{R}$ can be represented as:

$$\mathbf{R} = \begin{bmatrix} r_{11} & r_{12} & \cdots & ? & r_{1n} \\ r_{21} & ? & \cdots & r_{2i} & ? \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ ? & r_{m2} & \cdots & r_{mi} & r_{mn} \end{bmatrix}.$$

Note that the symbol “?” denotes missing values, as users generally rate only a subset of available items.

Mathematically, given a set of known entries $\mathcal{K} = \{(u, i, r_{ui})\}_{(u,i) \in \Omega}$, where $\Omega \subseteq \mathcal{U} \times \mathcal{I}$ denotes the observed user-item interactions, the goal is to learn a function $f : \mathcal{U} \times \mathcal{I} \rightarrow \mathbb{R}$ that estimates the missing preference scores:

$$\hat{r}_{ui} = f(u, i; \Theta),$$

where $\hat{r}_{ui}$ is the predicted score for user u and item i , and Θ represents the model parameters learned from the observed data.

One of the most widely used methodologies for solving this problem is **matrix factorization**. In this approach, it is assumed that user-item interactions can be approximated by the product of two low-rank matrices, where the dimensions of these matrices represent the **latent factors** of the model.

Definition 2.1.2. A *latent factor* is an unobserved, underlying characteristic that influences the interactions between users and items in a recommender system. Formally, given a user-item matrix $\mathbf{R}$, a latent factor represents a dimension in the lower-dimensional space into which the user-item interactions are mapped during the matrix factorization process. Each user and item is associated with a vector of latent factors, where the interaction between a user $u \in \mathcal{U}$ and an item $i \in \mathcal{I}$ can be approximated as the inner product of their respective latent factor vectors

$$\hat{r}_{ui} = \mathbf{p}_u^T \mathbf{q}_i,$$

where $\mathbf{p}_u \in \mathbb{R}^k$ is the latent factor vector for user u , $\mathbf{q}_i \in \mathbb{R}^k$ is the latent factor vector for item i , and k is the number of latent factors. The latent factors capture hidden patterns and preferences that are not explicitly observed in the user-item matrix.

Therefore, considering this approximation, the rating matrix $\mathbf{R}$ can be expressed as the product of two low-rank matrices,

$$\mathbf{R} \approx \mathbf{U}\mathbf{V}^T,$$

where $\mathbf{U} \in \mathbb{R}^{m \times k}$ is the user latent factor matrix and $\mathbf{V} \in \mathbb{R}^{n \times k}$ is the item latent factor matrix. Each row $\mathbf{U}_u$ in $\mathbf{U}$ encodes the latent preferences of user u , while each row $\mathbf{V}_i$ in $\mathbf{V}$ captures the latent attributes of item i , both represented in a shared latent space of dimension k . With this formulation, the missing entries in the rating matrix can be estimated by computing the inner product between the corresponding latent vectors

$$\hat{r}_{ui} = \mathbf{U}_u \mathbf{V}_i^T.$$

In this framework, the objective is to find representations of the matrices $\mathbf{U}$ and $\mathbf{V}$ such that the predicted ratings $\hat{r}_{ui} = \mathbf{U}_u \mathbf{V}_i^T$ closely approximate the true observed values r_{ui} . This is typically achieved by minimizing a loss function that quantifies the discrepancy between the predicted and actual ratings. In our case, we adopt the **mean squared error** (MSE) as the optimization criterion, which is defined over the set of known entries $\mathcal{K} \subseteq U \times I$ as

$$\text{MSE} = \frac{1}{|\mathcal{K}|} \sum_{(u,i) \in \mathcal{K}} (r_{ui} - \hat{r}_{ui})^2.$$

2.1.2. Ranking-Based Recommendation (Top-k Recommendation)

The second widely used approach in recommender systems is known as **ranking-based recommendation**. In contrast to prediction-based methods, which aim to estimate the exact rating a user would give to an item, ranking-based approaches focus solely on identifying and ordering the most relevant items for each user. In this framework, the system

does not need to predict precise rating values; instead, it seeks to determine which items are relatively more preferred.

Definition 2.1.3. Let $\mathcal{U}$ be the set of users and $\mathcal{I}$ the set of items. A ranking-based recommendation system aims to learn a scoring function

$$s : \mathcal{U} \times \mathcal{I} \rightarrow \mathbb{R}$$

that assigns a **relevance score** $s(u, i)$ to each item $i \in \mathcal{I}$ for a given user $u \in \mathcal{U}$. The goal is not to predict an exact rating, but rather to induce a preference ordering over items such that, for any pair of items $i, j \in \mathcal{I}$, $s(u, i) > s(u, j)$ implies that item i is more relevant to user u than item j .

Given this definition of scoring function, the system recommends the top- k items with the highest scores

$$\text{Top-}k(u) = \arg \max_{i \in \mathcal{I}} \text{top-}k \ s(u, i).$$

Remark. It is important to note that transforming a prediction-based system into a ranking-based system is relatively straightforward. Once we have the predicted ratings for different items for a given user u , we can simply sort these predictions in descending order. By doing so, we can generate the ranking of items based on their predicted relevance for the user.

Continuing with the theoretical objectives of recommender systems, the primary goal would be to find the model that best approximates the ratings given by users. From a more practical perspective, however, the ultimate business objective of these systems is often to **increase user engagement and sales**. This is achieved through a combination of more specific operational goals. These operational goals not only focus on improving the accuracy of the system but also aim to reflect the overall user satisfaction. This includes measuring various characteristics that, while not easily quantifiable in a technical sense, are essential for enhancing user experience with the recommender system. These operational goals can be divided into four main branches, each addressing a different aspect of the recommender system's impact on user experience and business outcomes. These are Relevance, Novelty, Serendipity and Diversity.

Definition 2.1.4. (*Relevance*) The relevance of recommendations reflects how much users actually like the recommended items. We compute it as the average of the true ratings (when available) for the top- N recommendations,

$$\text{Relevance}(u) = \frac{1}{|R_u|} \sum_{i \in R_u} r_{ui}, \quad (2.1)$$

where R_u is the set of top- N items recommended to user u for which ground truth ratings r_{ui} are available.

Relevance measures how much users actually like the recommended items, computed as the average of true ratings for recommended items. In our practical case, ratings range from 0 to 5, so we aim for this metric to be as high as possible, ideally close to 5, indicating that the recommendations are highly relevant to users.

Definition 2.1.5. (*Novelty*) A recommendation system is more effective when it introduces users to items they have not encountered before. Recommending only well-known or previously consumed items reduces the added value of the system. It is calculated as

$$\text{Novelty}(u) = \frac{1}{|R_u|} \sum_{i \in R_u} -\log_2 P(i), \quad (2.2)$$

where $P(i)$ is the popularity of item i , typically estimated as the proportion of users who have interacted with i in the training data. Higher values indicate less popular (thus more novel) items.

Novelty assesses the system's ability to recommend less popular or previously unknown items to users. Calculated using the inverse logarithm of item popularity. A high novelty score is desired, meaning the system presents fresh, non-trivial content that adds value to the user experience.

Definition 2.1.6. (*Serendipity*) While novelty ensures users discover new items, serendipity goes further by suggesting unexpected yet appealing options. A recommendation is considered serendipitous when it pleasantly surprises the user, potentially uncovering latent interests they were not actively seeking. Serendipity is computed as

$$\text{Serendipity}(u) = \frac{1}{|R_u|} \sum_{i \in R_u} (1 - \text{Pred}_{\text{baseline}}(u, i)) \cdot \mathbb{I}[r_{ui} \geq \tau], \quad (2.3)$$

where $\text{Pred}_{\text{baseline}}(u, i)$ is the score predicted by a non-personalized or popularity-based baseline recommender, and r_{ui} is the true rating. It captures how much better the model performed compared to an obvious prediction.

Serendipity captures the ability to pleasantly surprise users by recommending unexpected yet appealing items, by comparing model predictions against a baseline popularity model. We want serendipity to be high, reflecting that the system uncovers latent user interests beyond obvious suggestions.

Definition 2.1.7. (*Diversity*) A well-balanced recommendation list includes a **variety of item types**, reducing the risk of redundancy. By increasing diversity, recommender systems enhance user engagement and prevent the experience from becoming monotonous. It is calculated as

$$\text{Diversity}(u) = \frac{2}{|R_u|(|R_u| - 1)} \sum_{\substack{i, j \in R_u \\ i \neq j}} (1 - \text{Sim}(i, j)), \quad (2.4)$$

where $\text{Sim}(i, j)$ is a similarity measure (e.g., cosine similarity) between items i and j . The higher the dissimilarity, the higher the diversity.

Diversity measures the variety within the recommendation list by evaluating the dissimilarity between recommended items. The goal is for diversity to be high (close to 1), which indicates recommendations cover different item types or categories, keeping users engaged and avoiding monotony.

Beyond these primary goals, recommender systems also contribute to broader user satisfaction and business insights. Personalized recommendations can increase customer loyalty, while analyzing user preferences helps businesses refine their offerings. Additionally, providing **explanations for recommendations** enhances transparency and trust in the system.

As recommender systems continue to evolve, their objectives extend beyond traditional product recommendations. Platforms like social networks leverage recommendation techniques to suggest **connections, content, and experiences**, demonstrating the versatility and impact of these systems in diverse domains.

2.2. Reality in Recommender Systems

The theoretical idea of predicting ratings based on these mathematical models is straightforward to understand, but as we move closer to real-world scenarios, significant challenges arise. In particular, the recommendation matrix R , which represents the interactions between m users and n items, is generally highly sparse. This sparsity arises because only a small subset of users interact with or provide feedback on items, leaving most of the matrix entries unspecified.

The problem of sparsity introduces the first **complication** and peculiar aspect of recommendation systems. Unlike traditional machine learning tasks, where the dataset is split by distinct users or items for training and testing, in recommendation systems, the train-test division is not done based on separating users. Instead, **each individual entry in the matrix will either belong to the training set or the test set**, meaning that each entry is exclusively assigned to one partition, and **both users and items can appear in either the training and testing sets**. This presents a unique challenge in how the data is managed and evaluated.

It is also worth noting that the recommendation systems we will present are foundational models. In modern applications, these systems have been significantly extended and generalized to address more complex needs. For example, demographic recommender systems take into account user characteristics such as age or location, while hybrid methods combine multiple recommendation approaches to enhance performance and address the limitations of individual algorithms. Additionally, ensemble-based systems, a common technique in advanced recommender systems, aggregate predictions from different

models to reduce variance and bias, leading to more robust and accurate recommendations.

2.3. Methodological Approach

As mentioned previously, in this work, we will follow the framework presented in the book *Recommender Systems: The Textbook* (Aggarwal, 2016). The general assumption is that we are working with discrete ratings, primarily binary in nature, although we will delve deeper into specific aspects when necessary. We will also study the challenge of sparsity of the recommendation matrix, along with the long-tail distribution, where a small set of popular items dominate user interactions. These issues require careful handling to ensure both accuracy and diversity in recommendations.

In the following sections, we will explore the main types of recommender systems, examining their structure, various algorithms, and the advantages and disadvantages of each approach. It is important to note that within each category of recommender systems, there is a broad range of algorithms and methods, each offering unique perspectives and functionalities. While it is not feasible to cover every individual algorithm or technique in detail, we will focus on the fundamental concepts that underlie these systems. This will provide a solid foundation for understanding how recommender systems function.

3. COLLABORATIVE FILTERING

Collaborative filtering is one of the most widely used approaches in recommender systems. It works by **leveraging user-item interaction data**, such as ratings or purchase history, to predict what a user might like based on the preferences of similar users. Collaborative filtering can be divided into two main categories: **neighborhood-based** and **model-based**.

1. Neighborhood-based collaborative filtering relies on finding similarities between users or items. In this method, the system identifies users with similar preferences to the target user and recommends items that those similar users have liked. This approach is computationally simple but can struggle with scalability and sparse data. we follow the methodology and formalization outlined in Section 2 of *Recommender Systems: The Textbook* by Aggarwal, 2016, which provides a rigorous treatment of neighborhood-based collaborative filtering techniques, including similarity computation and prediction strategies.

2. Model-based collaborative filtering, on the other hand, involves building a predictive model based on the user-item interaction matrix. Various algorithms, such as matrix factorization, are used to infer hidden patterns in the data, allowing the system to make recommendations by learning from the structure of the user-item interactions. Model-based methods tend to handle scalability better and are often more accurate in large, sparse datasets. In this second Section on collaborative filtering, we follow the framework presented in Section 3 of *Recommender Systems: The Textbook* by Aggarwal, 2016, which provides a detailed exposition of model-based approaches, including matrix factorization, probabilistic models, and optimization techniques.

Both approaches have their strengths and weaknesses, and the choice between them depends on the specific context and requirements of the recommender system.

3.1. Neighborhood-Based Collaborative Filtering

Neighborhood-based collaborative filtering algorithms, also known as **memory-based algorithms**, were among the earliest techniques for collaborative filtering. These methods are built on the principle that similar users exhibit similar rating patterns, and similar items tend to receive similar ratings. Depending on how similarity is defined and utilized, we can distinguish between two main types of collaborative filtering methods: **user-based** and **item-based** neighborhood models.

1. User-based collaborative filtering: The objective is to predict the rating that a user u would assign to an item i . To achieve this, we first identify a set of users $\mathcal{L}(u) \subset \mathcal{U}$ who exhibit similar rating behavior to user u . The predicted rating $\hat{r}_{ui}$ is then computed as a

weighted average of the ratings given to item i by users in $\mathcal{L}(u)$, where the weights are determined by a similarity function between each user $v \in \mathcal{L}(u)$ and the target user u .

2. Item-based collaborative filtering: As before, the goal is to predict the rating that a user u would assign to an item i . To do so, a set of items $\mathcal{S}(i) \subset \mathcal{I}$ that are most similar to item i is first identified. The predicted rating $\hat{r}_{ui}$ is then obtained as a weighted average of the ratings that user u has given to items in $\mathcal{S}(i)$, where the weights reflect the similarity between each item $j \in \mathcal{S}(i)$ and the target item i .

The key distinction between user-based and item-based collaborative filtering is that user-based filtering predicts ratings using ratings from similar users, while item-based filtering relies on a user's own ratings for similar items. In the former, neighborhoods are defined by similarities between users (rows in the ratings matrix), while in the latter, neighborhoods are defined by similarities between items (columns in the ratings matrix). These methods offer complementary approaches with different types of recommendations.

3.1.1. User-Based Neighborhood Models

User-based neighborhood methods aim to identify similar users to the target user for whom rating predictions are being made. The first step in these methods is to define **a similarity function** that measures how similar one user is to another based on their specified ratings. This is crucial for determining the neighborhood of the target user.

Definition 3.1.1. *A similarity function is a mapping*

$$Sim : \mathcal{X} \times \mathcal{X} \rightarrow \mathbb{R},$$

such that for any two entities $x, y \in \mathcal{X}$, $Sim(x, y)$ quantifies the degree of similarity between x and y . In the context of collaborative filtering, $\mathcal{X}$ typically corresponds to the set of users $\mathcal{U}$ or items $\mathcal{I}$. The function is often designed so that higher values of $Sim(x, y)$ indicate greater similarity.

Let $R \in \mathbb{R}^{m \times n}$ denote the user item ratings matrix, where m is the total number of users and n is the total number of items. Each entry $r_{uj} \in R$ represents the explicit feedback (such as a rating) provided by user $u \in \mathcal{U}$ for item $j \in \mathcal{I}$. Since users generally interact with only a limited portion of the available items, the matrix R tends to be sparse, meaning that a large proportion of its entries are unobserved or missing.

We define $\mathcal{I}_u \subseteq \mathcal{I}$ as the set of items for which user u has provided a rating, that is,

$$\mathcal{I}_u = \{j \in \mathcal{I} \mid r_{uj} \text{ is observed}\}.$$

Similarly, for a second user v , we define $\mathcal{I}_v$ as the set of items rated by user v . The intersection

$$\mathcal{I}_{uv} = \mathcal{I}_u \cap \mathcal{I}_v$$

corresponds to the set of co-rated items, meaning those items that both users u and v have rated. These shared ratings are essential for computing user similarity, as they provide a common set of observations upon which comparisons can be made.

There are cases where $\mathcal{I}_{uv} = \emptyset$, that is, if there are no overlapping ratings between users u and v , then it is not possible to directly calculate a similarity score using conventional techniques. This limitation, driven by the sparsity of the data, presents a major challenge in collaborative filtering, particularly in cases involving new users with few interactions. On the other hand, when $\mathcal{I}_{uv} \neq \emptyset$, the set of co-rated items enables the use of similarity functions such as cosine similarity or Pearson correlation, to assess the degree of alignment between the preferences of users u and v .

In order to ensure that ratings are considered relative to each user's individual preferences, it is necessary to compute the mean rating μ_u for each user. The mean rating represents the average of the ratings given by the user, and is computed as

$$\mu_u = \frac{\sum_{k \in I_u} r_{uk}}{|I_u|}, \quad (3.1)$$

where r_{uk} denotes the rating provided by user u for item k , and $|I_u|$ is the total number of items rated by user u . This mean rating serves as a baseline, allowing for the comparison of ratings in relation to the user's typical rating behavior.

Once the mean ratings for each user have been computed, the similarity between users u and v is calculated. In the literature, there exist a wide variety of similarity functions, each with its own advantages and unique characteristics. These functions are designed to measure how closely the preferences of two users align. In this work, we will focus on presenting the most commonly used similarity functions, such as the Pearson correlation and raw cosine similarity, which are frequently applied in collaborative filtering algorithms due to their simplicity and effectiveness.

Definition 3.1.2. *Mathematically, the **Pearson correlation** between two users u and v , $Pearson(u, v)$, is defined as*

$$Sim(u, v) = Pearson(u, v) = \frac{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)(r_{vk} - \mu_v)}{\sqrt{\sum_{k \in I_u \cap I_v} (r_{uk} - \mu_u)^2} \sqrt{\sum_{k \in I_u \cap I_v} (r_{vk} - \mu_v)^2}}, \quad (3.2)$$

where r_{uk} and r_{vk} represent the ratings provided by users u and v for item k , and μ_u and μ_v denote the mean ratings of users u and v , respectively.

The Pearson correlation ranges from -1 to 1 , where 1 indicates a perfect positive linear relationship, -1 indicates a perfect negative linear relationship, and 0 indicates no linear relationship.

Note: It is essential to note that, in the theoretical definition of the mean rating μ_u , it is typically assumed that only the items rated by both users u and v are considered when

computing the deviation from the mean for the similarity calculation. Specifically, the mean rating for user u would be computed as

$$\mu_u = \frac{\sum_{k \in I_u \cap I_v} r_{uk}}{|I_u \cap I_v|},$$

where $I_u \cap I_v$ represents the intersection of the set of items rated by users u and v , and r_{uk} is the rating given by user u to item k . However, a more computationally efficient and commonly employed approach is to compute μ_u independently for each user u , without conditioning on the specific user v with whom the Pearson similarity is being evaluated. This simplified method prevents potential issues that may arise when users u and v share a minimal overlap, such as a single co-rated item. In this approach, the mean rating for user u is computed as in Equation (3.1).

Once the pairwise user similarities have been computed, the next step is to estimate the rating of user u for item j by aggregating the ratings of similar users through a weighted average. A naive approach would be to compute a direct weighted average of raw ratings

$$\hat{r}_{uj} = \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot r_{vj}}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|},$$

where $P_u(j) \subseteq \mathcal{U}$ denotes the set of the k most similar users to u who have rated item j . However, this formulation assumes that all users rate items on a comparable scale, which is rarely the case in practice.

To address user-specific rating biases, a more effective strategy is to **mean-center** the ratings of each user prior to aggregation. This involves subtracting the individual user's average rating μ_u from each of their observed ratings. The mean-centered rating of user u for item j is defined as

$$s_{uj} = r_{uj} - \mu_u, \quad \forall u \in \{1, \dots, m\}, \quad \forall j \in \mathcal{I}_u.$$

The prediction for user u on item j , $\hat{r}_{uj}$, is then obtained by computing a weighted average over the mean-centered ratings s_{vj} of the peer users, followed by reintroducing the target user's mean rating μ_u to map the prediction back to the original rating scale,

$$\hat{r}_{uj} = \mu_u + \frac{\sum_{v \in P_u(j)} \text{Sim}(u, v) \cdot (r_{vj} - \mu_v)}{\sum_{v \in P_u(j)} |\text{Sim}(u, v)|}.$$

This formulation adjusts for inter-user rating scale differences, ensuring that similarity-weighted deviations from individual rating baselines are aggregated rather than raw ratings. In practice, $P_u(j)$ may exclude users with weak or negative similarity scores to enhance the reliability of predictions. Such normalization is essential for improving the robustness and interpretability of neighborhood-based collaborative filtering methods.

Variants of the method

As previously discussed, once user mean ratings have been computed to normalize individual rating scales, the next step involves selecting an appropriate similarity function. While the Pearson correlation coefficient is widely used due to its ability to account for individual rating tendencies, other similarity functions are frequently employed in neighborhood-based collaborative filtering methods. These alternatives differ from the Pearson correlation in how they handle normalization and rating biases.

Definition 3.1.3. (*Raw Cosine similarity*) Given two users u and v , the raw cosine similarity is defined over the set of items they have both rated, denoted by $I_u \cap I_v$, and is computed as

$$Sim_{cos}(u, v) = \frac{\sum_{k \in I_u \cap I_v} r_{uk} \cdot r_{vk}}{\sqrt{\sum_{k \in I_u \cap I_v} r_{uk}^2} \cdot \sqrt{\sum_{k \in I_u \cap I_v} r_{vk}^2}}. \quad (3.3)$$

This metric evaluates the cosine of the angle between the two users' rating vectors, yielding values in the range $[0, 1]$ when all ratings are non-negative. A value closer to 1 indicates a smaller angle and hence greater similarity in rating behavior.

The raw cosine similarity does not account for user-specific rating biases or systematic differences in rating scales. As a result, it may yield suboptimal similarity estimates in scenarios where users exhibit consistent rating patterns (e.g., one user persistently assigns higher or lower ratings than another regardless of item preference).

Definition 3.1.4. (*Significance weighting*) Let $Sim(u, v)$ be a similarity measure between users u and v , and let $I_u \cap I_v$ denote the set of items rated by both users. The significance-weighted similarity, denoted $DiscountedSim(u, v)$, is defined as

$$DiscountedSim(u, v) = Sim(u, v) \cdot \frac{\min(|I_u \cap I_v|, \beta)}{\beta}, \quad (3.4)$$

where $\beta \in \mathbb{N}$ is a significance threshold parameter. This formulation introduces a linear discounting factor to penalize similarity estimates derived from a small number of co-rated items, thereby increasing the statistical robustness of the similarity computation.

It is important to highlight that generalizations and variations are not limited to the similarity computation phase; they can also be applied to the prediction step in neighborhood-based collaborative filtering. Various normalization and transformation techniques exist that aim to improve the robustness and interpretability of rating predictions, particularly when dealing with users who exhibit different rating behaviors or variances. One such technique is **Z-score normalization**, which replaces the standard mean-centering approach by standardizing each user's ratings.

Definition 3.1.5. (Z-score normalization) The standardized rating z_{uj} of user u for item j is defined as

$$z_{uj} = \frac{r_{uj} - \mu_u}{\sigma_u},$$

where μ_u is the mean and σ_u the standard deviation of ratings given by user u . This transformation rescales the ratings to have zero mean and unit variance, allowing better comparability across users with different rating ranges or dispersions.

Additionally, the similarity measure itself can be modified by applying an **amplification exponent** $\alpha > 1$ to the original similarity score.

Definition 3.1.6. The adjusted similarity based on the Pearson correlation (Eq. (3.2)) is computed as

$$\text{Sim}(u, v) = \text{Pearson}(u, v)^\alpha.$$

This exponentiation increases the influence of highly similar neighbors while suppressing the contribution of weakly similar ones, thereby sharpening the neighborhood definition. While these advanced techniques enhance flexibility and control, we focus our analysis on the core principles and do not explore these variants in further detail.

3.1.2. Item-Based Neighborhood Models

Similar to user-based methods, item-based neighborhood models aim to predict unknown ratings by leveraging similarity functions. The key distinction lies in the perspective: whereas user-based models compute similarities between users, item-based models focus on the relationships between items. In this case, peer groups are constructed around items rather than users, meaning that the similarity is calculated between items (i.e., columns in the user-item rating matrix).

Before computing these similarities, the rating matrix is typically mean centered along the user dimension. For each user, their average rating is subtracted from each of their individual item ratings. This normalization step adjusts for user-specific rating tendencies, producing the mean centered ratings s_{uj} and ensuring that the similarity computation reflects relative preferences rather than absolute rating scales.

In this case, we will adapt the previously mentioned Cosine similarity (see Eq. (3.3)) to an item-based setting by incorporating a normalization step that corrects for user-level biases. The result is known as the **Adjusted Cosine Similarity**, a metric specifically designed for item-based collaborative filtering, where the goal is to compute similarities between item vectors.

Definition 3.1.7. (Adjusted Cosine) Let U_i denote the set of users who have rated item i . The mean-centered rating of user u for item i is given by

$$s_{ui} = r_{ui} - \mu_u,$$

where μ_u is the average rating provided by user u . Using these adjusted ratings, the similarity between items i and j is defined as

$$\text{AdjustedCosine}(i, j) = \frac{\sum_{u \in U_i \cap U_j} s_{ui} \cdot s_{uj}}{\sqrt{\sum_{u \in U_i \cap U_j} s_{ui}^2} \cdot \sqrt{\sum_{u \in U_i \cap U_j} s_{uj}^2}}.$$

The summations are restricted to $U_i \cap U_j$, the set of users who have rated both items i and j . By focusing on deviations from each user's mean, the adjusted cosine similarity mitigates the influence of varying rating scales and provides a more robust measure of item-item similarity in collaborative filtering settings.

This adjusted cosine measure typically performs better than the Pearson correlation coefficient when applied to columns of the ratings matrix due to its ability to handle mean-centered data effectively.

Once we have computed the similarity between the items using a metric such as *Adjusted Cosine Similarity*, the next step is to make the predictions for the ratings. To predict the rating $\hat{r}_{ut}$ that user u would assign to the target item t , the model identifies the k most similar items to t that have been rated by user u . This set is denoted as $Q_t(u)$. The predicted rating is then calculated as a weighted average of the ratings that the user has given to these similar items, where the weights are determined by the adjusted cosine similarity

$$\hat{r}_{ut} = \frac{\sum_{j \in Q_t(u)} \text{AdjustedCosine}(j, t) \cdot r_{uj}}{\sum_{j \in Q_t(u)} |\text{AdjustedCosine}(j, t)|}.$$

This approach leverages the user's own ratings on items similar to the target item to make predictions. For example, in a movie recommendation system, the peer group for an action movie would typically consist of other action movies. The user's previous ratings for these movies serve as strong indicators of their preferences for the target movie.

Due to the similarities between user-based and item-based methods, many of the variants discussed for user-based models, such as significance weighting or similarity amplification, can also be applied to item-based algorithms. These adaptations enhance the model's flexibility and predictive accuracy while maintaining the core principles.

3.1.3. Advanced methodology for Neighborhood Models

As mentioned in Section 2.2, in practical applications, more advanced models and techniques are employed to address challenges such as data sparsity. In this section, we will briefly explore some generalizations and advanced concepts that can enhance the effectiveness of recommendation systems. While we will not go into great detail on these

topics, they merit a concise discussion to offer insight into their potential for improving recommendations.

The basic neighborhood-based methods discussed earlier have notable strengths, including their simplicity, interpretability, and ease of implementation. User-based and item-based approaches are intuitive and often allow for clear justification of recommendations, especially in item-based methods. Additionally, these systems are relatively stable when new users or items are added. However, they also have significant limitations.

One key challenge is the **offline computation** required in user-based methods. This process involves precomputing similarities between all users in order to facilitate real-time recommendations. Given a dataset with m users, the offline phase requires at least $O(m^2)$ time and space, which can become impractical in large-scale environments where m is in the order of millions. Furthermore, sparsity in the ratings matrix can lead to reduced coverage and unreliable similarity calculations when few items are mutually rated. The computational cost and data sparsity must be addressed when designing scalable recommendation systems.

A promising perspective is to view user-based and item-based methods in a unified framework. These approaches are almost identical once the rows of the ratings matrix are mean-centered. For example, the Pearson correlation coefficient (Eq. (3.2)) between rows becomes equivalent to cosine similarity in such cases. This unification allows one to combine similarity information along rows and columns simultaneously, rather than ignoring either dimension. For a target entry (u, j) in the ratings matrix, the most similar entries can be determined using a combination of similarities across rows and columns, such as the sum of cosine similarities. Predictions can then be made using a weighted combination of the ratings associated with these similar entries.

In addition to this unified perspective, several advanced topics further enhance recommendation systems by addressing their inherent limitations. In this section, we will briefly discuss concepts such as **Clustering in Neighborhood-Based Methods** and **Dimensionality Reduction**. These topics provide alternative perspectives and techniques that improve the scalability, robustness, and predictive accuracy of recommendation systems.

Clustering in Neighborhood-Based Methods

Clustering-based methods provide a solution to the scalability challenges of neighborhood-based approaches by replacing the computationally expensive offline nearest-neighbor phase with an offline clustering process. Instead of calculating individual peer groups for each user or item, clustering groups similar users or items into representative clusters, significantly reducing the computational complexity from $O(m^2)$ operations to a more manageable scale.

Once clusters are formed, rating predictions follow a similar process to standard neighbor-

hood methods, but similarity calculations are restricted to the members of each cluster, making the approach more efficient. However, this efficiency comes at a trade-off, as the quality of the identified nearest neighbors is reduced. The granularity of clustering impacts this trade-off: finer clusters improve efficiency but can slightly reduce prediction accuracy. To handle incomplete data, clustering techniques such as k-means can be adapted, iteratively assigning users or items to cluster representatives based on a similarity function like Euclidean or Manhattan distance. This distance metric serves as a key component of similarity-based clustering, enabling the system to cluster items or users based on their proximity in the rating space. As data sparsity increases, adjustments are made by considering only the available ratings, ensuring robust clustering even in the presence of missing data.

This clustering approach extends seamlessly to both user-based and item-based methods, where clustering can be applied to either rows (users) or columns (items) of the ratings matrix. By reducing computational demands while maintaining reasonable prediction quality, clustering offers a practical alternative for large-scale recommendation systems, addressing both efficiency and the need for similarity-based grouping.

Dimensionality Reduction in Neighborhood Methods

Dimensionality reduction techniques enhance neighborhood-based methods by improving computational efficiency and prediction robustness. Sparse rating matrices make direct similarity computation challenging, but dimensionality reduction transforms data into a lower-dimensional latent space, creating a dense representation that facilitates similarity estimation. These are referred to as **latent factor models**.

In recommender systems, dimensionality reduction can be applied by compressing either user or item dimensions into latent factors, or by simultaneously learning latent representations for both users and items. The first method is more applicable to neighborhood models, while the second extends beyond them.

For user-based collaborative filtering, **Singular Value Decomposition (SVD)** or **Principal Component Analysis (PCA)** are used to decompose the ratings matrix R into a compact representation R' , where users are mapped to a d -dimensional latent space ($d \ll n$). This reduces sparsity issues and computational demands, making similarity computation more efficient and scalable.

This approach **aligns more with model-based methods**, as it learns a compact predictive model (latent factors) rather than relying on pre-computed neighborhood similarities. Dimensionality reduction enables the system to predict ratings without needing direct neighborhood-based similarity calculations, offering a more efficient and scalable alternative for large datasets.

3.2. Model-Based Collaborative Filtering

Model-based collaborative filtering differs from neighborhood-based methods in that it builds a structured model from the data rather than relying on instance-based similarity computations. While neighborhood-based approaches share similarities with **k-nearest neighbor classifiers**, model-based methods resemble **traditional machine learning algorithms**, such as decision trees, regression models, and neural networks, where the training phase is explicitly separated from the prediction phase.

One key advantage of model-based approaches is their ability to **generalize patterns** across the dataset, improving prediction accuracy and mitigating sparsity issues. Unlike neighborhood-based methods, where similarity computations must be performed at query time, model-based techniques precompute representations, allowing for efficient recommendation generation. Additionally, these models tend to be more space-efficient, avoiding the quadratic complexity of storing pairwise similarities.

The relationship between collaborative filtering and classification problems provides useful insights for designing model-based recommender systems. Unlike standard classification, where a clear distinction exists between independent and dependent variables, collaborative filtering treats all entries as potentially missing values that need to be predicted. This generalization enables the adaptation of machine learning techniques for recommendation tasks, including ensemble methods, latent factor models, and optimization-based frameworks, which improve scalability and robustness.

3.2.1. Decision Trees and Their Extension to Collaborative Filtering

Among model-based recommendation techniques, tree-based methods like decision trees and regression trees are widely used due to their interpretability and flexibility in handling both categorical and continuous variables. In traditional settings, **decision trees** are applied to classification tasks, while **regression trees** handle prediction of numerical outcomes. These models iteratively split the data based on feature values to form a tree structure where each path corresponds to a sequence of decision rules and leaf nodes provide the final prediction.

The choice of split at each node is guided by impurity measures such as the **Gini index** or **entropy**.

Definition 3.2.1. (*Gini*) The Gini index is an impurity measure used to evaluate splits in decision trees. It quantifies the likelihood of an incorrect classification by randomly assigning labels according to the class distribution. It is defined as

$$Gini(S) = 1 - \sum_{i=1}^C p_i^2,$$

where p_i is the proportion of instances of class i in subset S , and C is the total number of classes. Lower Gini values indicate purer splits.

Definition 3.2.2. (Entropy) Entropy measures the amount of disorder or uncertainty in the class distribution of a dataset. It is commonly used in decision trees to select features that provide the highest information gain. The entropy of a subset S is given by:

$$\text{Entropy}(S) = - \sum_{i=1}^C p_i \log_2 p_i,$$

where p_i is the proportion of instances in class i and C is the total number of classes. An entropy of zero indicates a perfectly pure node.

When applied to recommender systems, however, several challenges arise. Unlike traditional classification tasks, user-item rating matrices are highly sparse and lack a fixed dependent variable. Instead of predicting a single target variable, recommender systems aim to estimate user preferences across many items. To address this, a common strategy is to train one decision or regression tree per item. That is, for an $m \times n$ user-item matrix (with m users and n items), n trees are built, each one learning to predict the rating for a specific item using the user's ratings on other items as input features.

Note: A major difficulty in this approach is the presence of missing ratings in the input features. If a particular item is used as a splitting attribute and a user has not rated it, the system must handle this absence. Possible solutions include assigning such users to both child nodes (increasing computational cost), or applying dimensionality reduction techniques such as PCA to project the data into a lower-dimensional, dense space where all features are defined.

3.2.2. Naive Bayes in Collaborative Filtering

Among the various probabilistic approaches to recommender systems, Naive Bayes provides a simple yet effective framework for predicting user preferences. This method models the rating prediction task by applying Bayes' theorem under the assumption of conditional independence between features. Consider a user-item rating matrix $R \in \mathbb{R}^{m \times n}$, where each entry r_{uj} represents the rating given by user u to item j , and many entries are missing. The goal is to infer these unobserved ratings by estimating the probability distribution over possible rating values, conditioned on the available user-item interactions. This probabilistic framework is particularly suited when ratings are treated as discrete categorical variables.

Definition 3.2.3. (Naive Bayes estimation) Let $v_s \in \{v_1, v_2, \dots, v_l\}$ denote a possible rating value. The goal is to compute the posterior probability of user u assigning rating v_s to item j , given their observed ratings in the set of items $I_u \subset \{1, \dots, n\} \setminus \{j\}$. Applying Bayes' theorem, the conditional probability is given by

$$P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u}) = \frac{P(r_{uj} = v_s) \cdot P(\{r_{uk}\}_{k \in I_u} \mid r_{uj} = v_s)}{P(\{r_{uk}\}_{k \in I_u})}.$$

Since the denominator does not depend on the rating value v_s , it acts as a normalizing constant. Thus, the expression simplifies to

$$P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u}) \propto P(r_{uj} = v_s) \cdot P(\{r_{uk}\}_{k \in I_u} \mid r_{uj} = v_s).$$

The prior probability $P(r_{uj} = v_s)$ is estimated as the empirical frequency of rating v_s for item j among all users. Assuming conditional independence among the observed ratings given the unobserved rating, the likelihood term factorizes as follows

$$P(\{r_{uk}\}_{k \in I_u} \mid r_{uj} = v_s) = \prod_{k \in I_u} P(r_{uk} \mid r_{uj} = v_s).$$

Substituting these into the posterior expression, we obtain

$$P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u}) \propto P(r_{uj} = v_s) \cdot \prod_{k \in I_u} P(r_{uk} \mid r_{uj} = v_s).$$

Once the posterior distribution over possible ratings $P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u})$ has been computed, there are two principal strategies for estimating the unknown rating r_{uj} . The first selects the rating with the highest posterior probability, while the second computes a weighted average over all possible ratings using the posterior as weights. Each method has its own advantages depending on whether the rating scale is categorical or ordinal.

1. Maximum Likelihood Estimation

The first estimation strategy seeks to identify the most probable rating value that user u would assign to item j . For each possible rating level $v_s \in \{v_1, v_2, \dots, v_l\}$, the posterior probability is evaluated, and the value that maximizes this probability is selected.

$$\hat{r}_{uj} = \arg \max_{v_s} P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u}).$$

Substituting the expression derived from Bayes' theorem, the estimation becomes

$$\hat{r}_{uj} = \arg \max_{v_s} P(r_{uj} = v_s) \prod_{k \in I_u} P(r_{uk} \mid r_{uj} = v_s).$$

This method selects the rating v_s that has the highest posterior probability given the observed ratings of user u . However, it **treats ratings purely as categorical values**, disregarding any natural ordering among them. This approach is most effective when the number of possible ratings is small.

2. Weighted Average Estimation

Alternatively, instead of selecting the most probable discrete rating, the prediction can be obtained by computing a weighted average over all possible rating levels, where the posterior probabilities serve as weights.

$$\hat{r}_{uj} = \sum_{s=1}^l v_s P(r_{uj} = v_s \mid \{r_{uk}\}_{k \in I_u}).$$

By applying Bayes' theorem, the posterior probabilities can be expressed in terms of prior and likelihood terms, yielding the following formulation:

$$\hat{r}_{uj} = \frac{\sum_{s=1}^l v_s P(r_{uj} = v_s) \prod_{k \in I_u} P(r_{uk} \mid r_{uj} = v_s)}{\sum_{s=1}^l P(r_{uj} = v_s) \prod_{k \in I_u} P(r_{uk} \mid r_{uj} = v_s)}.$$

This weighted average estimation is particularly advantageous when the rating scale exhibits ordinal properties or when the rating domain is sufficiently granular. Unlike the MAP approach, it produces a continuous-valued prediction, capturing more nuanced preferences.

Note. Although the Naive Bayes model described here uses the user's ratings across different items, following an *item-based* collaborative filtering approach, the same principle can be applied from a *user-based* perspective by conditioning on ratings from other users for the same item. Combining both views often leads to improved performance as it integrates both item and user similarities.

Handling Overfitting in Naive Bayes Collaborative Filtering

Overfitting in Naive Bayes collaborative filtering arises primarily from data sparsity. When few users rate an item, probability estimates become unreliable, especially for conditional probabilities, where missing values can reduce entire expressions to zero.

To address this, Laplacian smoothing is applied by adding a parameter α to avoid zero probabilities. For prior probabilities

$$P(r_{uj} = v_s) = \frac{q_s + \alpha}{\sum_{t=1}^l q_t + l\alpha}.$$

And for conditional probabilities

$$P(r_{uk} \mid r_{uj} = v_s) = \frac{q_{k,s} + \alpha}{\sum_{t=1}^l q_{k,t} + l\alpha}.$$

This technique ensures non-zero probabilities even with limited data, improving robustness and reducing overfitting. The choice of α controls the trade-off between stability and sensitivity to observed ratings.

3.3. Latent Factor Models

Within the model based approach to collaborative filtering, latent factor models play a central role. These models exploit correlations between users and items to uncover a compact, lower dimensional representation of the user item interaction matrix. By identifying latent structures that reflect user preferences and item characteristics, they reduce redundancy and allow for the accurate estimation of missing entries using low rank approximations. This approach addresses sparsity and enhances generalization in recommendation tasks.

In this section, we focus on dimensionality reduction in the latent space and study two representative models: unconstrained matrix factorization and singular value decomposition (SVD).

3.3.1. Dimensionality Reduction and Matrix Factorization

The key advantage of latent factor models is their ability to construct a **fully specified representation from an incomplete dataset**. Since user preferences and item attributes are often correlated, missing entries can be inferred through dimensionality reduction techniques. These methods rotate the axis system to remove pairwise correlations, creating a structured and concise data representation.

Matrix factorization techniques such as Singular Value Decomposition (SVD) and Principal Component Analysis (PCA) are commonly used for dimensionality reduction.

As we discussed in Section 2.1, the user-item interaction matrix R can be approximated using matrix factorization techniques, which decompose R into lower-dimensional representations. Specifically, R is expressed as

$$\mathbf{R} \approx \mathbf{U}\mathbf{V}^T.$$

- $\mathbf{U} \in \mathbb{R}^{m \times k}$ is the **user latent factor matrix**, where each row $\mathbf{U}_u$ represents the preference profile of user u in a lower-dimensional space of size k .
- $\mathbf{V} \in \mathbb{R}^{n \times k}$ is the **item latent factor matrix**, where each row $\mathbf{V}_i$ represents the characteristics of item i in the same latent space.

The estimation of missing values is achieved through the multiplication of latent factor matrices

$$\hat{r}_{ui} = \mathbf{U}_u \mathbf{V}_i^T.$$

The goal is to adjust the parameters $\mathbf{U}$ and $\mathbf{V}$ to ensure that the predicted ratings $\hat{r}_{ui}$ align as closely as possible with the actual values r_{ui} . This refinement process involves minimizing an appropriate loss function, such as the **mean squared error** (MSE), which quantifies the discrepancy between predictions and observed ratings.

Within this framework, $\mathbf{U}$ is an $m \times k$ matrix encoding user-specific latent factors, while $\mathbf{V}$ is an $n \times k$ matrix capturing item-related latent features. The error in reconstructing the original rating matrix is measured using the squared Frobenius norm

$$\|\mathbf{R} - \mathbf{UV}^T\|_2,$$

where $\|\cdot\|_2$ represents the sum of squared differences between the entries of $\mathbf{R}$ and its low-rank approximation $\mathbf{UV}^T$. This residual matrix effectively accounts for variations in user preferences that cannot be fully expressed through the latent factor representation, encapsulating noise and unique individual behaviors.

By iteratively minimizing this error function, the model learns optimal latent representations that not only closely approximate observed ratings but also provide reliable estimates for missing values, enhancing the effectiveness of recommendations.

3.3.2. Unconstrained factorization for Fully-Specified Matrices

Unconstrained matrix factorization represents the most fundamental form of matrix decomposition, in which no restrictions are imposed on the factor matrices $\mathbf{U}$ and $\mathbf{V}$. Although often referred to as Singular Value Decomposition (SVD) in recommendation literature, this is a misnomer since SVD strictly requires the columns of $\mathbf{U}$ and $\mathbf{V}$ to be orthogonal. To avoid confusion, we distinguish unconstrained matrix factorization from true SVD.

Definition 3.3.1. (*Unconstrained optimization problem*) Given a fully observed user–item rating matrix $\mathbf{R} \in \mathbb{R}^{m \times n}$, the objective of matrix factorization is to find two low-rank matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$ such that their product approximates $\mathbf{R}$

$$\mathbf{R} \approx \mathbf{UV}^T,$$

where $k \ll \min(m, n)$ is the dimensionality of the latent space. The factor matrices $\mathbf{U}$ and $\mathbf{V}$ encode the latent representations of users and items, respectively, capturing underlying patterns in the interaction data. In the unconstrained setting, no additional restrictions (such as non-negativity or orthogonality) are imposed on the elements of $\mathbf{U}$ and $\mathbf{V}$.

This approximation can be optimized by minimizing the squared Frobenius norm of the residual matrix

$$J = \frac{1}{2} \|\mathbf{R} - \mathbf{UV}^T\|_2^2,$$

where $\|\cdot\|_2$ denotes the sum of squared differences between the entries of $\mathbf{R}$ and its low-rank approximation. The lower the value of J , the better the quality of the factorization. Gradient descent techniques can be applied to optimize $\mathbf{U}$ and $\mathbf{V}$.

In real-world recommender systems, the matrix R is often incomplete, meaning that only a subset of its entries is known. The optimization function must be adapted accordingly to consider only the observed ratings.

Definition 3.3.2. (*Incomplete matrix factorization*) Let $R \in \mathbb{R}^{m \times n}$ be a partially observed user–item rating matrix, and let $S = \{(i, j) \mid R_{ij} \text{ is observed}\}$. We aim to approximate R by the product of two low-rank matrices $\mathbf{U} \in \mathbb{R}^{m \times k}$ and $\mathbf{V} \in \mathbb{R}^{n \times k}$, such that

$$\hat{r}_{ij} = \sum_{s=1}^k U_{is} V_{js}$$

is the predicted value for entry (i, j) . The prediction error for observed entries is given by

$$e_{ij} = r_{ij} - \hat{r}_{ij} = r_{ij} - \sum_{s=1}^k U_{is} V_{js}.$$

In this setting, the unconstrained optimization problem for a partially observed matrix can be formulated as minimizing the error functional over the observed entries

$$J = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 = \frac{1}{2} \sum_{(i,j) \in S} \left(r_{ij} - \sum_{s=1}^k U_{is} V_{js} \right)^2,$$

which captures the total squared deviation between the known ratings and their low-rank approximations. Once the factor matrices $\mathbf{U}$ and $\mathbf{V}$ are learned, the entire matrix R can be reconstructed through the approximation $R \approx \mathbf{UV}^T$, providing estimates for missing entries in a single step.

The optimization of matrix factorization relies on refining the latent factor matrices so that the reconstructed data closely approximates the observed values. To achieve this, methods such as **gradient descent** and **stochastic gradient descent (SGD)** iteratively adjust the factor matrices by minimizing the error between real and predicted ratings.

1. **Gradient descent** follows a systematic approach, computing the partial derivatives of the loss function with respect to each parameter. The update rule for the factor matrices is given by

$$U_{iq} \leftarrow U_{iq} + \alpha e_{ij} V_{jq}, \quad V_{jq} \leftarrow V_{jq} + \alpha e_{ij} U_{iq}.$$

2. **Stochastic gradient descent** refines the model by adjusting parameters based on a single randomly chosen observed entry at a time. This incremental strategy makes it particularly effective for large datasets, achieving faster convergence while introducing some noise in the optimization process.

As we previously mentioned, the matrix R is not only incomplete but also highly sparse, meaning that only a small fraction of its entries are observed. This sparsity can lead to overfitting, where the model learns patterns from a limited sample of data instead of generalizing well for missing values. To mitigate this issue, **regularization** is introduced to

stabilize the learning of latent factors and prevent excessively large values in the matrices U and V . This stabilization is achieved by adding a penalty term to the optimization function

$$J = \frac{1}{2} \sum_{(i,j) \in S} e_{ij}^2 + \frac{\lambda}{2} \left(\sum_{i=1}^m \sum_{s=1}^k U_{is}^2 + \sum_{j=1}^n \sum_{s=1}^k V_{js}^2 \right),$$

where e_{ij} denotes the error corresponding to the difference between the observed and predicted values, and λ represents the regularization parameter that controls the degree of penalization applied to the model.

This formulation biases the optimization toward simpler solutions by discouraging excessively large coefficients, leading to improved generalization. Also, notice that fine-tuning λ is essential to balancing the trade-off between reducing overfitting and maintaining enough flexibility to learn meaningful user-item interactions, ultimately improving the robustness of the recommendation model.

Beyond this standard approach, various extensions exist to refine matrix factorization further. **Incremental Latent Component Training** focuses on progressively updating latent factors, allowing for dynamic adjustments as more data becomes available. **Incorporating User and Item Biases** introduces bias terms to account for systematic variations in user preferences and item popularity, improving predictive accuracy. However, these techniques are highly specialized and are applicable only in certain scenarios, so we will not delve deeper into them here as they do not necessarily apply to general cases.

3.3.3. Singular Value Decomposition in Recommender Systems

In the previous section, we examined the problem of unconstrained optimization. One approach aimed at reducing the computational cost of these methods is the use of Singular Value Decomposition (SVD) of matrices. Singular Value Decomposition is a matrix factorization technique where the columns of the factor matrices U and V are constrained to be mutually orthogonal. This orthogonality ensures that the latent dimensions are independent, allowing for geometric interpretations in scatterplots. However, the semantic interpretation of such decompositions can be challenging due to the constraints imposed by orthogonality and the presence of both positive and negative values in the latent vectors. The reduction in computational cost is primarily due to the ability of SVD to decompose the original matrix into a product of smaller matrices, enabling efficient approximation of the original data by retaining only the most significant singular values and their corresponding vectors. This dimensionality reduction leads to faster computations and less memory usage.

For a fully specified matrix R , SVD can be computed efficiently using eigendecomposition methods. A common approach is **truncated SVD**, which reduces the rank of the decomposition to k , where $k \ll \min(m, n)$, capturing the most relevant patterns while eliminating noise. The decomposition is expressed as

$$R \approx Q_k \Sigma_k P_k^T,$$

The matrices Q_k and P_k contain the top k eigenvectors of RR^T and $R^T R$, respectively, which define the principal components used in the decomposition. These eigenvectors capture essential structural relationships within the data, enabling a compact representation. The diagonal matrix Σ_k holds the square roots of the top k eigenvalues, providing the scaling factors necessary to reconstruct the original matrix in a lower-dimensional space.

The matrix P_k captures item-item correlations, while $Q_k \Sigma_k$ provides a reduced representation of users in the transformed space. These components enable dimensionality reduction and latent factor extraction. Although SVD decomposes R into three matrices, the diagonal matrix Σ_k can be absorbed into either Q_k or P_k , yielding conventional user and item factor matrices

$$U = Q_k \Sigma_k, \quad V = P_k.$$

Thus, the factorization can be expressed as $R \approx UV^T$, maintaining orthogonality constraints.

Optimization Formulation

SVD can be formulated as an optimization problem similar to the previous example. However, unlike the unconstrained case, this minimization problem imposes orthogonality conditions on the factor matrices.

$$\text{Minimize } J = \frac{1}{2} \|R - UV^T\|^2, \quad \text{s.t. } U^T U = I, \quad V^T V = I.$$

These constraints restrict the solution space compared to unconstrained matrix factorization. Interestingly, for fully specified matrices without regularization, the optimal error J is identical for both SVD and unconstrained factorization. However, when R is incomplete, unconstrained factorization typically achieves lower error on observed entries, while the generalization performance on unobserved entries may vary.

SVD is widely used in recommender systems for dimensionality reduction and latent factor extraction, enabling efficient and accurate predictions in sparse data environments.

4. CONTENT-BASED RECOMMENDER SYSTEMS

In the previous section, we focused on collaborative filtering methods, which rely on the relationships and similarities between different users to generate recommendations. In contrast, this section shifts the focus to content-based recommender systems, which generate personalized suggestions by analyzing the characteristics of items that a user has previously interacted with or rated positively. Unlike collaborative filtering methods, these systems do not rely on the preferences or behavior of other users. Instead, they operate using information about the items' attributes and the user's individual profile. The item attributes can be either structured, such as genre, brand, or price, or unstructured, such as free-text descriptions or keywords. The user profile is typically constructed from explicit feedback, like ratings, or implicit behavior, such as clicks or browsing patterns.

This approach is particularly effective when items have rich descriptive data but limited community interaction, such as in cold-start scenarios for newly introduced products. By relying solely on the user's own data, content-based methods can generate relevant recommendations even in environments with a small or inactive user base. Nonetheless, the absence of information from other users leads to some limitations. Recommendations tend to lack diversity, as they focus on suggesting items similar to those already consumed. This can reduce novelty and limit the user's exposure to unexpected but potentially interesting items. Furthermore, content-based systems struggle with new users who lack sufficient interaction history, since building a meaningful user profile requires a minimum amount of data. These systems are conceptually related to knowledge-based recommenders, which also utilize item attributes but allow users to explicitly state their preferences through interactive interfaces. In practice, many systems combine the learning-based nature of content filtering with elements of user interaction, resulting in hybrid models that aim to enhance both personalization and control.

In content-based recommender systems, the quality of recommendations depends heavily on how item characteristics and user preferences are represented. These systems often process heterogeneous and unstructured data sources, making it essential to convert the information into a standardized format. This typically involves transforming item descriptions into structured features or sets of keywords, enabling machine learning algorithms to handle diverse data formats such as text, numerical attributes, or categorical information.

A complete content-based recommendation pipeline consists of three primary phases. The first two occur offline and focus on data preparation and model training, while the final stage happens online, generating real-time recommendations.

The first phase is **preprocessing and feature extraction**, where relevant features are identified and extracted from item descriptions. Depending on the domain, these features can come from product metadata, textual content, audio signals, or other specific sources. The extracted information is usually converted into a vector-space model, representing

each item as a numerical vector. The effectiveness of the recommendation system relies significantly on selecting the most informative and discriminative features.

The second phase involves **learning user profiles**, where a personalized model is created for each user based on interactions with past items. These interactions include explicit feedback, such as ratings and purchases, and implicit behaviors, like clicks or browsing patterns. By combining these user interactions with extracted item features, classification or regression models are trained to predict interest in unseen items.

The final phase is **filtering and recommendation**, in which trained user models generate real-time predictions. Each candidate item is evaluated based on its relevance to the user's profile, and the highest-scoring items are presented as recommendations. Since this process occurs online, it must be highly optimized to ensure seamless user experiences.

Feature extraction, user modeling, and recommendations form the core of any content-based recommender system. Their adaptable design allows implementation across various domains, particularly those rich in descriptive information such as text, product catalogs, or media content. In the following sections, we will explore each of these components in detail, examining their role in the recommendation process and the methodologies used to optimize them.

4.1. Preprocessing and Feature Extraction

Content-based recommender systems rely on item descriptions and user-related data, often presented in unstructured formats. To apply machine learning techniques effectively, it is necessary to convert this information into standardized representations, typically by extracting keywords from textual content and representing them in a vector space. The initial step involves feature extraction, which identifies item attributes that are predictive of user preferences. This process is domain-specific and may involve both structured fields, such as category or price, and unstructured descriptions.

To enhance the quality of the input data, preprocessing techniques are applied, including the removal of non-informative terms, normalization of word forms, and detection of meaningful multi-word expressions. The resulting features are often weighted using the term frequency-inverse document frequency model, which balances the relevance of a term within a document with its rarity across the dataset. A damping function is typically applied to reduce the influence of excessively repeated terms.

User preferences, whether expressed as explicit ratings, behavioral signals, or textual opinions, are collected and used as target variables during the training process. These preferences allow the system to learn associations between item features and user interests.

4.2. Learning User Profiles

The learning phase of user profiles in content-based recommender systems is closely related to classification and regression modeling, depending on the nature of the available ratings. When ratings are binary (e.g., “like” or “dislike”), the problem is akin to text classification. When treated as numerical values, it resembles regression modeling. For simplicity, item descriptions are often represented as documents, though these techniques generalize well to any type of multidimensional data. Each active user is associated with a labeled training set DL , which includes the content descriptions of items they have previously rated, along with the corresponding labels. These labels may be binary, unary, or numeric. The resulting model learned from this training set forms the user profile and is used to make predictions or recommendations on a separate, unlabeled set DU of candidate items that the user has not yet interacted with.

Unlike collaborative filtering models, which often construct a global model across all users such as in matrix factorization, content-based methods learn one independent model per user. Thus, learning is entirely personalized and does not utilize ratings from other users.

The documents in DU represent items that may be recommended to the user. The system uses the user-specific model to predict either explicit ratings or to produce a ranked list of top- k recommendations. This formulation is highly similar to supervised learning problems in text processing, particularly when content is represented using methods like term frequency-inverse document frequency (tf-idf), as discussed previously.

The learning algorithms employed in content-based recommendation are largely similar to those used in collaborative filtering systems, with necessary adaptations to account for content features instead of user-item interactions. Commonly used learning methods include **k-Nearest Neighbors (k-NN)**, **Naive Bayes** or **Support Vector Machines (SVM)** and **Regression-Based Models** among many other options.

These methods, although adapted for the content-based setting, follow the same foundational principles as those discussed in the previous section on collaborative filtering, illustrating the close relationship between both approaches in terms of learning strategy.

4.2.1. Nearest Neighbor Classification

One of the simplest yet effective techniques for classification in content-based recommender systems is the *nearest neighbor classifier*. This method relies on measuring similarity between items to predict user preferences. The most commonly used similarity function in text-based scenarios is the **cosine similarity** (see Eq. (3.3)), which accounts for the direction rather than the magnitude of document vectors, making it suitable for documents of varying lengths.

Let $X = (x_1, x_2, \dots, x_d)$ and $Y = (y_1, y_2, \dots, y_d)$ be two document vectors, where each x_i and y_i represents the normalized frequency of the i -th term in documents X and Y , respectively. These frequencies are typically obtained using tf-idf weighting. The cosine similarity between X and Y is defined as

$$\text{Cosine}(X, Y) = \frac{\sum_{i=1}^d x_i y_i}{\sqrt{\sum_{i=1}^d x_i^2} \sqrt{\sum_{i=1}^d y_i^2}}.$$

Using this similarity measure, the predicted rating for a new item is computed by averaging the ratings of its k most similar items in the labeled training set. An enhancement to this method involves weighting each neighbor's rating by its similarity to the target item, improving prediction quality.

While conceptually straightforward, the nearest neighbor approach can be computationally expensive, especially for large training sets. The total complexity is proportional to $|DL| \times |DU|$, where DL is the set of labeled documents and DU is the set of documents to be recommended. Strategies like clustering can be used to reduce the number of comparisons, significantly improving efficiency at recommendation time.

4.2.2. Bayes Classifier

We have already seen a non-standard use of the **Bayes classifier** in the context of collaborative filtering (Section 3), where missing ratings were inferred from the known ones. In contrast, its application in content-based recommender systems is more conventional, closely resembling *text classification*. Here, the training data set DL contains documents labeled by the active user (e.g., like = +1, dislike = -1), forming the user profile. Each document is represented as a binary vector of word presence (i.e., using the **Bernoulli model**). Given a test document $X \in DU$ with binary features $(x_1, x_2, \dots, x_d)$, the goal is to compute

$$P(c(X) = 1 \mid x_1, \dots, x_d).$$

Using Bayes' theorem with the *naive independence assumption*, which assumes that the occurrences of words in a document are conditionally independent given the class, we get

$$P(c(X) = 1 \mid x_1, \dots, x_d) \propto P(c(X) = 1) \cdot \prod_{i=1}^d P(x_i \mid c(X) = 1).$$

This naive assumption allows simplifying the conditional probability of the entire feature vector $x_1, x_2, \dots, x_d$ as the product of the individual word probabilities $P(x_i \mid c(X) = 1)$ for each feature x_i . Similarly, we can compute the posterior for the class $c(X) = -1$

$$P(c(X) = -1 \mid x_1, \dots, x_d) \propto P(c(X) = -1) \cdot \prod_{i=1}^d P(x_i \mid c(X) = -1).$$

The class with the highest posterior probability is predicted. To rank items by relevance, the normalization constant K must also be computed

$$K = \left(P(c = 1) \cdot \prod_{i=1}^d P(x_i \mid c = 1) + P(c = -1) \cdot \prod_{i=1}^d P(x_i \mid c = -1) \right)^{-1}.$$

Thus, items in DU can be ranked based on their probability of being liked, $P(c(X) = 1 \mid x_1, \dots, x_d)$. The required probabilities are estimated from the training data DL , typically with Laplace smoothing to avoid overfitting

$$P(c = 1) = \frac{|D_L^+| + \alpha}{|D_L| + 2\alpha}.$$

The Bayesian classifier is particularly well suited for binary preferences, offering a principled probabilistic framework for ranking candidate items in content-based recommendation tasks.

4.2.3. Regression-based Models

Regression models estimate the relationship between a dependent variable y and independent variables $\mathbf{x} = (x_1, \dots, x_p)$ as

$$y = f(\mathbf{x}) + \epsilon,$$

where $f(\mathbf{x})$ is the prediction function and ϵ is the error term. The goal is to find $\hat{f}$ that minimizes a loss function, often the sum of squared errors

$$\hat{f} = \arg \min_f \sum_{i=1}^n (y_i - f(\mathbf{x}_i))^2.$$

In linear regression, $f(\mathbf{x})$ is assumed linear

$$y = \beta_0 + \sum_{j=1}^p \beta_j x_j + \epsilon,$$

with coefficients β_j learned from data to predict y .

Regression models have the advantage of being applicable to various types of ratings, such as binary ratings, interval ratings, or numerical ratings. Among the most common regression models are linear models, logistic regression, and ordered probit models. In

this section, we focus on the simplest model, linear regression, which can be used to predict ratings in content-based recommender systems.

Let DL be an $n \times d$ matrix, where n represents the number of documents in the labeled training set, and d is the size of the lexicon. Let y be an n -dimensional column vector containing the ratings of the active user for the n documents in the training set. The core idea of linear regression is to assume that the ratings can be modeled as a linear function of the word frequencies in the documents.

Let W be a d -dimensional row vector representing the coefficients of each word in the linear function relating word frequencies to ratings. The linear regression model assumes that the word frequencies in the training matrix DL are related to the rating vector y as

$$y \approx DLW^T.$$

Here, $(DLW^T - y)$ is the vector of prediction errors. To optimize the prediction quality, we minimize the squared norm of this vector. Additionally, a regularization term $\lambda \|W\|_2^2$ is added to the objective function to prevent overfitting. Therefore, the objective function can be written as

$$\text{Minimize } O = \|DLW^T - y\|_2^2 + \lambda \|W\|_2^2.$$

To find the optimal value of W , we take the gradient of this objective function with respect to W and set it to zero, leading to the following condition

$$D_L^T(D_L W^T - y) + \lambda W^T = 0.$$

Rearranging, we get the following

$$(D_L^T D_L + \lambda I)W^T = D_L^T y,$$

where I is a $d \times d$ identity matrix. Since the matrix $(D_L^T D_L + \lambda I)$ is positive definite and invertible, we can directly solve for the weight vector W^T as

$$W^T = (D_L^T D_L + \lambda I)^{-1} D_L^T y.$$

Once the weight vector W^T is obtained, the predicted rating for a new document X from the unlabeled set DU is calculated as the dot product:

$$\hat{y} = W^T X.$$

In addition to $L2$ -regularization (also known as Tikhonov regularization), it is also possible to use $L1$ -regularization (Lasso), which replaces the regularization term $\|W\|_2^2$ with

$\|W\|_1$. This modification leads to an optimization problem that does not have a closed-form solution and requires the use of gradient descent methods. The $L1$ -regularization has the added benefit of performing feature selection, as it produces sparse coefficient vectors for W , where most components of W are zero. This makes the model more interpretable and helps identify important subsets of features for the recommendation process.

5. THEORETICAL FOUNDATIONS OF FAIRNESS

This section is primarily based on the comprehensive treatment of fairness in machine learning presented in Barocas et al., 2023. Additionally, we have contrasted and extended our understanding with the previous work by Pérez-Peralta, 2024, who had already explored this topic in depth. It explores fairness from a mathematical perspective, aiming to establish a rigorous theoretical foundation for its understanding and application in recommender systems. To achieve this, we will first contextualize the problem within the broader framework of statistical classification, formally introducing the fundamental concepts that underpin this field.

The discussion will focus on three key criteria for assessing fairness in algorithms: independence, separation, and sufficiency. These principles not only provide a means to quantify different notions of fairness but also serve as essential tools for developing strategies to mitigate bias in machine learning models. Their detailed analysis will allow for a deeper comprehension of fairness as a mathematical construct, ensuring a precise and structured approach to its integration into algorithmic decision-making.

The theoretical foundations presented here build upon established work in the field, particularly the formal treatment of fairness metrics found in the literature. This approach will enable a systematic exploration of fairness within recommender systems, ensuring that the concepts discussed are both mathematically sound and practically applicable.

5.1. The Framework of Classification Problems

Statistical classification is based on the relationship between a set of covariates X and a target variable Y , where Y represents discrete classes. The goal is to define a function f , known as the *classifier*, that assigns a prediction $\hat{Y} = f(X)$ to new observations. Although there is a formal distinction between f and its predictions $\hat{Y}$, they are often used interchangeably in the literature.

To evaluate the quality of a classifier, performance measures are needed. The simplest is *accuracy*, defined as $P(\hat{Y} = Y)$, which represents the proportion of correct predictions. However, in many cases, a *loss function* $l(\hat{y}, y)$ is more appropriate, as it assigns a cost to misclassifications and can account for the relative importance of different types of errors.

A key assumption in this framework is that the covariates X do not influence the outcome Y . That is, the joint distribution of (X, Y) is considered fixed, meaning that predictions $\hat{Y}$ cannot affect the true value of Y . However, in practice, this assumption may not hold, particularly in scenarios where predictions lead to actions that alter the outcome. For example, if a medical model predicts that a patient is at high risk of developing a disease, doctors might take preventive measures that reduce the actual likelihood of the disease.

occurring.

Since accuracy alone is not always a sufficient criterion, *statistical decision theory* provides tools to evaluate different aspects of a classifier. For instance, a model that always predicts that a student will not fail an exam might appear to perform well in terms of accuracy if most students indeed pass. However, such a model would be uninformative in identifying students who actually need academic support.

To better understand classification errors and conditional probabilities, in the basic case where the response variable is binary (i.e., 0 or 1), we can directly apply a **confusion matrix** (Table 5.1), which provides a structured way to analyze these concepts.

	$\hat{Y} = 1$	$\hat{Y} = 0$
$Y = 1$	TPR = $\mathbb{P}(\hat{Y} = 1 \mid Y = 1)$	FNR = $\mathbb{P}(\hat{Y} = 0 \mid Y = 1)$
$Y = 0$	FPR = $\mathbb{P}(\hat{Y} = 1 \mid Y = 0)$	TNR = $\mathbb{P}(\hat{Y} = 0 \mid Y = 0)$

Table 5.1

Confusion matrix

The components of the confusion matrix are as follows:

- **True Positive Rate (TPR):** $\mathbb{P}(\hat{Y} = 1 \mid Y = 1)$
Measures the proportion of actual positive cases ($Y = 1$) that are correctly classified as positive ($\hat{Y} = 1$). Also known as *sensitivity* or *recall*.
- **False Positive Rate (FPR):** $\mathbb{P}(\hat{Y} = 1 \mid Y = 0)$
Represents the proportion of actual negative cases ($Y = 0$) that are incorrectly classified as positive ($\hat{Y} = 1$). This is also referred to as the *Type I error*.
- **False Negative Rate (FNR):** $\mathbb{P}(\hat{Y} = 0 \mid Y = 1)$
Denotes the proportion of actual positive cases ($Y = 1$) that are incorrectly classified as negative ($\hat{Y} = 0$). Also known as the *Type II error*.
- **True Negative Rate (TNR):** $\mathbb{P}(\hat{Y} = 0 \mid Y = 0)$
Measures the proportion of actual negative cases ($Y = 0$) that are correctly classified as negative ($\hat{Y} = 0$). Also called *specificity*.

To identify the optimal classifier, we will leverage the concept of minimizing the loss function introduced earlier. Given that the loss function is a random variable, direct analysis can be complex. Consequently, we define the optimal classifier as the one that **minimizes the expected value of the loss function**, offering a more practical and effective approach to solving the problem.

Definition 5.1.1. *Given a random variable X representing the input features and a target variable Y , the optimal classifier is defined as the function $f : \mathbb{R}^n \rightarrow \mathcal{Y}$ that minimizes the expected loss function,*

$$f = \arg \min_g \mathbb{E} [l(g(X), Y)],$$

where $l(\hat{Y}, Y)$ represents the loss function that measures the discrepancy between the predicted label $\hat{Y} = g(X)$ and the true label Y .

The general expression of this idea may lead to some confusion, especially when considering more complex cases. To clarify the concept, we will first present the scenario where our response variable Y is binary, as this provides a simpler and more intuitive understanding. Ultimately, this is the case we will focus on, as in recommendation systems, the response variable is typically discrete and takes a finite set of values. For example, the ratings of movies, which can range from 1 to 5 stars, are treated as discrete values in such systems.

Proposition 5.1.1. *Let Y be a binary target variable. Consider the simplest case where we assume a symmetric loss function such that*

$$l(0, 0) = l(1, 1) = 0, \quad l(1, 0) = l(0, 1) = 1.$$

Then, the optimal classifier is given by

$$f(x) = \begin{cases} 1, & \text{if } \mathbb{P}(Y = 1 \mid X = x) > \frac{1}{2}, \\ 0, & \text{otherwise.} \end{cases}$$

A detailed proof of this proposition is provided in Appendix A.

The optimal classifier determines the predicted outcome by comparing the probability of a positive result given the observed covariates X to a fixed threshold, which in this case is $1/2$. If this probability is greater than the threshold, the classifier assigns the outcome $Y = 1$; otherwise, it assigns $Y = 0$. This choice of threshold is specific to minimizing classification error, but if the loss function were different, the threshold would need to be adjusted accordingly. This adaptation is particularly relevant in situations where the costs of false positives and false negatives differ significantly.

When a classifier is defined as a threshold applied to a specific function, this function is referred to as a risk score. In the case of the optimal classifier we discussed earlier, it can be expressed as a threshold on this risk score function. The risk score represents a numerical value that estimates the likelihood of a particular outcome, and the classifier makes a decision based on whether the risk score exceeds a certain threshold. In the previous case, we have

$$r(x) = P(Y = 1 \mid X = x) = \mathbb{E}(Y \mid X = x).$$

We can think of the risk score as a way of taking the available evidence, represented by $X = x$, and calculating the expected result given the observed information. This is known as the posterior probability of the outcome Y given X , or $P(Y = 1 \mid X = x)$. The risk score thus quantifies the likelihood of a particular outcome based on the input features, serving as a continuous measure for making predictions.

Definition 5.1.2. For a target random variable Y given the covariates X , the risk score is defined as

$$r(x) := \mathbb{E}(Y \mid X = x).$$

Theorem 5.1.1. The risk score minimizes the square loss $\mathbb{E}[(Y - r(X))^2]$.

5.2. Statistical Criteria for Non-Discrimination

Discrimination is a complex issue that arises in many contexts. In particular, recommendation systems, which are integral to many online platforms, can inadvertently perpetuate biases and discrimination. These systems often rely on statistical classification rules to predict preferences or outcomes based on historical data. However, when such systems use sensitive attributes, such as race, gender, or socio-economic status, they may unknowingly create or reinforce biased decisions, particularly when such attributes are correlated with other features used in the system.

Discrimination, in this context, refers to treating individuals unfairly based on their membership in socially significant groups. These groups, which can be defined by factors like race, gender, religion or other protected categories, have historically experienced unjust and systematic disadvantages. When designing and deploying machine learning systems, including recommendation algorithms, it's important to be mindful of how sensitive attributes, which are correlated with these protected categories, are handled.

To address discrimination, we must define clear metrics for non-discrimination and fairness. These metrics aim to ensure that the decisions or recommendations made by the system do not systematically disadvantage any particular group. This statistical non-discrimination criteria help formalize fairness by defining specific conditions under which discrimination can be said to occur in the system's predictions.

The central challenge lies in rigorously defining and evaluating **fairness** within statistical learning frameworks. This necessitates the development of *precise and quantifiable criteria* for assessing whether a model behaves equitably across subpopulations. A prevalent strategy in the literature involves enforcing the *equality of specific statistical measures* across groups defined by a sensitive attribute A (e.g., gender, race).

To formalize these criteria, we examine the *joint distribution* over the relevant variables: the sensitive attribute A , the true label Y , the predicted label $\hat{Y} = f(X)$, and optionally the score R or input features X . By characterizing fairness constraints in terms of properties of this joint distribution, we obtain a principled and unambiguous framework for verifying whether a given fairness notion is satisfied.

Common fairness definitions derived from this perspective include the following:

- **Acceptance rate:** $P(\hat{Y} = 1)$,
- **Error rates:** Such as false negative and false positive rates,

- **Outcome frequency for a given score value:** $P(Y = 1 \mid R = r)$.

These criteria can also be generalized to score functions by using conditional independence statements. Specifically, we will explore the three key conditions for fairness: *Independence*, *Separation*, and *Sufficiency*. These conditions help define the relationships between sensitive attributes and the decision-making process and provide a framework for understanding fairness in machine learning models.

5.2.1. Independence

Before diving into the details of **Independence** as a fairness criterion, it's important to understand its definition and application in decision-making systems like machine learning models. Independence ensures that a decision outcome, represented by a classifier or score, is not influenced by sensitive attributes such as race, gender, or religion. Formally, two random variables, like a sensitive attribute A and a score R , are independent if there is no direct relationship between them.

Definition 5.2.1. (Independence). *Two random variables A and R are said to be independent if and only if their joint probability distribution factorizes, i.e.,*

$$P(A, R) = P(A)P(R).$$

This condition is equivalent to stating that $A \perp R$, meaning that knowing the value of A does not provide any additional information about R , and vice versa.

Proposition 5.2.1. *Let Y be a binary variable. If R is a score that satisfies independence with the sensitive variable A , then any classifier $\hat{Y} = \mathbb{I}(R > t)$, where t is a threshold, also satisfies independence, provided that the threshold t is independent of the group A .*

A detailed proof of this proposition is provided in Appendix A.

The concept of independence with respect to a classifier or score can be generalized to cases where the response variable is not binary. While the binary case is often used for simplicity, it does not cover the broader range of situations that arise in many real-world applications, such as recommendation systems. In these systems, scores can take a variety of forms, often continuous or ordinal, depending on the type of recommendation being made (e.g., ratings, rankings, or probabilities).

In these cases, instead of simply comparing probabilities of a binary outcome, we would need to consider how the score interacts with the sensitive attribute across the range of possible outcomes. The core idea of ensuring that the score does not depend on the sensitive attribute remains the same, but the mathematical formulation becomes more complex. For instance, instead of enforcing equal probabilities for acceptance or rejection, as in the binary case, we may require that the distribution of the scores across different groups remains consistent, or that the expected score for each group is the same.

Proposition 5.2.2. *Let $\hat{Y} = F(R, \theta)$ be a classifier, where R is the risk score and θ represents the parameters or thresholds, both independent of A . Then, if R and θ are independent of A , the classifier $\hat{Y}$ satisfies independence with respect to A . In other words, if the score R is independent of the sensitive attribute A , the decision-making function of the classifier will not depend on A , ensuring fairness in the prediction.*

A detailed proof of this proposition is provided in Appendix A.

In practice, requiring complete independence between the risk score R (or the score) and the sensitive attribute A may not be realistic, as certain characteristics related to a person's group membership can be implicitly present in the data. Even in systems designed to be fair, these characteristics could influence the decision probabilities in some way. To address this challenge, we can allow for a certain relaxation of perfect independence by introducing a tolerance parameter, known as "slack," denoted as $\epsilon > 0$.

This ϵ parameter allows the probabilities of acceptance or classification between different groups to differ by a maximum amount of ϵ . In other words, instead of requiring the probabilities to be identical for all groups $P(\hat{Y} = y \mid A = a) = P(\hat{Y} = y \mid A = b)$, we can accept that the probabilities may differ within a limit of ϵ . This approach offers a way to handle the imperfections inherent in real-world models, allowing the model to remain flexible enough to handle complex data while still maintaining a reasonable level of fairness in its decisions.

In the context of recommendation systems, this relaxation is particularly important because recommendation algorithms often rely on user data that can implicitly reflect sensitive attributes such as gender, age, or ethnicity. By introducing the ϵ -slack, we can make sure that the system does not disproportionately favor or disadvantage any group while still delivering useful and accurate recommendations. This helps strike a balance between fairness and practical effectiveness in the recommendations generated.

Limitations of independence

While independence is a common fairness criterion in machine learning, including recommendation systems, it may not always be practical in real-world applications. Independence assumes that all groups should receive equal treatment, meaning recommendations should be proportionally allocated across different groups. While this criterion is mathematically convenient, it may overlook important nuances.

For example, consider a recommendation system where Group A receives highly personalized recommendations based on detailed user data, while Group B receives more generic suggestions. Despite equal acceptance rates across groups, Group B may experience lower-quality recommendations, leading to a worse user experience. This discrepancy can create an unintended negative track record for Group B, even if both groups are formally treated equally.

Thus, while independence is a useful starting point, it may not always reflect the complexities of real-world recommendation systems, where disparities in data and personalization can influence outcomes.

5.2.2. Separation

While independence ensures that predictions are not influenced by sensitive attributes, it does not account for potential differences in actual outcomes across groups. **Separation** introduces a more nuanced perspective by requiring that the model's predictions are conditionally independent of the sensitive attribute given the true outcome.

Formally, for a binary response variable Y , we define **separation** as a condition that ensures fairness by requiring that predictions remain independent of the sensitive attribute when conditioned on the true outcome. Specifically, a classifier $\hat{Y}$ satisfies separation if

$$P(\hat{Y} = 1 \mid Y = y, A = a) = P(\hat{Y} = 1 \mid Y = y, A = b), \quad \forall a, b \in A, \quad y \in \{0, 1\}.$$

This means that the probability of a positive prediction remains the same for all groups when conditioned on the actual outcome.

In the context of recommendation systems, this criterion ensures that the likelihood of recommending an item to a user is consistent across different demographic groups, provided they share the same level of relevance or engagement. However, strict adherence to separation may be impractical in real-world applications, leading to relaxed formulations that allow for some deviation.

To extend the concept of separation beyond the binary classification case, we can define separation in a more general way. Instead of focusing solely on a classifier $\hat{Y}$, we consider a risk score R , which may take continuous values. This leads to the following definition.

Definition 5.2.2. (Separation) *Three random variables R , A , and Y satisfy separation if and only if*

$$R \perp A \mid Y.$$

This means that the risk score R is conditionally independent of the sensitive attribute A given the true outcome Y .

Separation and Error Rate Parity

Separation requires that a recommender system maintains equal error rates across all user groups, ensuring that both the false positive rate (i.e., irrelevant recommendations) and the false negative rate (i.e., missed relevant recommendations) remain consistent. In other words, all groups should experience the same cost of misclassification.

However, enforcing strict equality of error rates has been a topic of debate. In recommender systems, an optimal model does not necessarily achieve equal error rates across

all user groups, especially when the base rates of engagement or preference differ. For instance, some groups might have fewer interactions, leading to less accurate predictions and higher error rates. Enforcing strict error parity in such cases could result in sub-optimal performance for certain groups, and raises the critical question: who bears the cost of misclassification? Higher error rates often disproportionately affect historically marginalized groups, which could exacerbate existing biases.

Moreover, separation assumes that the target variable in the recommender system accurately reflects user preferences or merit, which is not always the case. If the target variable is biased (e.g., reflecting historical preferences or engagement patterns), ensuring separation will not address the underlying issue of bias. This highlights the importance of critically assessing the data and target variables used in recommender systems and ensuring that fairness is appropriately handled.

A useful way to visualize separation in recommender systems is by plotting group-specific precision-recall or ROC curves. A recommender system that satisfies separation must achieve similar trade-offs between true positive and false positive rates across all user groups. In practice, achieving this may require adjusting recommendation thresholds or using randomized classifiers. The feasible region of trade-offs is given by the intersection of the areas under the curves for different groups, which emphasizes the challenges of balancing fairness and system performance in real-world applications.

Generalized Conditional Fairness Criteria

A useful generalization of independence and separation criteria in recommendation systems is to consider conditional acceptance rates, where we introduce a random variable W and require that the recommendation decision Y be conditionally independent of the group status A given W . Formally, we demand that for all values of w and groups a and b , it holds that

$$P(Y = 1 \mid W = w, A = a) = P(Y = 1 \mid W = w, A = b).$$

This approach extends the separation criterion by conditioning on a subset of the features relevant to the recommendation process, represented by W . For example, we could condition on factors such as user preferences, historical interactions, or demographic attributes to ensure that the recommendation decision is independent across different groups with similar characteristics.

The key advantage of this criterion is that it allows for fairness while still considering relevant factors in the recommendation process. However, careful consideration is required when selecting the conditioning variable W to avoid masking potential biases in the recommendation system.

5.2.3. Sufficiency

An alternative approach to addressing the limitations of independence in recommendation systems is to develop a risk score that encapsulates all relevant information about the outcome variable. This score should enable accurate predictions of the target variable without requiring additional information about sensitive groups, ensuring that predictions are unaffected by these attributes.

Definition 5.2.3 (Sufficiency). *Three random variables R , A , and Y are said to satisfy sufficiency if and only if*

$$Y \perp A \mid R.$$

Proposition 5.2.3. *In the case of a binary outcome, sufficiency is valid if and only if*

$$P(Y = 1 \mid R = r, A = a) = P(Y = 0 \mid R = r, A = b).$$

Sufficiency is closely related to the concept of *calibration*, which plays a crucial role in several machine learning applications, including recommendation systems. In contexts where the output of a model is a score, it is often desirable to interpret these scores as probabilities. Calibration ensures that the predicted scores in the model can be treated as such, i.e., a score assigned to an instance corresponds to the estimated probability of the positive outcome.

In recommendation systems, calibration is essential because it allows the system's score to be understood in probabilistic terms. For example, if a recommendation model assigns a score of 0.7 to a product for a user, calibration guarantees that the probability that the user interacts with the product is approximately 70%. This interpretation is critical when using the recommendation system for decision-making processes, as it ensures consistency in the output.

For the simplest case where the target variable Y is binary, we can formalize the following concepts.

Definition 5.2.4 (Calibration). *Let R be a score assigned by a model. We say that R is calibrated if its values are bounded within the interval $[0, 1]$, and for all values r within the support of R , the following holds.*

$$P(Y = 1 \mid R = r) = r.$$

This means that for a given score r , the proportion of positive results among all instances assigned this score should be equal to r . Calibration ensures that the predicted probability aligns with the actual frequency of positive instances, but it does not provide individual predictions for specific users or groups.

In contexts where fairness is a concern, such as recommendation systems, we extend the notion of calibration to account for different subgroups of individuals. This extension is called *calibration by group*.

Definition 5.2.5 (Calibration by Group). *A score R satisfies calibration by group if, for each group $a \in A$, the following condition holds:*

$$P(Y = 1 \mid R = r, A = a) = r.$$

Calibration by group ensures that, for each group defined by the sensitive attribute A , the score reflects the same proportion of positive outcomes. In recommendation systems, this is particularly important because it ensures that the model is equally accurate for all demographic groups, thus mitigating potential biases in predictions.

Once we have clarified these concepts, we can highlight several important results that follows from the definition of sufficiency.

Theorem 5.2.1. *If a score R satisfies calibration by groups, then it also satisfies sufficiency.*

Notice that these notions naturally generalize beyond the binary case. For a discrete variable Y with multiple classes, the definition of calibration extends as follows

$$P(Y = y \mid R = r, A = a) = P(Y = y \mid R = r), \quad \forall y \in \mathcal{Y}, \forall a \in A.$$

This ensures that the predicted scores remain consistent across groups, regardless of the number of possible outcomes for Y .

Similarly, for a continuous target variable Y , calibration can be expressed in terms of the conditional cumulative distribution function

$$F_{Y|R}(y \mid r) = P(Y \leq y \mid R = r),$$

which means that the score R correctly predicts the conditional distribution of Y given $R = r$. In this setting, calibration implies that R provides well-calibrated probabilistic estimates rather than single-point predictions.

In the context of recommender systems, these generalizations are particularly relevant. The output of a recommender model can take different forms depending on the task: a binary relevance score (e.g., whether a user interacts with an item or not), a categorical rating (e.g., star ratings in movie recommendations), or a continuous preference score (e.g., estimated engagement time). Ensuring proper calibration across these different types of outcomes is crucial to maintaining fairness and interpretability. If a recommender system assigns probabilistic scores to recommendations, then a well-calibrated model ensures that these scores accurately reflect real-world likelihoods of user interaction. When dealing with continuous preference scores, calibration guarantees that the model's predicted distributions align with observed user behavior.

After establishing the fundamental concepts of calibration and sufficiency, we now explore additional theoretical results that highlight the relationship between these notions.

One particularly important result states that sufficiency implies the existence of a transformed score that satisfies calibration by group. This result provides a constructive way of ensuring group fairness when a model satisfies sufficiency.

Proposition 5.2.4. *If a score R satisfies sufficiency, then there exists a function $\ell : [0, 1] \rightarrow [0, 1]$ such that $\ell(R)$ satisfies calibration by group.*

A detailed proof of this proposition is provided in Appendix A.

This proposition is particularly relevant in the context of recommender systems. If a model’s scoring function satisfies sufficiency, we can apply the transformation ℓ to obtain a calibrated score that ensures fairness across demographic groups. This provides a theoretical justification for post-processing techniques aimed at mitigating bias while preserving the predictive power of the model.

Calibration by Group as a Consequence of Unconstrained Learning

In many real-world situations, *sufficiency* appears naturally in supervised learning, even without explicitly adding fairness constraints. This happens because supervised learning aims to find the best prediction function, one that minimizes the error when predicting Y from the input features.

The key idea is this: if a model is optimal, it should estimate the expected value of Y as accurately as possible based on the features. If the group membership (such as gender or age group) is part of those features, or can be inferred from them, then the model’s predictions will also be well-calibrated across those groups. In other words, the model’s output will mean the same thing for everyone, regardless of group. This is what we call *calibration by group*, and it is a direct consequence of *sufficiency*.

We now formalize this observation in the following proposition.

Proposition 5.2.5. *Let $R(X) = \mathbb{E}[Y \mid X]$ be the optimal score function obtained from an unconstrained supervised learning model. Then $R(X)$ satisfies calibration by group. Specifically, for any group attribute A and any measurable subset S of the feature space, we have*

$$P(Y = 1 \mid R = r, X \in S) = r.$$

A detailed proof of this proposition is provided in Appendix A.

5.3. Incompatibility of Criteria

In the design and development of recommender systems, multiple performance criteria are typically used to ensure that recommendations are both accurate and fair. However, these criteria are not always compatible. When multiple criteria are imposed simultaneously, they can interact in complex ways, sometimes constraining the system’s ability to find a

viable solution. Specifically, enforcing certain criteria may lead to trade-offs that cause other criteria to be violated.

For example, consider two criteria: independence and sufficiency. Independence refers to the idea that sensitive attributes, such as demographic information, should not influence the system's predictions. Sufficiency, on the other hand, requires that the predicted scores fully capture all relevant information for making predictions, including the influence of these sensitive attributes. In recommender systems, these two goals can often conflict, particularly when group membership or other factors related to user identity impact user preferences.

This section will explore the relationships between different criteria and how they interact. We will see that imposing both independence and sufficiency simultaneously is generally not feasible, especially when sensitive attributes have an impact on the outcome variable (e.g., user preferences). Additionally, the principles of separation and sufficiency will be discussed, showing that enforcing both conditions can also lead to problematic solutions.

In the following propositions, we will formalize these relationships and demonstrate that imposing certain criteria can inadvertently violate others.

Proposition 5.3.1. *Assume that A and Y are not independent. Then, sufficiency and independence cannot both hold.*

A similar result of mutual exclusivity applies to independence and separation. However, the argument in this case is more intricate and requires the additional assumption that the target variable Y is binary. Moreover, we assume that the score is not independent of the target, which is a reasonable condition, as any meaningful score function should exhibit some correlation with the target variable.

Proposition 5.3.2. *Assume that Y is binary, A is not independent of Y , and R is not independent of Y . Then, independence and separation cannot both hold.*

Finally, we address the relationship between separation and sufficiency. Both conditions require a non-trivial conditional independence relationship between the three variables A , R , and Y . Enforcing both simultaneously results in a degenerate solution space, as demonstrated by the following proposition.

Proposition 5.3.3. *Assume that all events in the joint distribution of (A, R, Y) have positive probability, and assume $A \perp Y$. Then, separation and sufficiency cannot both hold at the same time.*

The formal proofs for the three propositions showing the incompatibility between independence, separation, and sufficiency are provided in Appendix A.

6. FAIRNESS APPLIED TO RECOMMENDER SYSTEMS

In Section 5, we introduced the theoretical foundations of fairness and discussed how these concepts can be generalized across different contexts. We also briefly outlined their potential applications to classification problems. In this section, however, we shift our focus to the technical integration of fairness into recommender systems. Specifically, we will explore how to design algorithms that are fairness-aware, addressing various types of bias that can arise in personalized recommendation settings. This section is primarily based on the comprehensive survey by Jin et al., 2023, which provides an in-depth overview of fairness-aware recommender systems.

6.1. Factors Contributing to Unfairness in Recommender Systems

Unfairness in recommender systems can emerge at every stage of their lifecycle, including data collection, model training, and user interaction. Based on its source, bias is typically categorized into three types: data bias, model bias, and feedback bias.

6.1.1. Data Bias in Recommender Systems

Data bias occurs when the data collected during the acquisition phase contains inherent imbalances or distortions, leading to unfair recommendations. In the context of movie recommender systems, several types of bias can arise:

- 1. User Attribute Bias:** This occurs when recommendations are influenced by sensitive user characteristics like age, gender, or location. For example, recommending action movies primarily to younger users may limit their exposure to other genres, like classic films, that might better suit their tastes.
- 2. User Selection Bias:** This bias arises when user behavior, such as clicks or ratings, doesn't fully reflect true preferences due to external factors like multitasking or accidental clicks. For instance, a user might rate a movie highly while distracted, skewing the system's understanding of their actual preferences.
- 3. Exposure Bias:** This happens when users are only shown a limited set of popular items, reducing the chance to discover less popular, but potentially more relevant, content. For example, the system may prioritize blockbuster films over independent or niche movies, limiting variety in recommendations.
- 4. Time Bias:** Time bias favors newer content over older films, potentially ignoring relevant older movies that might match a user's preferences. For example, a system might push new releases while overlooking classic films that could be more aligned with a user's interests.

5. Cold-Start Bias: This bias affects new users or items with limited historical data. For new users who haven't interacted with the system, or new movies without user feedback, the system may struggle to provide personalized recommendations, resulting in less relevant suggestions.

6.1.2. Model Bias in Recommender Systems

Model bias arises during the training stage when algorithms learn patterns from the collected data. Common optimization techniques, such as pointwise or pairwise loss functions like mean squared error or Bayesian personalized ranking, tend to prioritize frequently occurring items. As a result, these algorithms often reinforce patterns that favor popular items, especially in the absence of fairness constraints. This leads to recommendation lists that amplify existing trends in the data, making it more difficult for less popular or niche items to be recommended.

6.1.3. Feedback Bias in Recommender Systems

Feedback bias emerges after the system is deployed and begins to adapt based on user interactions. These interactions are shaped by earlier recommendations, which may have already been biased. The most prominent and pervasive form of feedback bias is **popularity bias**. Popularity bias refers to the disproportionate recommendation and consumption of a **small set of highly popular items** while the majority of other items receive little to no exposure. Initially, popular items gain more visibility, which leads to increased user interaction. The system interprets these interactions as indicators of preference and continues to recommend the same items in subsequent iterations. Over time, this feedback loop causes the distribution of recommendations to become increasingly concentrated around a narrow set of popular content. This not only limits users' exposure to diverse or novel items but also undermines the long-term fairness and effectiveness of the recommender system.

6.2. Expressions of Fairness in Recommender Systems

A recommender system can operate across a variety of application domains, each with its own fairness challenges and objectives. Consequently, the notion of fairness in recommendation is multifaceted and context-dependent. In this section, we formally describe different fairness paradigms commonly studied in the literature, categorizing them based on individual vs. group fairness, static vs. dynamic fairness, and single-sided vs. multi-sided fairness.

6.2.1. Individual Fairness vs. Group Fairness.

Individual fairness focuses on treating similar users similarly, ensuring that recommendation outcomes do not vary unjustifiably between users with comparable profiles. In contrast, group fairness is concerned with equal treatment across demographic or otherwise defined user groups, aiming to mitigate systemic biases and disparities in aggregate outcomes.

Definition 6.2.1. *Given two similar users $u_i, u_j \in \mathcal{U}$, individually fairness-aware recommender systems aim to produce similar prediction scores for a given item $v \in \mathcal{V}$, i.e.,*

$$f(u_i, v) \approx f(u_j, v),$$

where $f(\cdot, \cdot)$ denotes the scoring function of the recommender system.

Methods enforcing individual fairness address statistical parity through pairwise comparisons between users with similar characteristics. For example, in a movie recommendation platform, two users with similar viewing histories, such as consistent preferences for independent films and documentaries, should receive recommendations of comparable quality and relevance.

Group fairness, on the other hand, seeks to ensure that protected user groups receive equitable treatment in comparison to advantaged groups. A standard notion of group fairness is *demographic parity*, which requires that the probability of receiving a positive recommendation (e.g., high predicted rating or top-k inclusion) for users in a protected group (e.g., young adults) is equal to that of an advantaged group (e.g., middle-aged adults).

Definition 6.2.2. *For a protected group $\mathcal{U}_i$ and an advantaged group $\mathcal{U}_j$, this can be expressed as*

$$\Pr(y_{\mathcal{U}_i, v} = 1 \mid \mathcal{U}_i) = \Pr(y_{\mathcal{U}_j, v} = 1 \mid \mathcal{U}_j),$$

where $y_{\mathcal{U}, v}$ indicates whether item v is recommended to users in group $\mathcal{U}$.

For instance, if highly-rated dramas are predominantly recommended to one demographic group while underrepresented in another, the system may fail to satisfy demographic parity. As can be seen, the notion of group fairness described in the previous section, which ensures that the probability of receiving a positive recommendation is equal across different groups (e.g., males and females), reflects the concept of independence that we had presented in Section 5.

6.2.2. Static Fairness vs. Dynamic Fairness.

Static fairness refers to fairness criteria that are evaluated in a fixed context, without considering temporal dynamics or evolving system conditions. In this framework, the definition of protected groups remains constant throughout the recommendation process. Traditional approaches such as matrix factorization are typically designed to optimize users'

immediate satisfaction, assuming that user preferences are stable over time. While this can be effective in the short term, it may neglect users' long-term interests and contribute to the reinforcement of existing biases.

In contrast, dynamic fairness incorporates temporal factors and adapts to changes in the recommendation environment. One of its primary goals is to reduce temporal bias, which occurs when recently recommended or popular items are favored over older or less exposed ones. A dynamically fair system aims to ensure that the probability of recommending an item does not decrease solely due to its age.

Definition 6.2.3 (Dynamic Fairness). *Let v_t and v_{t+1} denote two items that appeared at time steps t and $t + 1$, respectively. Dynamic fairness requires that the conditional probability of an item being recommended, given that it was available at a specific time, remains stable across time*

$$\mathbb{P}(y_{u,v_t} = 1 \mid v_t) = \mathbb{P}(y_{u,v_{t+1}} = 1 \mid v_{t+1}),$$

where $y_{u,v_t} = 1$ indicates that user u interacted with item v_t . This condition ensures that recommendation decisions are not biased toward more recent items solely due to their recency.

To introduce flexibility, a slack parameter $\epsilon \geq 0$ can be used to tolerate small deviations

$$\left| \mathbb{P}(y_{u,v_t} = 1 \mid v_t) - \mathbb{P}(y_{u,v_{t+1}} = 1 \mid v_{t+1}) \right| \leq \epsilon.$$

This formulation enables the system to maintain temporal neutrality while accounting for natural fluctuations in user preferences or item relevance over time.

6.2.3. Single-sided Fairness vs. Multi-sided Fairness.

Recommender systems often involve several types of participants, which means fairness can be understood from different perspectives. What is known as *single side fairness* refers to ensuring fair treatment for only one group, usually the users. The goal in this case is to make sure each user receives recommendations that are appropriate and equitable, without considering how those recommendations affect other actors in the system.

On the other hand, *multi group fairness* aims to consider the interests of different participant groups at once, such as users and item providers. While users want personalized and relevant suggestions, content providers seek fair visibility for their products or services. This form of fairness tries to maintain a balance between the goals of both parties.

One way to promote fairness across multiple groups is to encourage diversity in the recommendation results. This means ensuring that items from different groups receive attention proportional to their average usefulness. Ideally, the exposure each group receives should match its contribution to overall utility. This can be written mathematically as

$$\frac{1}{|V_i|} \sum_{v_i \in V_i} \Pr(y_{u,v_i} = 1 \mid V_i) = \frac{1}{|V_j|} \sum_{v_j \in V_j} \Pr(y_{u,v_j} = 1 \mid V_j),$$

where V_i and V_j are two separate item categories, and $\Pr(y_{u,v} = 1 \mid V)$ is the probability that a user u receives an item from group V as a recommendation.

In addition to user- or provider-centered fairness, there are other, less common perspectives on fairness in recommender systems. One such approach is ranking fairness, which aims to ensure that items from protected groups receive adequate exposure in top positions, promoting that similar items appear in similar ranks. Another is causal fairness, which goes beyond observational data by incorporating causal relationships to ensure that recommendations remain consistent under both factual and counterfactual scenarios. Long-term fairness considers how fairness interventions influence user preferences and system behavior over time. Lastly, envy-freeness fairness suggests that each user should prefer their own recommendations over those of others. While these perspectives are not as widely explored, they expand the conceptual framework for addressing fairness in recommender systems.

6.3. Techniques for Enhancing Fairness in Recommendations

Designing recommender systems that account for fairness requires a structured approach to identifying and addressing potential sources of bias. One effective framework for achieving this is to categorize mitigation strategies based on the stage of the recommendation pipeline where they are applied. These stages include pre-processing, in-processing, and post-processing. Each phase provides unique opportunities to introduce fairness constraints or corrective measures, depending on whether interventions target the input data, the learning algorithm, or the final outputs.

Pre-processing techniques aim to correct or reduce bias in the training data before model training occurs. In-processing methods integrate fairness considerations directly into the model's learning process. Post-processing strategies adjust the system's predictions or recommendations after training is complete, ensuring equitable outcomes without modifying the model or data.

6.3.1. Pre-Processing Approaches for Fairness in Recommender Systems

Pre-processing represents the earliest intervention point for addressing unfairness in recommender systems. The main goal at this stage is to mitigate existing biases embedded in the training data before any learning algorithm is applied. These biases, if left unaddressed, can propagate through the model and amplify disparities in the recommendations produced.

Conceptually, pre-processing techniques aim to reshape the data landscape to ensure that

the model is trained on a more balanced and representative input. This can be achieved through various strategies. One common approach involves *re-labeling*, where the annotations or feedback signals are adjusted to correct for noisy or misleading data, such as implicit signals that do not accurately reflect user intent. Another strategy is *re-sampling*, which tackles data imbalance by altering the distribution of training samples. This may involve adjusting the representation of different user groups or item categories to prevent skewed learning outcomes, particularly in scenarios where certain populations or content types are underrepresented. A third category is *data modification* or *augmentation*, where the training set is enriched or transformed to compensate for missing, incomplete, or biased information. This can include text normalization in content-based systems or matrix completion techniques in collaborative filtering models dealing with sparse user-item interactions.

6.3.2. In-processing methods for fairness-aware recommender systems

In-processing methods focus on incorporating fairness considerations directly into the model training phase of recommender systems. Unlike pre-processing approaches that modify data before training, in-processing techniques aim to learn models that inherently reduce or eliminate bias during the optimization process. This is crucial, as many recommendation algorithms rely on correlations in historical data, which may contain embedded biases such as popularity bias, user-oriented bias, or cold-start bias.

These methods can be broadly categorized into several types. First, regularization-based approaches introduce fairness-related penalty terms into the loss function to balance accuracy and fairness objectives. By penalizing unfair predictions, the model is guided to produce fairer recommendations while maintaining performance. Second, causal inference techniques utilize causal graphs and statistical methods to identify and adjust for sources of bias by understanding the underlying causal relationships within the data. Third, adversarial learning frameworks incorporate adversarial components that aim to remove sensitive attribute information from learned representations, ensuring that the recommender's outputs are decoupled from protected features. Other categories include reinforcement learning and ranking-specific fairness methods, which also address fairness from different angles during model training.

6.3.3. Post-processing methods for fairness-aware recommender systems

Post-processing approaches aim to enhance fairness after a recommender system has generated its predictions. These methods treat the trained model as a black box and focus on adjusting the final recommendation outputs, typically through re-ranking strategies. The rationale behind post-processing is that fairness constraints are often not directly considered during model training, potentially leading to biased recommendation results.

Re-ranking techniques are central in this context, where the original ranked lists are re-

arranged to better satisfy fairness objectives while preserving recommendation quality. These methods are broadly classified into two categories: non-parametric and parametric re-ranking.

Non-parametric re-ranking involves heuristic or rule-based algorithms that do not require additional training. These strategies often model the re-ranking task as an optimization problem, such as 0–1 integer programming or knapsack formulations, where fairness constraints are incorporated to adjust item exposure or representation across user or item groups.

Parametric re-ranking, on the other hand, employs learnable models to refine the ranking results. These approaches aim to learn how to balance relevance and fairness through training processes. They have been shown to be more effective in reducing certain types of biases, such as unequal opportunity, and often utilize models like neural networks or autoencoders to adaptively re-rank the recommendations.

Overall, post-processing methods provide a flexible and model-agnostic means of improving fairness, particularly useful when retraining or modifying the original recommender is impractical.

7. CASE STUDY: MOVIELENS

In this project, we address the problem of fairness in recommender systems using the MovieLens 100k dataset. This dataset, which includes user-item interactions along with demographic information (such as age and gender) and movie metadata (such as genres), is a reduced version of the MovieLens 1M dataset. It retains the same structure but significantly lowers computational complexity, allowing us to implement and analyze a wide range of models in a more tractable setting.

Our work is divided into two main components. The first part focuses on the development and evaluation of various recommender system approaches, including memory-based collaborative filtering and latent factor models (e.g., matrix factorization). We will assess their performance in terms of accuracy and personalization, identifying their respective strengths and limitations.

In the second part, we will study the fairness and popularity bias in recommender systems by adapting the methodology proposed in *All The Cool Kids, How Do They Fit In?: Popularity and Demographic Biases in Recommender Evaluation and Effectiveness* (Ekstrand et al., 2018), which was originally applied to the MovieLens 1M dataset. The article explores how recommendation algorithms may systematically favor popular items, leading to unequal exposure and reduced diversity, and how these effects can disproportionately impact different user groups. We will reproduce and extend their fairness evaluation framework on the MovieLens 100k dataset.

7.1. Methodology used for this analysis

In this study, we adopt a two-stage methodology, aligning with the structure of our project: model evaluation and fairness analysis. Both stages employ rigorous experimental setups inspired by standard practices in the recommender systems literature, particularly those detailed by Ekstrand et al., 2018, as previously mentioned.

7.1.1. Collaborative Filtering Evaluation

In the first part of our analysis, we focus on training and evaluating collaborative filtering algorithms, both user-based and item-based. These models will be implemented using a variety of similarity measures (e.g., cosine, Pearson correlation), and we will explore different values of the neighborhood parameter k , i.e., the number of neighbors considered for making a recommendation. This allows us to observe how these factors influence the performance metrics previously defined, such as relevance (Eq. (2.1)), novelty (Eq. (2.2)), serendipity (Eq. (2.3)) and diversity (Eq. (2.4)).

To ensure that our performance results are statistically robust, we use a 5-fold cross-validation strategy. Following the approach proposed in Ekstrand et al., 2018, we split each user’s data so that 5 randomly selected items form the test set, and the remaining items compose the training set. This setup allows the model to consider sufficient training data while evaluating its ability to predict unseen items per user.

7.1.2. Fairness and Popularity Bias Evaluation

For the second part, as presented in Ekstrand et al., 2018, we study Bellogín’s UIR method (Bellogín et al., 2011), a training and evaluation method designed to obtain results that account for popularity bias. This method also uses a 5-fold cross-validation approach but differs in that the folds are created by items rather than users. Each test set is constructed so that every item is included exactly once as a test instance, regardless of how many ratings it has received. For each selected user-item pair, the recommender generates a ranked list in an attempt to recover the held-out item. This design ensures that all items are equally represented in the evaluation, preventing models from gaining an unfair advantage by over-recommending popular items that appear more frequently in the training data.

It is important to highlight that this methodology does not fall under the category of fairness-enhancing techniques in recommender systems, as presented in Section 6.3. Instead, it represents a new approach focused on modifying the training and evaluation procedures of recommendation models to mitigate the influence of popularity bias from the outset. This distinction underscores a shift from post-hoc fairness corrections to proactive bias-aware design in the evaluation pipeline.

7.1.3. Evaluation Metrics

As previously mentioned, the metrics we will use include the classical error measures MAE and RMSE, along with the recommender system–specific metrics defined in Section 2, such as novelty, serendipity, diversity, and relevance. In contrast, the paper by Ekstrand et al., 2018 employs a different metric focused on top- N recommendation evaluation that may provide more conclusive insights into the effects of popularity bias: the Normalized Discounted Cumulative Gain (nDCG).

Definition 7.1.1. *Normalized Discounted Cumulative Gain (nDCG) quantifies the expected utility a user derives from a ranked recommendation list by accounting for both the relevance of individual items and their positions in the list. Formally, for a recommendation list L generated for user u , nDCG is computed as*

$$nDCG_{L,u} = \frac{DCG_{L,u}}{IDCG_u},$$

where the Discounted Cumulative Gain (DCG) is defined as

$$DCG_{L,u} = \mu_u(l_1) + \sum_{i=2}^{|L|} \frac{\mu_u(l_i)}{\log_2 i}.$$

In the formula, l_i represents the i -th item in the list L , while $\mu_u(l_i)$ denotes the utility assigned by user u to item l_i .

The Ideal DCG (IDCG) corresponds to the DCG calculated on a list of the user’s rated items sorted in non-increasing order of utility, representing the optimal ranking.

In our experiments, we define $\mu_u(l_i)$ as the rating given by user u to movie l_i in the MovieLens 100k dataset. Items not rated by the user are assigned zero utility, a common practice despite its known limitations.

7.2. Dataset description & Exploratory Analysis

For this project, we use the **MovieLens 100k**¹ dataset, which consists of 100,000 explicit ratings collected from 943 users on 1,682 movies. The dataset is designed for research on recommender systems and contains three primary files that are relevant to our analysis.

1. u.data: This file contains the rating matrix in triplet format, with each line representing a tuple (user_id, item_id, rating, timestamp). The ratings are integer values ranging from 1 to 5. All users have rated at least 20 items, ensuring a minimum level of activity per user.

UserID	MovieID	Rating	Timestamp
196	242	3	881250949
186	302	3	891717742
22	377	1	878887116

2. u.item: This file provides item-level metadata. Each row includes the movie ID, title, release date, and a binary vector encoding membership across 19 distinct genres (e.g., *Action*, *Comedy*, *Drama*). A value of 1 in a given position indicates that the movie belongs to the corresponding genre. This information enables content-based filtering approaches.

MovieID	Title	Action	Children	Comedy	...	Sci-Fi	Western
1	Toy Story (1995)	0	1	1	...	0	0
2	GoldenEye (1995)	1	0	0	...	1	0
3	Four Rooms (1995)	0	0	0	...	1	0

3. u.user: This file contains demographic attributes for each user, such as user ID, age, gender, occupation, and zip code. For the purposes of our fairness analysis, we will con-

¹<https://grouplens.org/datasets/movielens/100k/>

sider gender and age as sensitive attributes when evaluating disparities in recommendation performance.

UserID	Age	Gender	Occupation	Zip-code
1	24	M	technician	85711
2	53	F	other	94043
3	23	M	writer	32067

We proceed with an initial exploratory data analysis (EDA) to characterize the structural properties and behavioral patterns of the ratings within the MovieLens 100k dataset. This analysis aims to identify underlying biases, popularity effects, and distributional characteristics of the ratings, which are critical factors that can influence the performance and fairness of subsequent recommendation models.

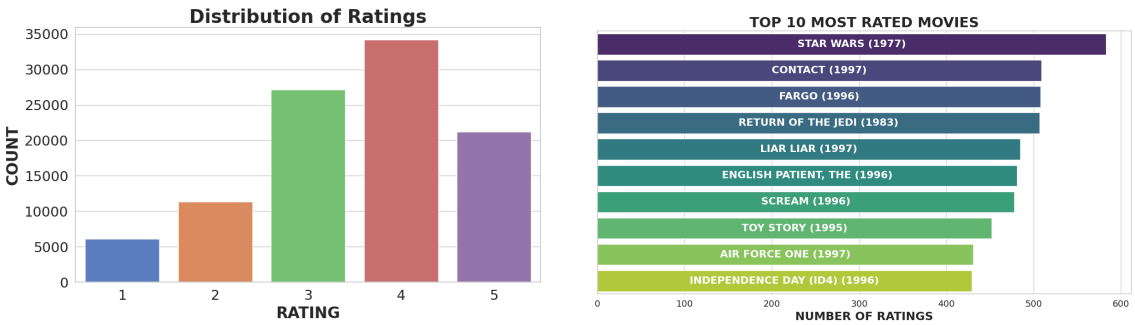


Figure 7.1
Exploratory Data Analysis (EDA) of the MovieLens 100k dataset. The plot on the left shows the distribution of ratings, while the plot on the right displays the top 10 most rated movies.

As we can observe in Figure 7.1, the majority of users tend to rate movies positively, with ratings between 3 and 5 stars being the most common. Among the most rated (and thus most popular) movies, Star Wars (1977) stands out prominently. Additionally, it is important to examine the distribution of the number of ratings per movie to assess whether these top 10 movies we previously highlighted also stand out significantly in terms of the volume of ratings compared to the rest. This analysis can help determine whether a clear case of popularity bias is present in the dataset.

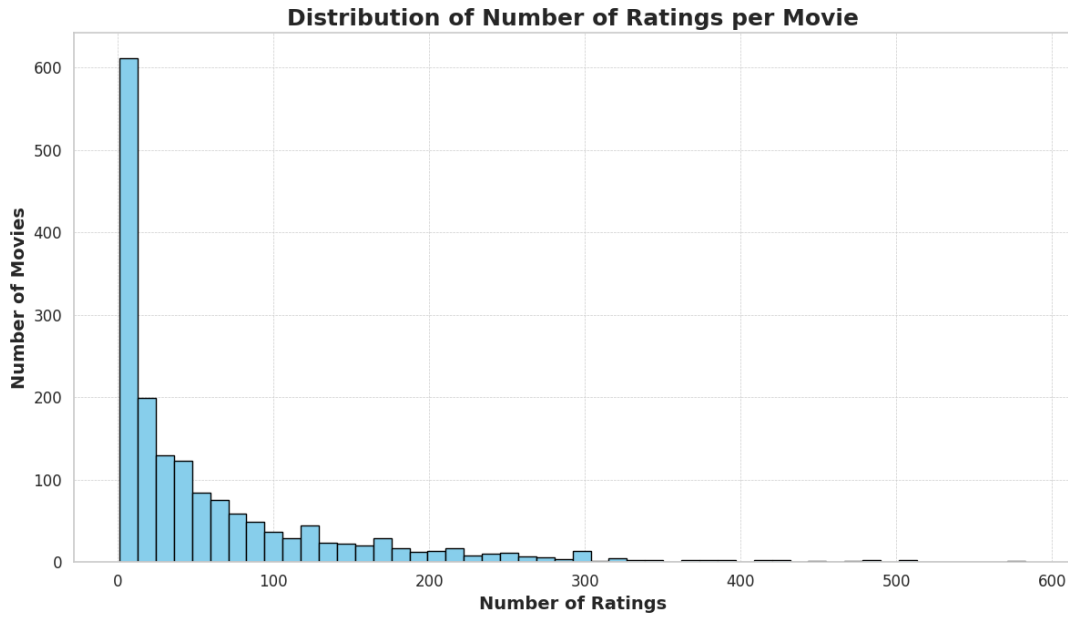


Figure 7.2

Distribution of ratings per movie in the MovieLens 100k dataset.

As we can observe in Figure 7.2, the majority of movies have fewer than 100 ratings. As the number of ratings increases, the number of movies receiving that level of attention decreases significantly. This long-tail distribution suggests that only a small subset of movies attracts the vast majority of user engagement, potentially amplifying the effect of popularity bias in recommendation systems.

7.3. Evaluation of Recommender System Performance

As mentioned at the beginning, this first section focuses on evaluating the different models we have developed previously.

Before evaluating the recommender system models we have been developing, it is essential to establish a set of benchmark models. These baselines serve as reference points and allow us to assess how much our proposed models improve over simple heuristics. By comparing against these benchmarks, we can better understand in which specific aspects, such as accuracy, novelty, or diversity, our models provide meaningful enhancements in prediction quality. The set of benchmark models considered is:

- **Global Mean:** Predicts all ratings with the average of all observed ratings in the training data.
- **Item Mean:** Predicts each item's ratings using the average rating that item has received across users.
- **User Mean:** Predicts each rating based on the average rating given by the corresponding user.

- **Random:** Assigns random values within the rating range; it serves as a sanity check baseline.
- **Popularity:** Recommends items proportionally to their popularity (number of ratings), ignoring user preferences.

Model	RMSE	MAE	Relevance	Novelty	Serendipity	Diversity	nDCG
Global Mean	1.131	0.952	3.316	0.065	0.952	0.844	0.173
Item Mean	1.046	0.835	4.368	0.294	0.836	1.000	0.158
User Mean	1.061	0.849	3.316	0.065	0.849	0.844	0.173
Random	1.720	1.406	3.295	0.113	1.406	0.914	0.180
Popularity	1.855	1.565	3.832	0.003	1.565	0.669	0.153

Table 7.1

Benchmark Models used as reference points to evaluate the performance of different recommendation algorithms throughout the study.

Among the benchmark models evaluated, predicting based on the mean rating of each item proves to be the most accurate. From Table 7.1, we observe that this model achieves the lowest MAE of 0.835 and an RMSE of 1.046, indicating that leveraging item-level preferences offers a strong baseline for accuracy. On the other hand, the popularity-based recommender, which solely considers how frequently an item has been rated and ignores the actual rating values, yields the worst predictive performance with an MAE of 1.565 and RMSE of 1.855. Furthermore, as expected, this model exhibits the lowest novelty score across all benchmarks, with a value of just 0.003. This is not surprising since the popularity-based approach tends to recommend the most rated and thus most known items, failing to suggest less obvious or less popular alternatives. Consequently, while popularity-driven recommendations may align with mainstream user interest, they lack the capacity to promote discovery or personalization.

Now that we have established the benchmarks as our starting point, it is time to begin working with the recommender system models.

The first model we consider is the one described in Section 3.1.1, which corresponds to user-based models. For this, we evaluate these models by considering different values of k , indicating how many neighboring users will be taken into account when making predictions. Additionally, we analyze the different similarity metrics previously described, such as Pearson similarity (Eq. (3.2)), raw cosine similarity (Eq. (3.3)), and discounted Pearson similarity (Eq. (3.4)) for various values of the parameter β .

Model	Similarity	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	Pearson	0.980	0.772	50	0.381	3.750	0.772	0.883	0.162

Continued on next page

Model	Similarity	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	Pearson	0.975	0.767	150	0.382	3.500	0.767	0.883	0.161
CF	Pearson	0.976	0.769	500	0.382	3.333	0.769	0.883	0.161
CF	Pearson	0.976	0.769	all	0.382	3.333	0.769	0.883	0.161

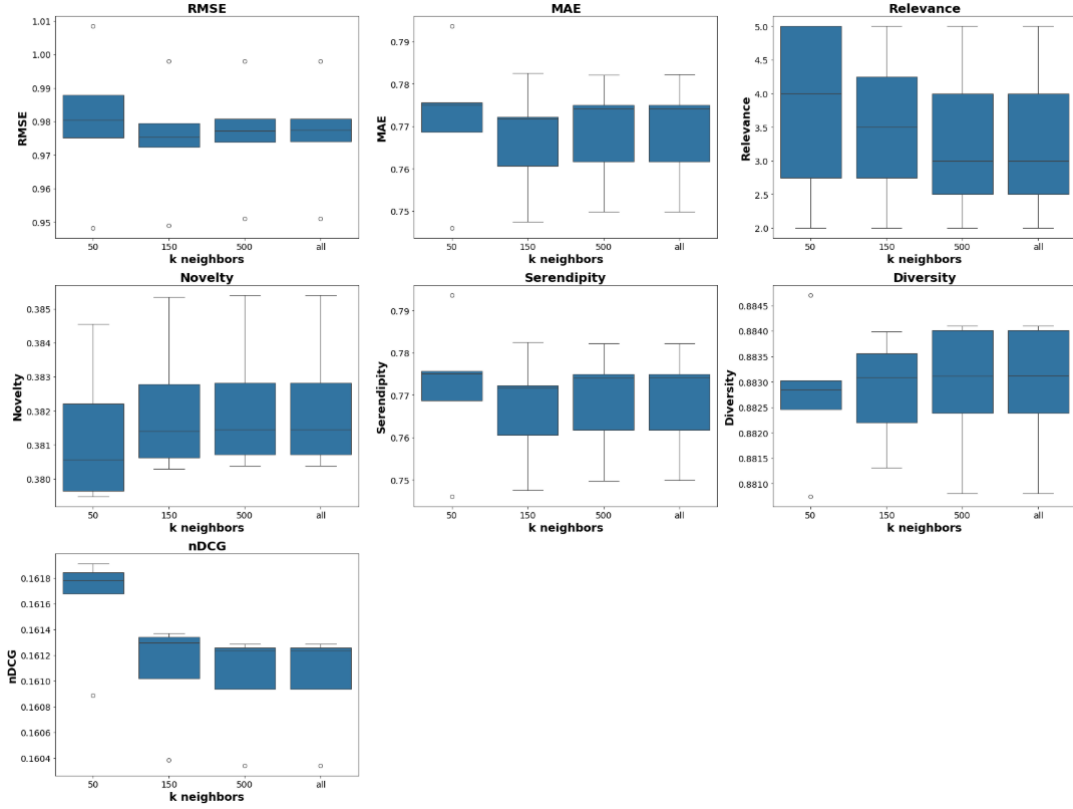
Table 7.2

Results of the user-based models with Pearson similarity for different values of k (number of neighbors), showing the impact of neighborhood size on recommendation accuracy.

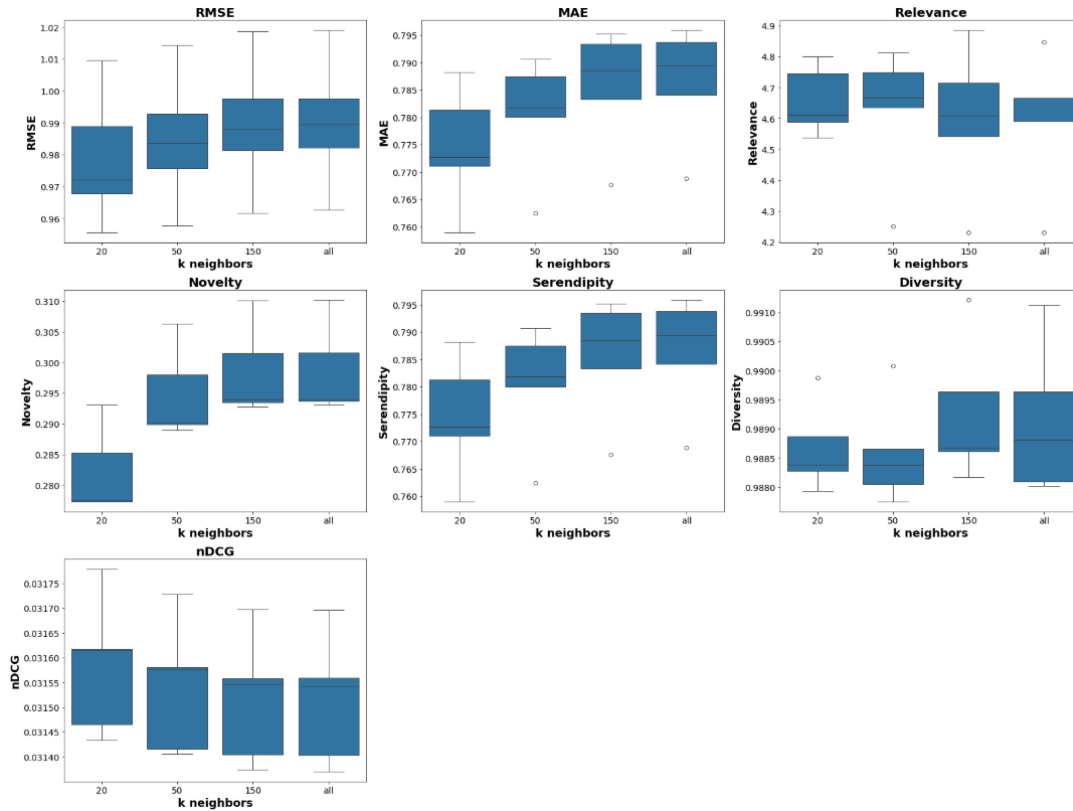
Model	Similarity	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	Cosine	1.032	0.820	50	0.274	4.461	0.820	0.889	0.157
CF	Cosine	1.037	0.826	150	0.280	4.450	0.826	0.891	0.157
CF	Cosine	1.041	0.830	500	0.282	4.483	0.830	0.889	0.157
CF	Cosine	1.041	0.830	all	0.282	4.472	0.830	0.889	0.157

Table 7.3

Results of the user-based models with Cosine similarity for different values of k (number of neighbors), showing the impact of neighborhood size on recommendation accuracy.



(a) Boxplot displaying the distribution of results obtained with user-based models using Pearson similarity, illustrating the impact of different values of k (number of neighbors) on performance.



(b) Boxplot displaying the distribution of results obtained with user-based models using Cosine similarity, illustrating the impact of different values of k (number of neighbors) on performance.

Figure 7.3

Comparison of boxplots for user-based models with Pearson and Cosine similarity across the different defined metrics.

When comparing the results of the Pearson and Cosine similarity measures, two of the most basic and widely used metrics, as shown in **Tables 7.2** and **7.3**, we observe several notable differences. The Pearson similarity consistently achieves lower RMSE and MAE values, indicating better predictive accuracy. It also obtains higher nDCG scores, suggesting more relevant ranking of recommendations.

By contrast, when focusing on relevance, serendipity, and diversity, Cosine similarity achieves notable improvements over Pearson similarity, indicating that Cosine is better at capturing diverse and surprising recommendations that may be more engaging for users.

Finally, focusing on the nDCG metric, we observe that neither model surpasses the benchmark results presented earlier. This suggests that in this case, nDCG may not be the most appropriate measure to evaluate the performance of these recommendation models, and alternative metrics should be considered to capture their practical effectiveness more accurately.

Model	Similarity	Beta	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	Disc. Pear.	30	0.960	0.751	50	0.374	4.696	0.751	0.879	0.161
CF	Disc. Pear.	30	0.963	0.755	150	0.379	4.700	0.755	0.879	0.161
CF	Disc. Pear.	30	0.965	0.759	500	0.379	5.000	0.759	0.879	0.161
CF	Disc. Pear.	30	0.965	0.759	all	0.379	5.000	0.759	0.879	0.161
CF	Disc. Pear.	100	0.961	0.750	50	0.367	4.558	0.750	0.880	0.161
CF	Disc. Pear.	100	0.962	0.753	150	0.374	4.733	0.753	0.880	0.161
CF	Disc. Pear.	100	0.964	0.756	500	0.375	5.000	0.756	0.880	0.161
CF	Disc. Pear.	100	0.964	0.756	all	0.375	5.000	0.756	0.880	0.161
CF	Disc. Pear.	300	0.961	0.750	50	0.365	4.558	0.750	0.880	0.161
CF	Disc. Pear.	300	0.962	0.753	150	0.372	4.733	0.753	0.880	0.161
CF	Disc. Pear.	300	0.964	0.756	500	0.373	5.000	0.756	0.880	0.161
CF	Disc. Pear.	300	0.964	0.756	all	0.373	5.000	0.756	0.880	0.161

Table 7.4

Results of the collaborative filtering models using the discounted Pearson similarity for different values of β (discounting factor) and k (number of neighbors considered).

Table 7.4 presents the performance of the collaborative filtering approach using the Discounted Pearson similarity measure across different values of the discounting parameter β . This parameter modulates the contribution of each neighbor by penalizing less similar users, thereby emphasizing the top-ranked similarities in the aggregation process.

The results indicate that variations in β exert a limited influence on the primary accuracy metrics (RMSE and MAE), which remain relatively stable regardless of the value of k . However, as β increases, a consistent decline in novelty is observed, especially at lower values of k , suggesting a tendency towards recommending more predictable or mainstream items. Despite this trade-off, the metrics for relevance, serendipity, diversity, and nDCG remain largely unaffected, implying that the Discounted Pearson similarity preserves the overall recommendation quality while enabling finer control over the balance between exploration and exploitation.

In comparison to the standard Pearson and Cosine similarity measures shown in **Tables 7.2** and **7.3**, the Discounted Pearson metric achieves competitive accuracy and maintains comparable performance in terms of diversity and nDCG, while offering an additional degree of tunability through β . This makes it a promising candidate for scenarios requiring adaptable recommendation behavior without sacrificing core performance.

As previously discussed in Section 3.1.2, we can also consider the relationships between items to improve prediction accuracy. For this purpose, we will employ item-based collaborative filtering models using two similarity measures: standard cosine similarity (Table 7.5) and adjusted cosine similarity (Table 7.6).

Model	Similarity	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF-item	Cosine	0.949	0.744	20	0.290	4.516	0.744	0.944	0.032
CF-item	Cosine	0.959	0.754	50	0.296	4.560	0.754	0.938	0.031
CF-item	Cosine	0.966	0.760	150	0.299	4.519	0.760	0.935	0.032
CF-item	Cosine	0.967	0.761	all	0.299	4.456	0.761	0.935	0.032

Table 7.5

Results of the collaborative filtering item-based model using cosine similarity, evaluated for different values of k (number of neighbors).

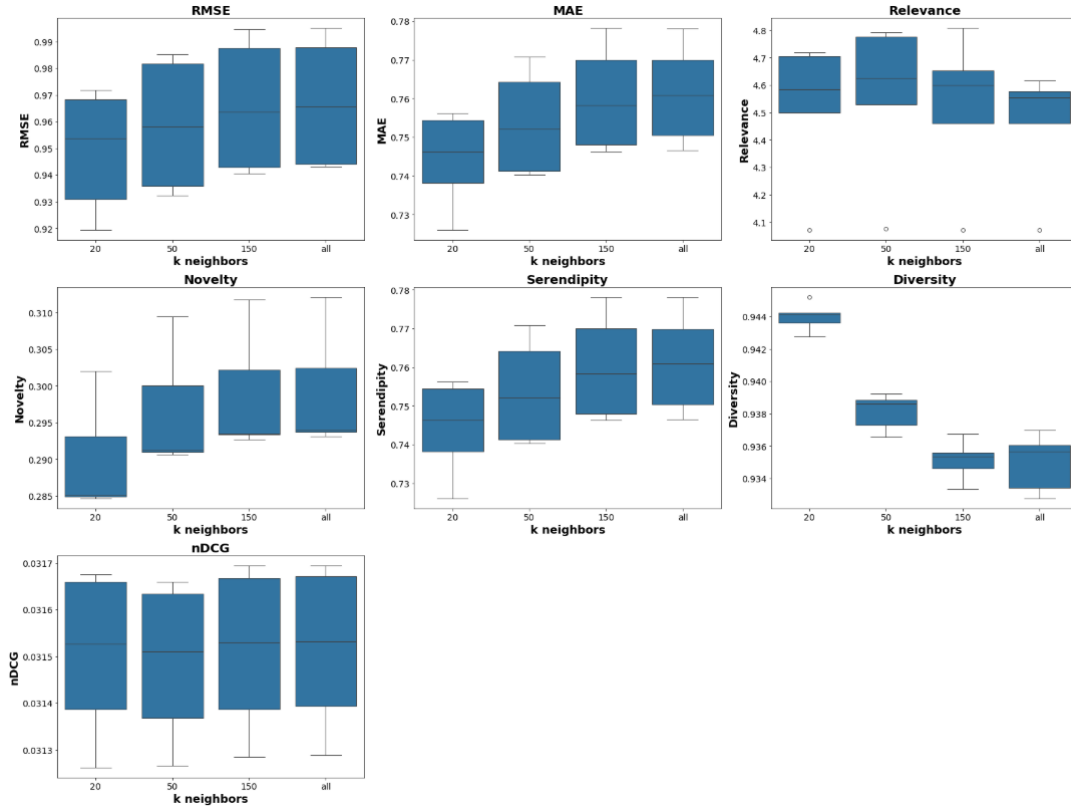
Model	Similarity	RMSE	MAE	k	Novelty	Relevance	Serendipity	Diversity	nDCG
CF-item	Adj. Cosine	0.979	0.774	20	0.282	4.657	0.774	0.989	0.032
CF-item	Adj. Cosine	0.985	0.780	50	0.295	4.623	0.780	0.989	0.032
CF-item	Adj. Cosine	0.989	0.786	150	0.298	4.596	0.786	0.989	0.032
CF-item	Adj. Cosine	0.990	0.786	all	0.299	4.585	0.786	0.989	0.032

Table 7.6

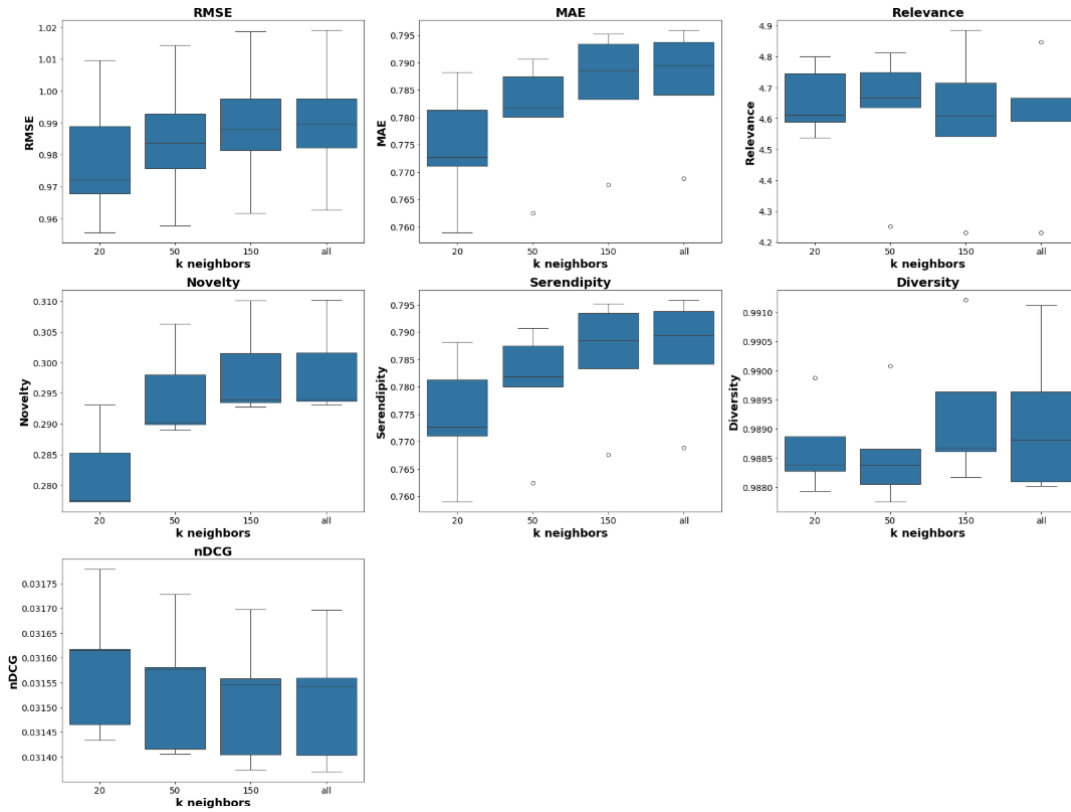
Results of the collaborative filtering item-based model using Adjusted cosine similarity, evaluated for different values of k (number of neighbors).

We want to highlight that the item-based collaborative filtering approach achieves slightly better results compared to the user-based method. This improvement is mainly because item-based models capture the relationships between items more effectively, which often leads to more accurate predictions, especially in scenarios where user preferences are sparse or diverse.

Note: However, it is important to note that these models come with a higher computational cost. Specifically, we move from calculating similarities using a user-user matrix of size approximately 900 x 900 to an item-item similarity matrix of about 1600 x 1600. This increase in matrix size results in roughly double the computation time, which can impact scalability and efficiency in larger datasets.



(a) Boxplot displaying the distribution of results obtained with item-based models using Cosine similarity, illustrating the impact of different values of k (number of neighbors) on performance.



(b) Boxplot displaying the distribution of results obtained with item-based models using Adjusted cosine similarity, illustrating the impact of different values of k (number of neighbors) on performance.

Figure 7.4

Comparison of boxplots for item-based Cosine and Adjusted Cosine similarity across the previously defined metrics.

Now, moving on to the model-based methods described in Section 3.2, these approaches have the advantage of being able to capture and exploit complex non-linear relationships within the data that may not be immediately evident. This ability often translates into improved predictive performance, especially in terms of metrics such as RMSE and MAE. However, a key limitation arises from the high sparsity of our rating matrix, which may significantly hinder the model’s ability to generalize, leading to a substantial loss in both precision and overall effectiveness.

We will focus first on decision tree models, specifically the XGBoost algorithm. XGBoost has been chosen mainly due to its ability to handle missing values (NA) effectively. As previously mentioned, our rating matrix is highly sparse, meaning that a large number of entries are missing. Therefore, it is crucial to use a method that is well-suited for dealing with incomplete data, and XGBoost proves to be an appropriate choice in this context.

Model	n_est.	max_depth	lr	RMSE	MAE	Novelty	Serendipity	Relevance	Diversity	nDCG
XGBoost	50	3	0.1	1.031	0.805	0.261	0.805	4.314	0.986	0.032
XGBoost	50	3	0.5	1.072	0.842	0.255	0.842	4.392	0.809	0.032
XGBoost	50	5	0.1	1.065	0.827	0.254	0.827	4.117	0.969	0.032
XGBoost	50	5	0.5	1.104	0.859	0.246	0.859	4.314	0.776	0.033
XGBoost	50	10	0.1	1.092	0.850	0.244	0.850	4.483	0.971	0.032
XGBoost	50	10	0.5	1.132	0.882	0.243	0.882	4.374	0.775	0.032
XGBoost	100	3	0.1	1.034	0.803	0.270	0.803	4.250	0.944	0.032
XGBoost	100	3	0.5	1.060	0.828	0.253	0.828	4.439	0.793	0.032
XGBoost	100	5	0.1	1.040	0.805	0.258	0.805	4.547	0.906	0.032
XGBoost	100	5	0.5	1.079	0.844	0.247	0.844	4.614	0.786	0.032
XGBoost	100	10	0.1	1.094	0.845	0.256	0.845	4.605	0.885	0.032
XGBoost	100	10	0.5	1.130	0.875	0.238	0.875	4.582	0.767	0.032
XGBoost	200	3	0.1	1.035	0.806	0.263	0.806	4.450	0.898	0.032
XGBoost	200	3	0.5	1.081	0.841	0.249	0.841	4.440	0.791	0.032
XGBoost	200	5	0.1	1.040	0.808	0.261	0.808	4.820	0.858	0.032
XGBoost	200	5	0.5	1.093	0.851	0.244	0.851	4.562	0.781	0.032
XGBoost	200	10	0.1	1.086	0.841	0.257	0.841	4.870	0.854	0.033
XGBoost	200	10	0.5	1.133	0.872	0.237	0.872	4.608	0.776	0.032
XGBoost	500	3	0.1	1.037	0.807	0.263	0.807	4.837	0.847	0.032
XGBoost	500	3	0.5	1.076	0.839	0.256	0.839	4.579	0.796	0.033
XGBoost	500	5	0.1	1.079	0.838	0.260	0.838	4.860	0.835	0.032
XGBoost	500	5	0.5	1.102	0.861	0.244	0.861	4.502	0.786	0.032
XGBoost	500	10	0.1	1.094	0.844	0.257	0.844	4.244	0.851	0.032
XGBoost	500	10	0.5	1.119	0.868	0.235	0.868	4.312	0.777	0.032

Table 7.7

XGBoost per-item model results with cross-validation for different number of estimators, maximum tree depth, and learning rate values.

The XGBoost model fails to produce relevant results (Table 7.7), with RMSE, MAE, and nDCG values that are only comparable to those of the benchmark. This underperformance is largely attributed to the high sparsity of the user-item interaction matrix, which contains a large number of missing values. Such sparsity severely limits the model’s ability to extract informative patterns, leading to shallow and fragmented decision trees that struggle to generalize. As a result, XGBoost is unable to compete with user-based or item-based collaborative filtering methods, which are inherently more robust to data sparsity.

Lastly, we will study the latent factor models presented in Section 3.3. We begin by showing the results of the unconstrained optimization approach (Table 7.8), where no restrictions were imposed on the matrix product during factorization. The second approach involves using the Singular Value Decomposition (SVD) of the ratings matrix, which allowed us to obtain a low-rank approximation of the data more efficiently. This not only reduced computational complexity but also significantly improved the time required for training and evaluation processes.

Model	Latent Factors (k)	Alpha	Lambda	RMSE	MAE	Diversity	Novelty	Serendipity	Relevance	nDCG
Matrix Fact.	3	0.001	0.01	0.989	0.779	0.710	0.015	0.779	4.323	0.156
Matrix Fact.	3	0.001	0.05	0.994	0.786	0.641	0.015	0.786	4.303	0.156
Matrix Fact.	3	0.005	0.01	0.962	0.749	0.832	0.124	0.749	4.314	0.158
Matrix Fact.	3	0.005	0.05	0.952	0.749	0.878	0.148	0.749	4.444	0.158
Matrix Fact.	5	0.001	0.01	0.988	0.778	0.712	0.015	0.778	4.347	0.156
Matrix Fact.	5	0.001	0.05	0.993	0.785	0.672	0.015	0.785	4.286	0.156
Matrix Fact.	5	0.005	0.01	0.958	0.751	0.774	0.093	0.751	4.349	0.158
Matrix Fact.	5	0.005	0.05	0.947	0.745	0.838	0.132	0.745	4.387	0.157
Matrix Fact.	10	0.001	0.01	0.984	0.775	0.702	0.015	0.775	4.335	0.156
Matrix Fact.	10	0.001	0.05	0.991	0.783	0.711	0.015	0.783	4.322	0.156
Matrix Fact.	10	0.005	0.01	0.971	0.755	0.732	0.058	0.755	4.276	0.158
Matrix Fact.	10	0.005	0.05	0.946	0.743	0.785	0.112	0.743	4.429	0.157
Matrix Fact.	20	0.001	0.01	0.979	0.771	0.686	0.015	0.771	4.382	0.156
Matrix Fact.	20	0.001	0.05	0.988	0.781	0.705	0.015	0.781	4.324	0.156
Matrix Fact.	20	0.005	0.01	1.010	0.784	0.705	0.033	0.784	4.181	0.159
Matrix Fact.	20	0.005	0.05	0.948	0.744	0.750	0.096	0.744	4.359	0.158

Table 7.8

Performance metrics of the Unconstrained Matrix Factorization model for different values of k (latent factors), α (learning rate), and λ (regularization parameter).

The unconstrained matrix factorization model outperforms the baseline item-based and user-based models, achieving lower MAE and RMSE values. This indicates a better fit in terms of rating prediction accuracy. However, despite these improvements in error metrics, the model does not yield a significant gain in terms of nDCG, suggesting that the enhancements in prediction accuracy do not necessarily translate into better recommendation ranking quality.

The second model employed to predict the Ratings Matrix uses an iterative Singular Value Decomposition (SVD) strategy to generate a low-rank approximation of the original data. This approach differs from unconstrained matrix factorization by explicitly centering the ratings and repeatedly reconstructing the missing entries through successive SVD computations.

In our implementation, the matrix is first centered by subtracting the mean rating of each user. Missing values are initially filled with zeros to allow decomposition. Truncated SVD is then applied to extract the top k latent components, which are used to reconstruct an approximation of the rating matrix. The missing entries are updated in each iteration with the values predicted from the reconstruction.

Unlike gradient-based models, this approach does not require hyperparameters such as learning rate or regularization terms. Instead, the only two main hyperparameters are the number of latent factors k and the number of iterative *steps*. These parameters control the dimensionality of the factorization and the convergence of the reconstruction process, respectively.

Model	Latent k	Steps	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
SVD fact.	3	20	0.963	0.755	0.009	4.262	0.755	0.635	0.050
SVD fact.	3	50	0.954	0.747	0.018	4.298	0.747	0.642	0.047
SVD fact.	3	100	0.954	0.746	0.034	4.344	0.746	0.659	0.046
SVD fact.	5	20	0.961	0.752	0.010	4.233	0.752	0.637	0.051
SVD fact.	5	50	0.961	0.751	0.019	4.273	0.751	0.662	0.047
SVD fact.	5	100	0.970	0.757	0.035	4.295	0.757	0.704	0.045
SVD fact.	10	20	0.967	0.756	0.013	4.239	0.756	0.652	0.051
SVD fact.	10	50	0.990	0.771	0.022	4.223	0.771	0.714	0.046
SVD fact.	10	100	1.019	0.788	0.032	4.215	0.788	0.781	0.043

Table 7.9

Evaluation results of the SVD factorization model across different latent factors and iteration steps.

The SVD model demonstrates considerable performance (Table 7.9), ranking among the best models we have evaluated. Notably, the training time required is minimal compared to other approaches, making it an excellent choice for scalability when handling large datasets. Its balance of accuracy and efficiency highlights SVD as a practical and effective method for recommender systems in real-world scenarios. On the other hand, when considering metrics such as nDCG, relevance, serendipity, diversity, and novelty, this model does not achieve significant improvements.

7.4. Evaluation of Bellogín’s U1R & U3R Models

This section focuses on the evaluation results of various models trained following the methodology proposed by Bellogín et al., 2011, as described in Section 7.1.2. In our study, we extend this approach by introducing an additional consideration aimed at achieving a broader understanding and more robust conclusions. The baseline method is U1R, which assumes a single rating per item. This approach provides models with a large amount of training data but results in fewer data points for testing.

We propose an alternative perspective by considering three ratings per item, referred to as U3R. This adjustment aims to produce a more precise evaluation, although this means having less data to train the models. We apply this approach both to gather relevant statistical information and to analyze how this shift in rating consideration impacts model performance.

Before diving into the evaluation of our proposed models, it is essential to first examine how this new methodology impacts our benchmark models. This preliminary analysis will provide a general understanding of the effects of the modified evaluation approach and help us anticipate how it may influence the performance of the other models studied.

Model	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
Global Mean	1.358	1.126	0.073	3.633	1.126	0.845	0.124
Item Mean	1.185	0.951	1.000	2.452	–	1.000	0.117
User Mean	1.134	0.905	0.073	3.633	0.905	0.845	0.124
Random	1.748	1.429	0.125	2.692	1.429	0.916	0.123
Popularity	2.098	1.750	0.003	3.949	1.750	0.670	0.125

Table 7.10

Evaluation results of baseline models under the Bellogín-inspired methodology (U1R).

Model	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
Global Mean	1.316	1.089	0.075	3.326	1.089	0.852	0.133
Item Mean	1.174	0.928	1.000	2.987	–	1.000	0.126
User Mean	1.108	0.882	0.075	3.326	0.882	0.852	0.133
Random	1.714	1.399	0.138	3.071	1.399	0.918	0.134
Popularity	2.110	1.781	0.003	3.882	1.781	0.669	0.133

Table 7.11

Evaluation results of baseline models under the new proposed methodology (U3R).

Comparing the results of the benchmark models obtained using the standard 5-fold cross-validation (see Table 7.1) with those derived under the Bellogín-inspired evaluation protocols, namely U1R (Table 7.10) and U3R (Table 7.11), reveals several important insights.

One of the key observations is that both in U1R and U3R, the User Mean model consistently emerges as the best-performing benchmark in terms of accuracy (RMSE and MAE). This differs from the outcome observed in the standard cross-validation setup, where the Item Mean model yielded the best performance. This contrast highlights how the evaluation protocol itself can significantly impact model rankings.

Moreover, we observe a general degradation in performance metrics, particularly RMSE and MAE, when using the U1R and U3R protocols compared to standard cross-validation. This drop is expected and can be attributed to the popularity bias inherent in the traditional splitting method. In 5-fold cross-validation, popular items tend to appear more frequently in both training and test sets. This makes them easier to predict and artificially inflates the accuracy of models, especially those that leverage item popularity.

In contrast, the U1R and U3R protocols enforce a more uniform distribution by ensuring that only one or a few ratings per item are included in the test set. This increases the likelihood of evaluating on less popular or infrequent items, which are naturally harder to

predict and thus result in lower overall accuracy. However, this comes with the important benefit of reducing bias and ensuring a more realistic and generalizable evaluation.

In summary, although the U1R and U3R setups yield lower numerical performance, they provide a fairer assessment of how models handle the long tail of items. These protocols are therefore better suited for selecting models that can generalize well across the full spectrum of items, not just the most frequently rated ones.

Once again, following the same approach as before, and having now established the benchmark models as a starting point, it is time to evaluate the methods we previously explored. The goal is to reassess which models prove to be the most effective, this time under the new evaluation methodology designed to yield a more accurate and objective assessment by mitigating the impact of popularity bias.

We begin this evaluation phase by presenting the results of the U1R & U3R protocols. As before, we start with collaborative filtering neighbourhood models, including both user-based and item-based approaches. This will help us to observe how these traditional models perform under the new evaluation protocol and how sensitive they are to the presence of popularity bias in the data.

Model	k	Similarity	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	50	Pearson	1.053	0.830	0.380	3.340	0.830	0.908	0.120
CF	150	Pearson	1.054	0.831	0.381	3.340	0.831	0.908	0.120
CF	500	Pearson	1.055	0.831	0.381	3.340	0.831	0.908	0.120
CF	All	Pearson	1.055	0.831	0.381	3.340	0.831	0.908	0.120
CF	50	Cosine	1.172	0.935	0.269	4.137	0.935	0.881	0.117
CF	150	Cosine	1.176	0.938	0.275	4.033	0.938	0.882	0.119
CF	500	Cosine	1.176	0.938	0.277	3.940	0.938	0.884	0.119
CF	All	Cosine	1.176	0.938	0.277	3.940	0.938	0.884	0.119

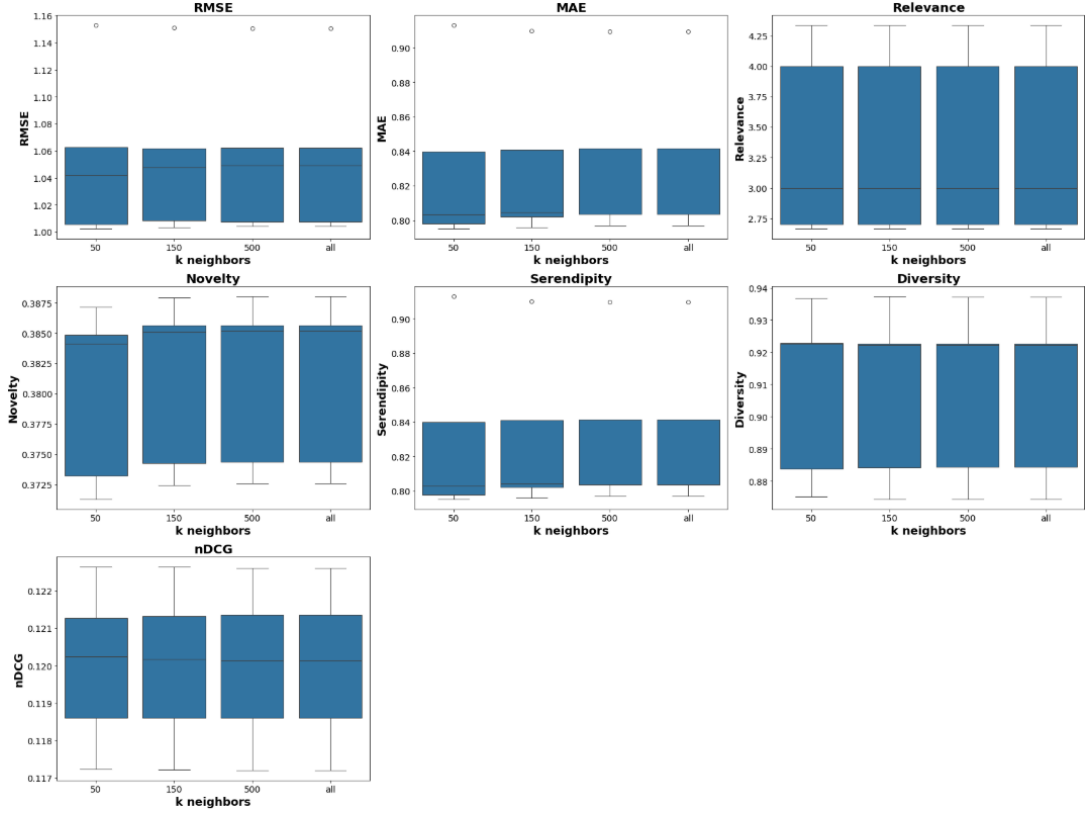
Table 7.12

Evaluation results of Memory-based Collaborative Filtering models for different values of k (neighbors) under the U1R protocol.

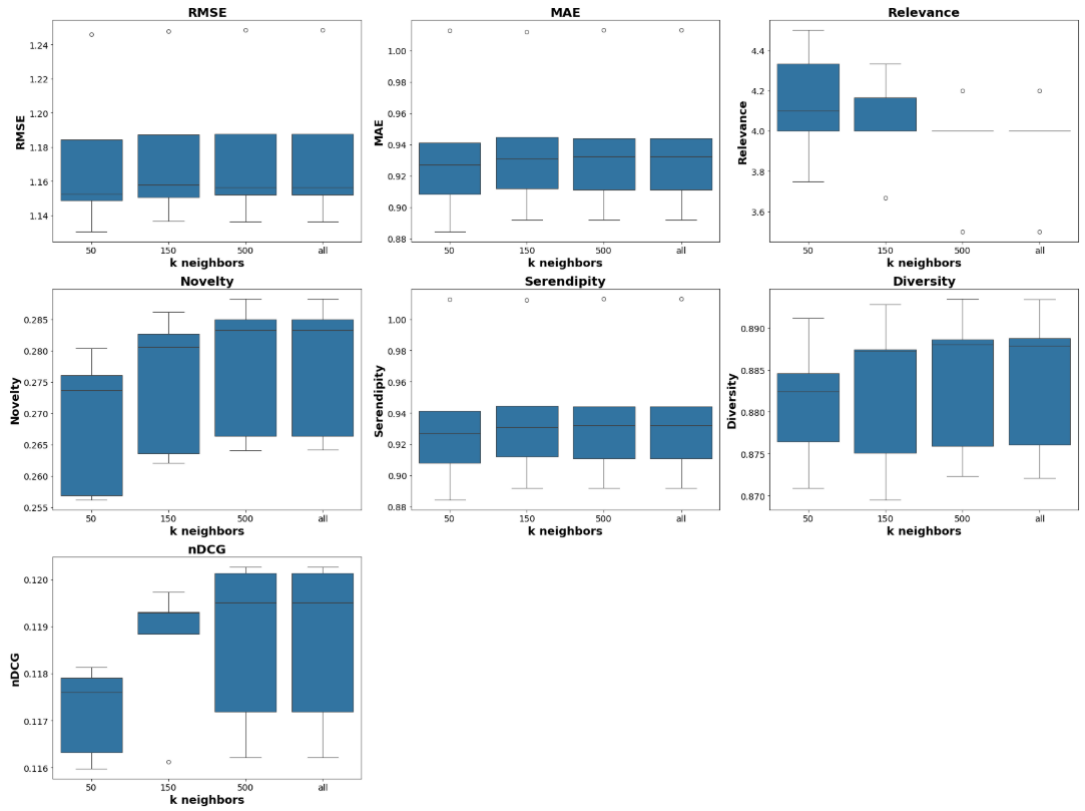
Model	k	Similarity	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
CF	50	Pearson	1.032	0.811	0.377	3.697	0.811	0.912	0.129
CF	150	Pearson	1.032	0.811	0.379	3.622	0.811	0.912	0.129
CF	500	Pearson	1.033	0.812	0.379	3.584	0.812	0.912	0.129
CF	All	Pearson	1.033	0.812	0.379	3.584	0.812	0.912	0.129
CF	50	Cosine	1.159	0.915	0.267	3.969	0.915	0.874	0.126
CF	150	Cosine	1.160	0.918	0.273	3.815	0.918	0.874	0.125
CF	500	Cosine	1.161	0.918	0.275	3.832	0.918	0.874	0.127
CF	All	Cosine	1.161	0.918	0.275	3.832	0.918	0.874	0.127

Table 7.13

Evaluation results of Memory-based Collaborative Filtering models for different values of k (neighbors) under the proposed U3R protocol.



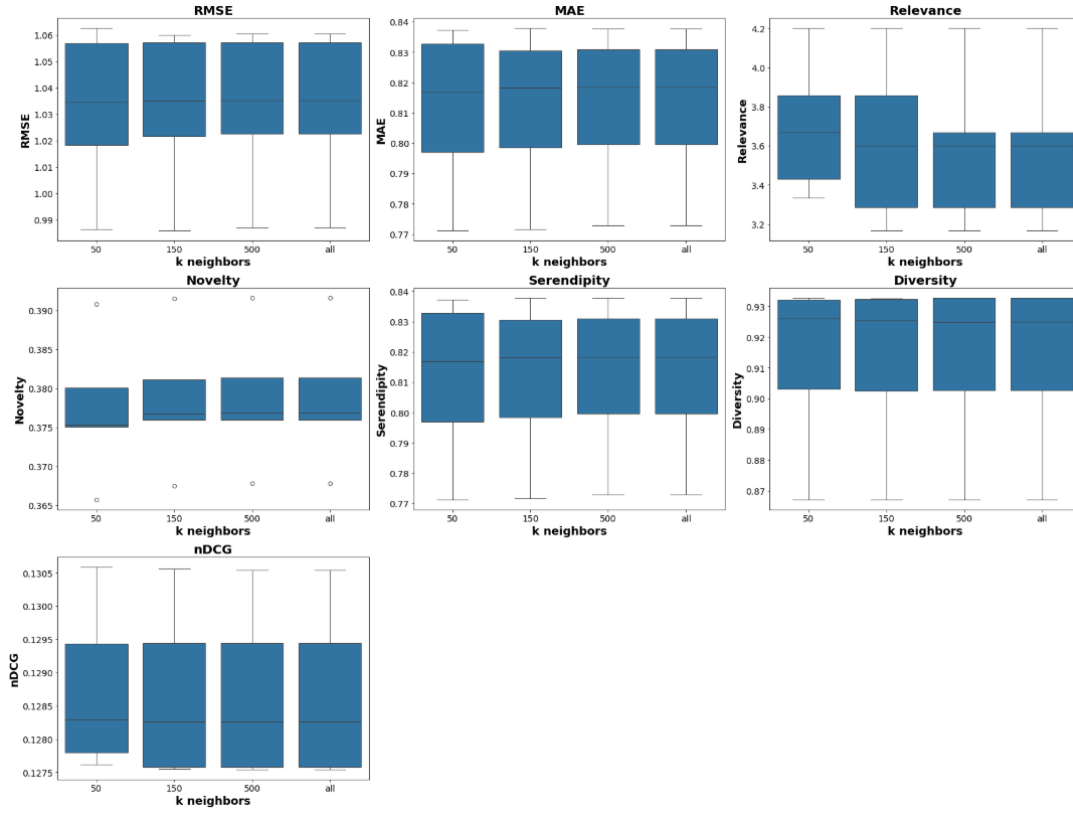
(a) Boxplot of results using Pearson similarity under the UIR evaluation protocol for different values of k (number of neighbors), illustrating performance variability.



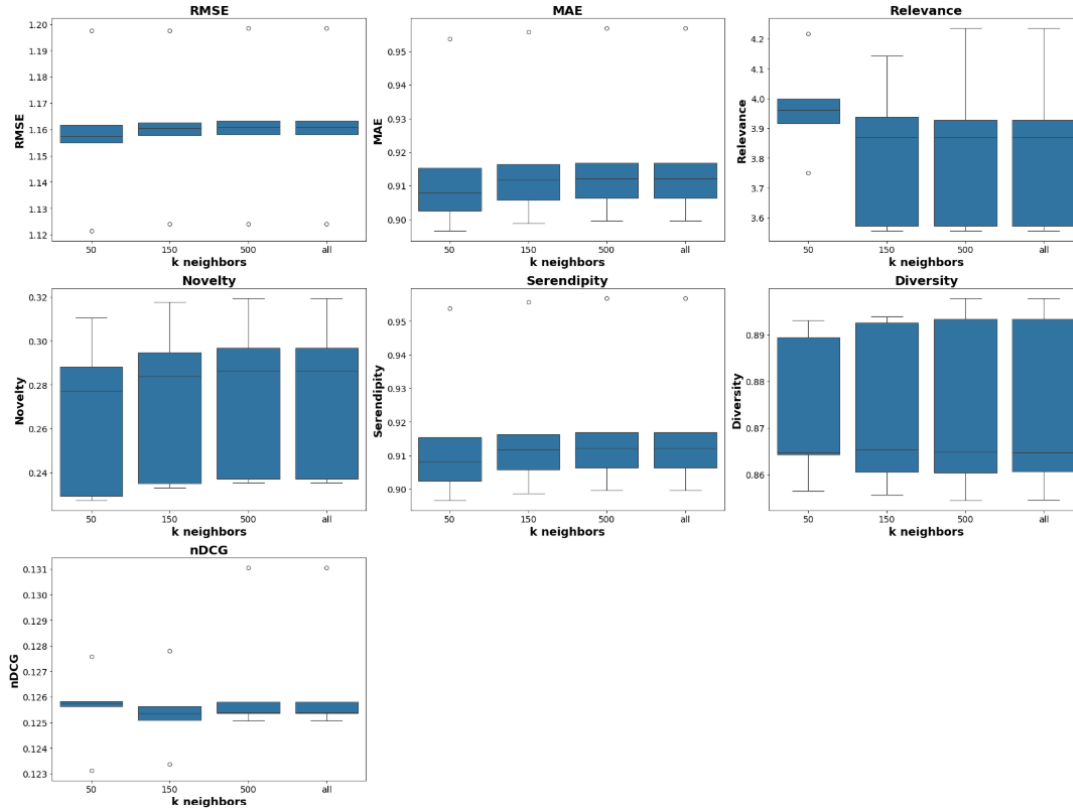
(b) Boxplot of results using Cosine similarity under the UIR evaluation protocol for different values of k (number of neighbors), illustrating performance variability.

Figure 7.5

Comparison of boxplots for Pearson and Cosine similarity under the UIR protocol.



(a) Boxplot of results using Pearson similarity under the U3R evaluation protocol for different values of k (number of neighbors), illustrating performance variability.



(b) Boxplot of results using Cosine similarity under the U3R evaluation protocol for different values of k (number of neighbors), illustrating performance variability.

Figure 7.6

Comparison of boxplots for Pearson and Cosine similarity under the U3R protocol.

Comparing the results of the user-based collaborative filtering models under the standard 5-fold cross-validation split (Tables 7.2 and 7.3) with those obtained using the U1R evaluation protocol (Table 7.12) and those obtained using the U3R evaluation protocol (Table 7.13), we observe the expected degradation in predictive accuracy. Specifically, both RMSE and MAE increase which is a direct consequence of predicting ratings for less popular (and thus harder to estimate) items. This effect stems from the fact that U1R & U3R forces the model to evaluate on a wider range of items, mitigating the popularity bias inherently present in standard evaluation splits.

Despite this decline in accuracy, the general trend regarding similarity functions remains consistent. Pearson-based models continue to outperform cosine-based models across most metrics. Notably, both in the U1R setup and in the U3R setup, the Pearson models still manage to beat the benchmark models in terms of RMSE and MAE, whereas cosine-based models do not consistently surpass this baseline. These results validate once again the robustness of Pearson similarity in user-based collaborative filtering, even under more demanding evaluation scenarios like U1R and U3R.

As before, we now proceed with the item-based approach of collaborative filtering in Tables 7.14 and 7.15.

Model	Similarity	k	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
CF-item	Cosine	20	1.254	0.945	0.279	4.222	0.945	0.944	0.029
CF-item	Cosine	50	1.262	0.957	0.285	4.242	0.957	0.937	0.029
CF-item	Cosine	150	1.273	0.968	0.287	4.102	0.968	0.933	0.029
CF-item	Cosine	All	1.276	0.971	0.287	4.060	0.971	0.934	0.029
CF-item	Adj. Cosine	20	1.325	1.013	0.272	3.940	1.013	0.988	0.029
CF-item	Adj. Cosine	50	1.322	1.014	0.284	4.131	1.014	0.988	0.030
CF-item	Adj. Cosine	150	1.328	1.024	0.287	4.219	1.024	0.988	0.029
CF-item	Adj. Cosine	All	1.331	1.028	0.287	4.200	1.028	0.988	0.030

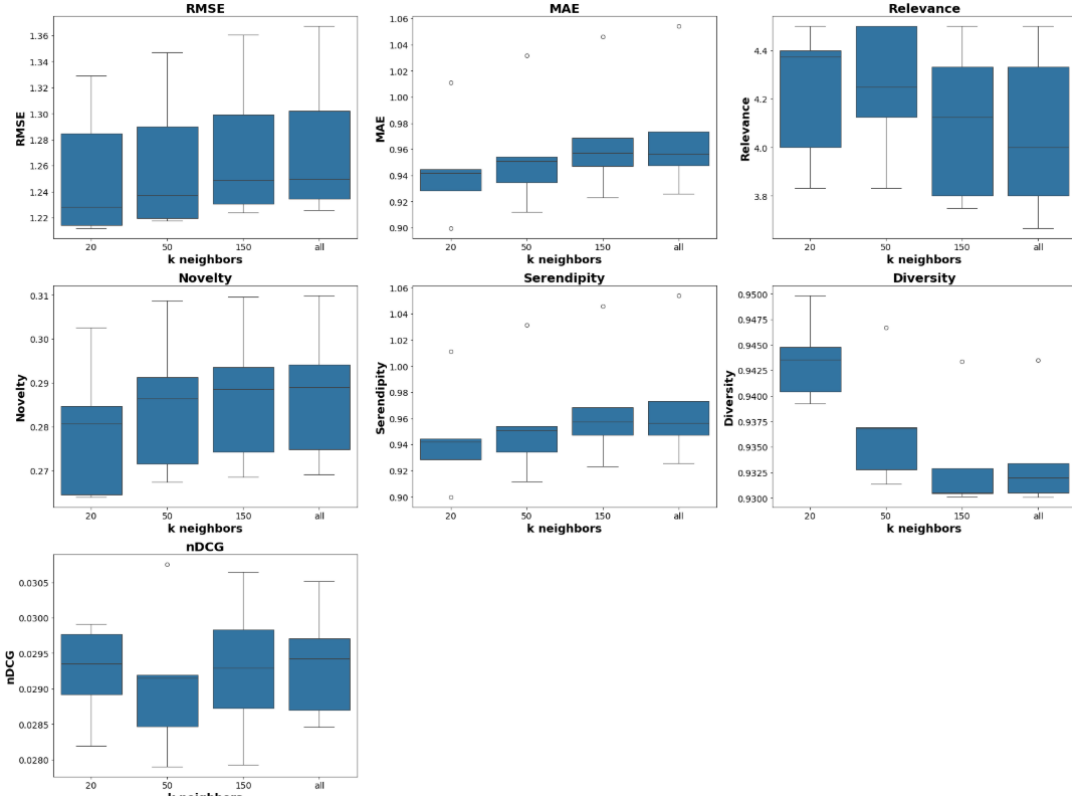
Table 7.14

Evaluation results of Item-based Collaborative Filtering models with Cosine and Adjusted Cosine similarity under the U1R protocol, for different values of k (number of neighbors).

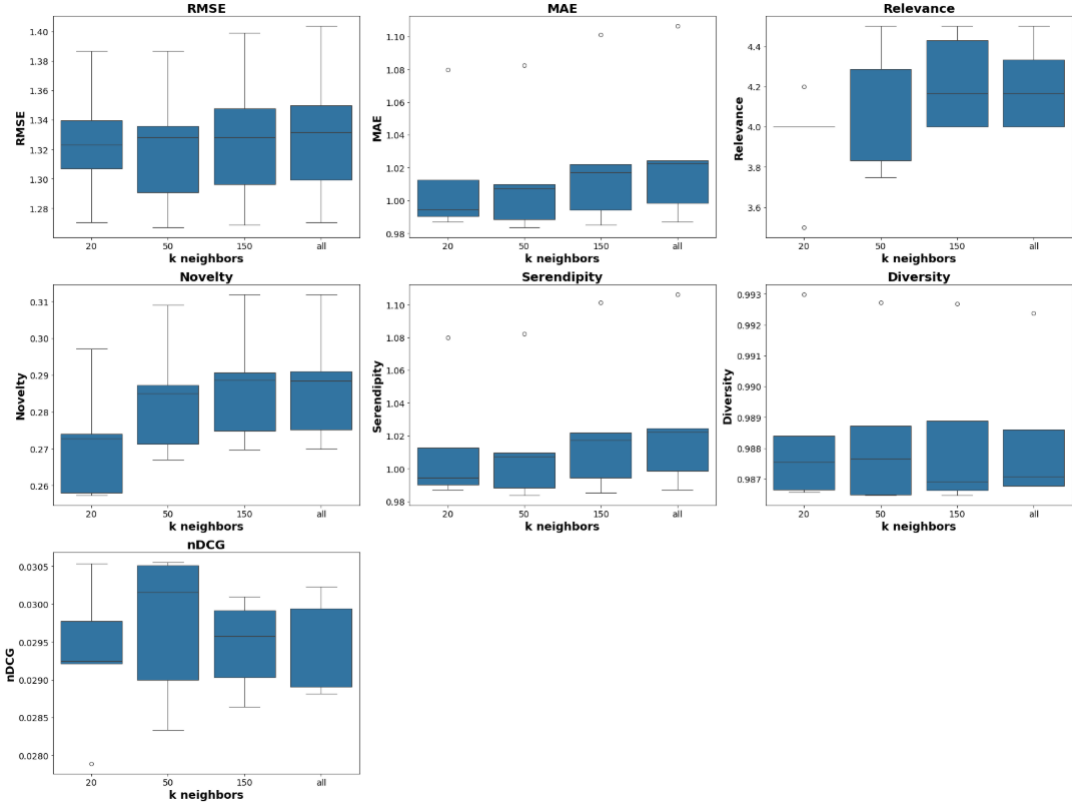
Model	Similarity	k	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
CF-item	Cosine	20	1.321	0.970	0.278	4.198	0.970	0.924	0.058
CF-item	Cosine	50	1.327	0.979	0.284	4.017	0.979	0.919	0.058
CF-item	Cosine	150	1.336	0.990	0.286	3.894	0.990	0.916	0.059
CF-item	Cosine	All	1.338	0.993	0.287	3.840	0.993	0.916	0.059
CF-item	Adj. Cosine	20	1.382	1.035	0.270	4.040	1.035	0.973	0.058
CF-item	Adj. Cosine	50	1.387	1.043	0.282	3.994	1.043	0.974	0.058
CF-item	Adj. Cosine	150	1.395	1.052	0.286	3.971	1.052	0.973	0.059
CF-item	Adj. Cosine	All	1.397	1.054	0.286	3.943	1.054	0.973	0.059

Table 7.15

Evaluation results of Item-based Collaborative Filtering models with Cosine and Adjusted Cosine similarity under the U3R protocol, for different values of k (number of neighbors).



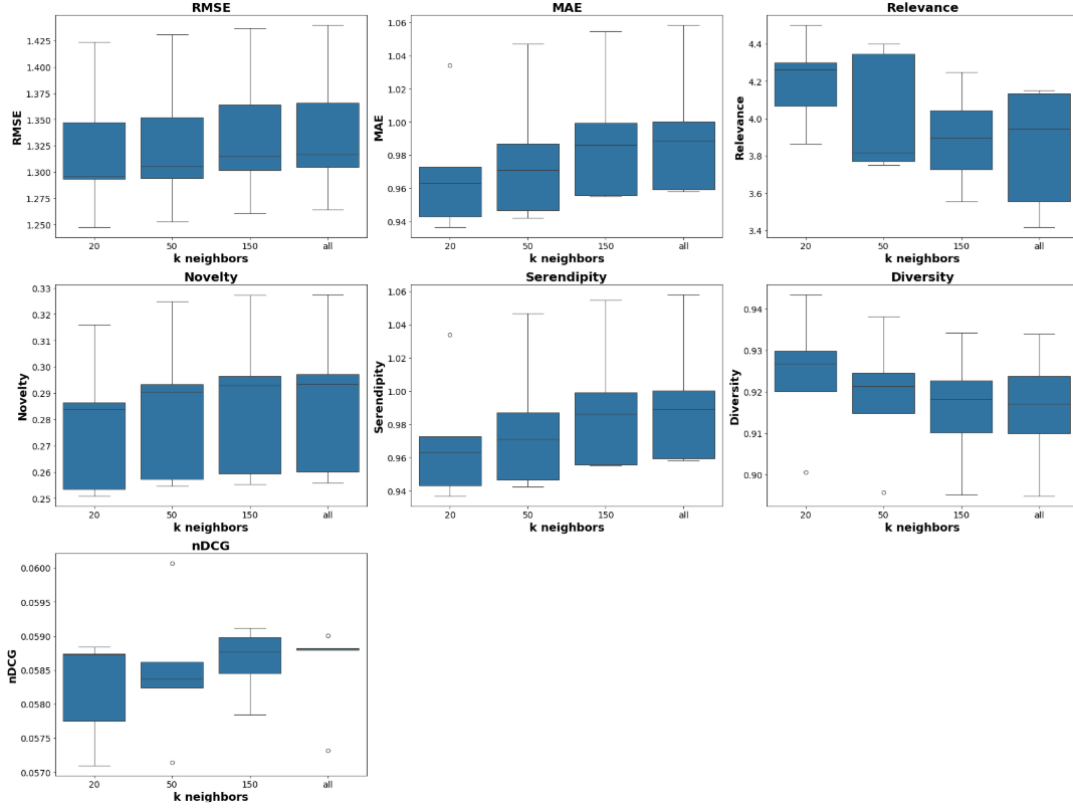
(a) Boxplot of the results for Item-based Collaborative Filtering using Cosine similarity, across different values of k (number of neighbors) under UIR protocol.



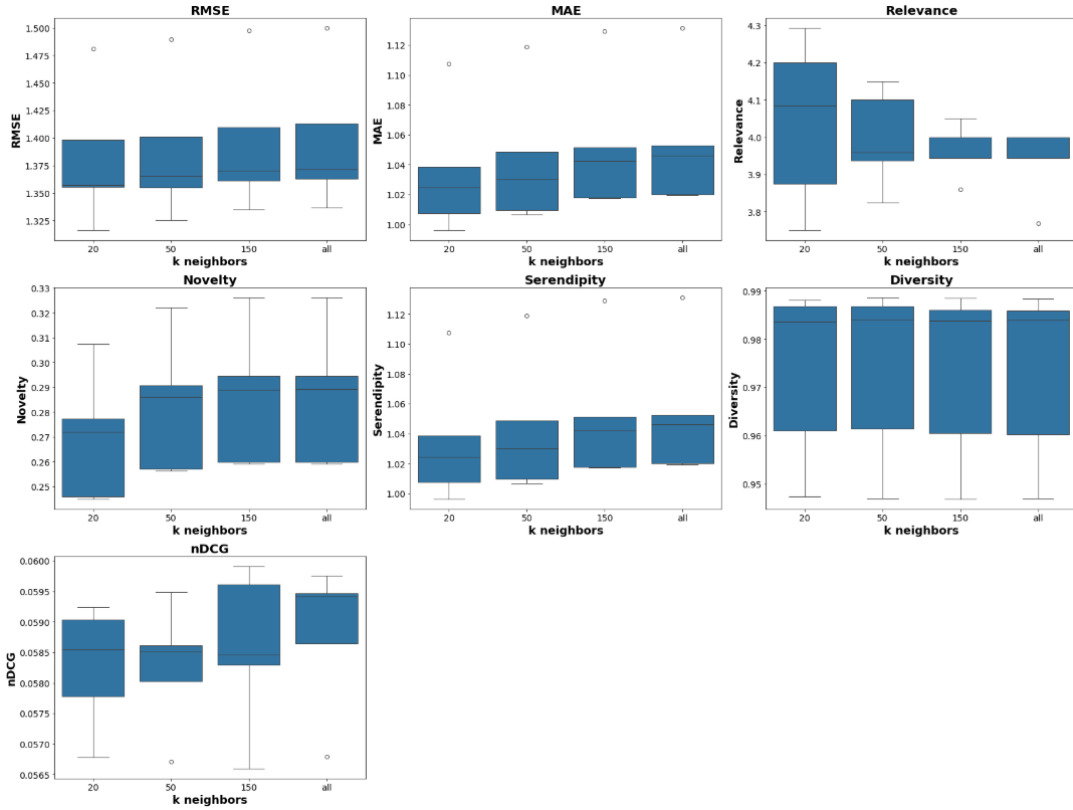
(b) Boxplot of the results for Item-based Collaborative Filtering using Adjusted Cosine similarity, across different values of k (number of neighbors) under UIR protocol.

Figure 7.7

Boxplot comparison of Item-based CF models using Cosine and Adjusted Cosine similarity for UIR



(a) Boxplot of the results for Item-based Collaborative Filtering using Cosine similarity, across different values of k (number of neighbors) under U3R protocol.



(b) Boxplot of the results for Item-based Collaborative Filtering using Adjusted Cosine similarity, across different values of k (number of neighbors) under U3R protocol.

Figure 7.8

Boxplot comparison of Item-based CF models using Cosine and Adjusted Cosine similarity for U3R

Surprisingly, item-based collaborative filtering models evaluated under both protocols show significantly worse performance compared to the results obtained using standard five-fold cross-validation. This performance drop is especially notable because item-based models had previously performed well when evaluated using the traditional approach.

A major factor behind this performance drop is the nature of our dataset, where many items have only a small number of ratings. The U1R & U3R protocol requires that each item appears exactly once (or three times) in the test set. As a result, even items with very limited interaction history must be included in the evaluation. This forces the model to make predictions for items about which it has very little information, leading to a significant decrease in performance.

This highlights a clear limitation of item-based collaborative filtering in scenarios where many items are sparsely rated. It also **emphasizes the importance of choosing evaluation protocols that are well aligned with the underlying data distribution** in order to ensure meaningful and fair model assessment.

To conclude, we now present the results obtained using the latent factor models. Specifically, we focus on the unconstrained matrix factorization model (Tables 7.16 and 7.17) and the SVD-based optimization model (Tables 7.18 and 7.19), as these were the two approaches introduced earlier that achieved the best overall performance across our evaluation metrics.

Model	Latent k	α	λ	RMSE	MAE	Diversity	Novelty	Serendipity	Relevance	nDCG
Matrix Fact.	3	0.001	0.01	1.398	1.054	0.686	0.016	1.054	4.329	0.125
Matrix Fact.	3	0.001	0.05	1.405	1.062	0.612	0.015	1.062	4.328	0.126
Matrix Fact.	3	0.005	0.01	1.267	0.942	0.834	0.120	0.942	3.971	0.125
Matrix Fact.	3	0.005	0.05	1.261	0.941	0.871	0.142	0.941	4.011	0.125
Matrix Fact.	5	0.001	0.01	1.396	1.053	0.684	0.016	1.053	4.170	0.125
Matrix Fact.	5	0.001	0.05	1.404	1.061	0.673	0.015	1.061	4.325	0.126
Matrix Fact.	5	0.005	0.01	1.280	0.957	0.768	0.092	0.957	3.990	0.125
Matrix Fact.	5	0.005	0.05	1.263	0.942	0.829	0.128	0.942	4.038	0.126
Matrix Fact.	10	0.001	0.01	1.395	1.051	0.672	0.015	1.051	4.233	0.126
Matrix Fact.	10	0.001	0.05	1.402	1.059	0.684	0.015	1.059	4.323	0.126
Matrix Fact.	10	0.005	0.01	1.305	0.971	0.730	0.058	0.971	3.953	0.124
Matrix Fact.	10	0.005	0.05	1.261	0.938	0.782	0.108	0.938	4.267	0.124
Matrix Fact.	20	0.001	0.01	1.391	1.048	0.668	0.015	1.048	4.097	0.126
Matrix Fact.	20	0.001	0.05	1.400	1.057	0.680	0.015	1.057	4.336	0.125
Matrix Fact.	20	0.005	0.01	1.309	0.977	0.704	0.032	0.977	4.089	0.124
Matrix Fact.	20	0.005	0.05	1.269	0.948	0.752	0.092	0.948	4.365	0.124

Table 7.16

Evaluation results for the Unconstrained Matrix Factorization model using the U1R setup, across different numbers of latent factors (k), learning rates (α), and regularization parameters (λ).

Model	Latent k	α	λ	RMSE	MAE	Diversity	Novelty	Serendipity	Relevance	nDCG
Matrix Fact.	3	0.001	0.01	1.443	1.066	0.693	0.015	1.066	4.184	0.133
Matrix Fact.	3	0.001	0.05	1.449	1.073	0.618	0.014	1.073	4.170	0.132
Matrix Fact.	3	0.005	0.01	1.328	0.969	0.831	0.113	0.969	4.187	0.133
Matrix Fact.	3	0.005	0.05	1.324	0.966	0.871	0.135	0.966	4.136	0.132
Matrix Fact.	5	0.001	0.01	1.441	1.064	0.690	0.015	1.064	4.163	0.133
Matrix Fact.	5	0.001	0.05	1.448	1.072	0.676	0.014	1.072	4.144	0.132
Matrix Fact.	5	0.005	0.01	1.336	0.972	0.763	0.088	0.972	4.272	0.132
Matrix Fact.	5	0.005	0.05	1.326	0.966	0.826	0.123	0.966	4.190	0.132
Matrix Fact.	10	0.001	0.01	1.440	1.062	0.677	0.015	1.062	4.236	0.133
Matrix Fact.	10	0.001	0.05	1.447	1.070	0.687	0.014	1.070	4.182	0.133
Matrix Fact.	10	0.005	0.01	1.357	0.987	0.727	0.056	0.987	4.142	0.131
Matrix Fact.	10	0.005	0.05	1.325	0.964	0.780	0.104	0.964	4.238	0.131
Matrix Fact.	20	0.001	0.01	1.438	1.059	0.670	0.015	1.059	4.200	0.133
Matrix Fact.	20	0.001	0.05	1.444	1.068	0.688	0.014	1.068	4.186	0.133
Matrix Fact.	20	0.005	0.01	1.374	1.005	0.703	0.031	1.005	4.083	0.131
Matrix Fact.	20	0.005	0.05	1.331	0.970	0.749	0.089	0.970	4.282	0.132

Table 7.17

Evaluation results for the Unconstrained Matrix Factorization model using the U3R setup, across different numbers of latent factors (k), learning rates (α), and regularization parameters (λ).

The results obtained using the unconstrained optimization model were significantly below expectations. In fact, we were not able to surpass the benchmark performance, even under various configurations. This contrasts with our earlier experiments using standard cross-validation, where the model did yield promising results.

The last model we considered was based on Singular Value Decomposition (SVD) factorization. This approach aims to approximate the original user-item rating matrix by decomposing it into lower-dimensional latent factors that capture underlying user preferences and item characteristics. In our implementation, we used an iterative strategy where missing values were initially imputed and then refined over multiple iterations using truncated SVD, progressively improving the estimation of unknown ratings.

Model	Latent k	Steps	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
SVD fact.	3	20	1.039	0.819	0.009	4.294	0.812	0.634	0.031
SVD fact.	3	50	1.025	0.804	0.017	4.288	0.794	0.642	0.031
SVD fact.	3	100	1.022	0.800	0.034	4.117	0.790	0.657	0.031
SVD fact.	5	20	1.071	0.840	0.010	4.316	0.821	0.633	0.032
SVD fact.	5	50	1.071	0.832	0.019	4.016	0.808	0.658	0.032
SVD fact.	5	100	1.075	0.831	0.035	3.797	0.807	0.701	0.031
SVD fact.	10	20	1.066	0.836	0.013	4.301	0.830	0.650	0.032
SVD fact.	10	50	1.079	0.842	0.022	4.282	0.835	0.712	0.031
SVD fact.	10	100	1.102	0.855	0.032	4.151	0.848	0.780	0.031

Table 7.18

Evaluation results for the SVD factorization model under the U1R setup, considering different values of k (latent factors) and number of training steps.

Model	Latent k	Steps	RMSE	MAE	Novelty	Relevance	Serendipity	Diversity	nDCG
SVD fact.	3	20	1.019	0.802	0.009	4.172	0.800	0.634	0.065
SVD fact.	3	50	1.006	0.788	0.017	4.127	0.786	0.644	0.064
SVD fact.	3	100	1.003	0.784	0.032	3.951	0.783	0.659	0.064
SVD fact.	5	20	1.034	0.812	0.010	4.146	0.801	0.635	0.066
SVD fact.	5	50	1.038	0.808	0.020	4.100	0.792	0.661	0.065
SVD fact.	5	100	1.049	0.811	0.035	4.058	0.794	0.703	0.064
SVD fact.	10	20	1.037	0.813	0.013	4.253	0.805	0.651	0.066
SVD fact.	10	50	1.058	0.824	0.022	4.244	0.812	0.713	0.065
SVD fact.	10	100	1.091	0.842	0.032	4.103	0.830	0.780	0.063

Table 7.19

Evaluation results for the SVD factorization model under the U3R setup, considering different values of k (latent factors) and number of training steps.

The results obtained using SVD factorization are surprisingly strong, clearly outperforming all other models across nearly all evaluation metrics. Not only does it achieve the best predictive accuracy and recommendation quality, but it also does so with minimal computational cost.

From this, we can conclude that among all the algorithms studied and analyzed in our work, SVD factorization stands out as the most robust and efficient approach.

8. DISCUSSION AND VISUALIZATION OF RESULTS

In this section, we provide a thorough discussion and comparison of the results obtained from the different recommendation models evaluated in this work. The aim is to provide a more precise interpretation of the metrics and outcomes by analyzing them in greater depth. To support our conclusions and enhance the interpretability of the findings, we also include a variety of visual representations. These visualizations serve to clarify the performance differences among models and offer valuable insights into their behavior across various evaluation dimensions.

8.1. Visualization and Analysis of Collaborative Filtering Results

In this first part, we aim to compare the results obtained from testing the different collaborative filtering models. To do so, we present the results corresponding to the best-performing parameter configurations for each model in Figures 8.1 and 8.2. This allows us to directly compare their accuracy and overall performance under optimal conditions, providing a clearer picture of how each approach behaves in practice.

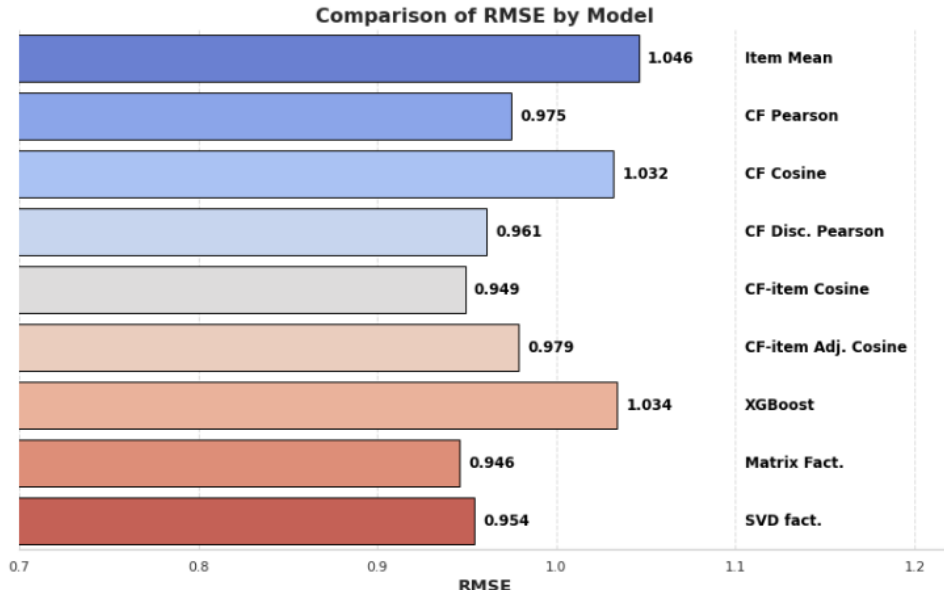


Figure 8.1

RMSE results of all the classical models considered, evaluated using 5-fold cross-validation.

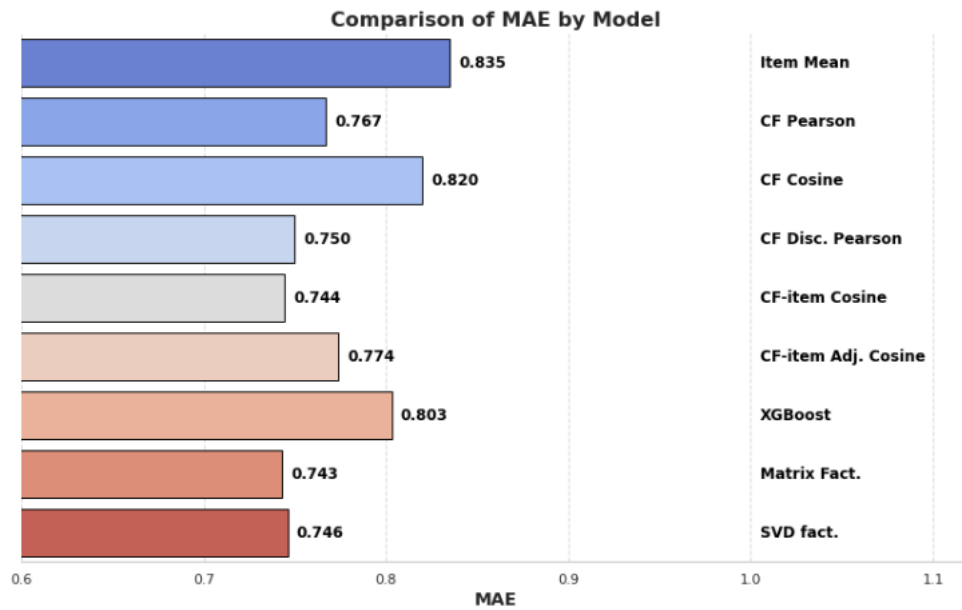


Figure 8.2

MAE results of all the classical models considered, evaluated using 5-fold cross-validation.

As previously mentioned, the model that achieves the best overall results is the unconstrained matrix factorization model, closely followed by the standard item-based collaborative filtering model using unadjusted cosine similarity. However, these models are also among the most computationally expensive in terms of training time. Therefore, an additional consideration must be made regarding whether the observed performance improvement is sufficient to justify the extra training cost.

The training times presented in Figure 8.3 correspond to the average training time per fold during the execution of the 5-fold cross-validation.

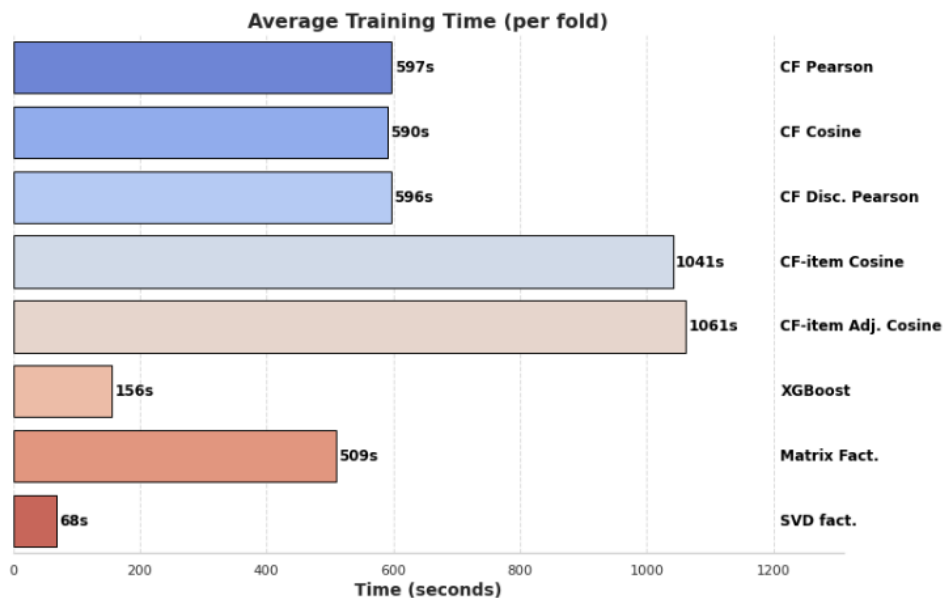


Figure 8.3

Average per-fold execution time of each model during cross-validation.

Among all the evaluated models, the SVD factorization approach clearly stands out in terms of computational efficiency. Its training time is significantly lower than that of both unconstrained matrix factorization and neighborhood-based collaborative filtering methods. This performance gain is due to the use of dimensionality reduction through Singular Value Decomposition, which compresses the rating matrix into a lower-dimensional latent space. By capturing the most relevant patterns with fewer latent factors, the model avoids the costly iterative optimization processes typically required by gradient-based methods.

Despite its simplicity and speed, the SVD model achieves an accuracy that is comparable to the best-performing approaches. In some configurations, it even approaches the top results. This balance between computational speed and predictive performance makes SVD factorization a particularly attractive option, especially in large-scale recommendation settings where efficiency is a key concern.

8.2. Visualization and Analysis of U1R & U3R Results

As we did previously, we now present the results obtained for the best configurations of each model and compare them across the three different evaluation setups we have considered: standard cross-validation, U1R, and U3R.

It is very important to emphasize that **we did not implement any explicit fairness-oriented improvements in the algorithms themselves**. What we have explored are different ways of training and evaluating our models, which remain the same throughout. Therefore, it is not meaningful to make direct comparisons between models across the different methods in terms of “better” or “worse” performance.

What we can study, however, is the relationship between models under each evaluation strategy. In principle, if a model is able to generalize well and perform strongly under the standard cross-validation setup, we would expect it to also achieve good results under the U1R and U3R setups.

As we have already observed, this expectation does not always hold, which suggests that model selection and hyperparameter tuning are not the only crucial elements. The training and evaluation framework itself also plays a key role in determining model effectiveness.

From Figures 8.4 and 8.5, we can draw several important insights that might have been harder to notice otherwise. As we can observe, under the standard 5-fold cross-validation setup, all the models evaluated are able to outperform the benchmark based on the item mean (blue dashed line). This suggests that each model is capable of generalizing and improving upon naive baselines. However, these improvements may be influenced by popularity bias, which can lead to results that are not truly generalizable across the full spectrum of user-item interactions.

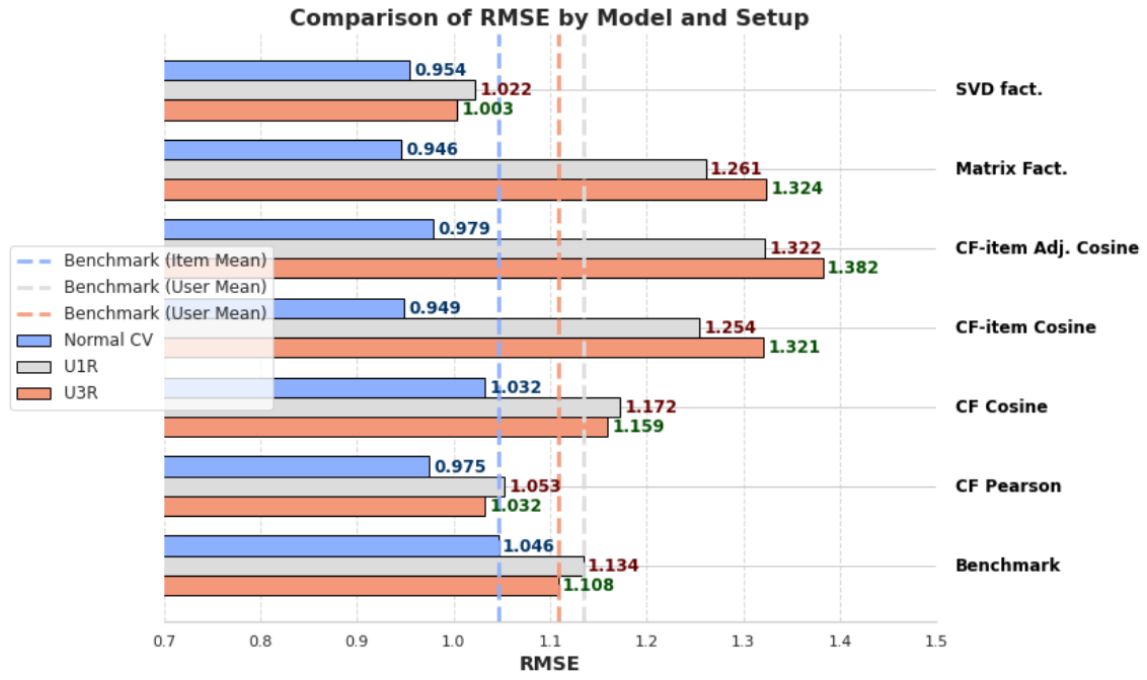


Figure 8.4
RMSE results for all classical models under the different evaluation setups.

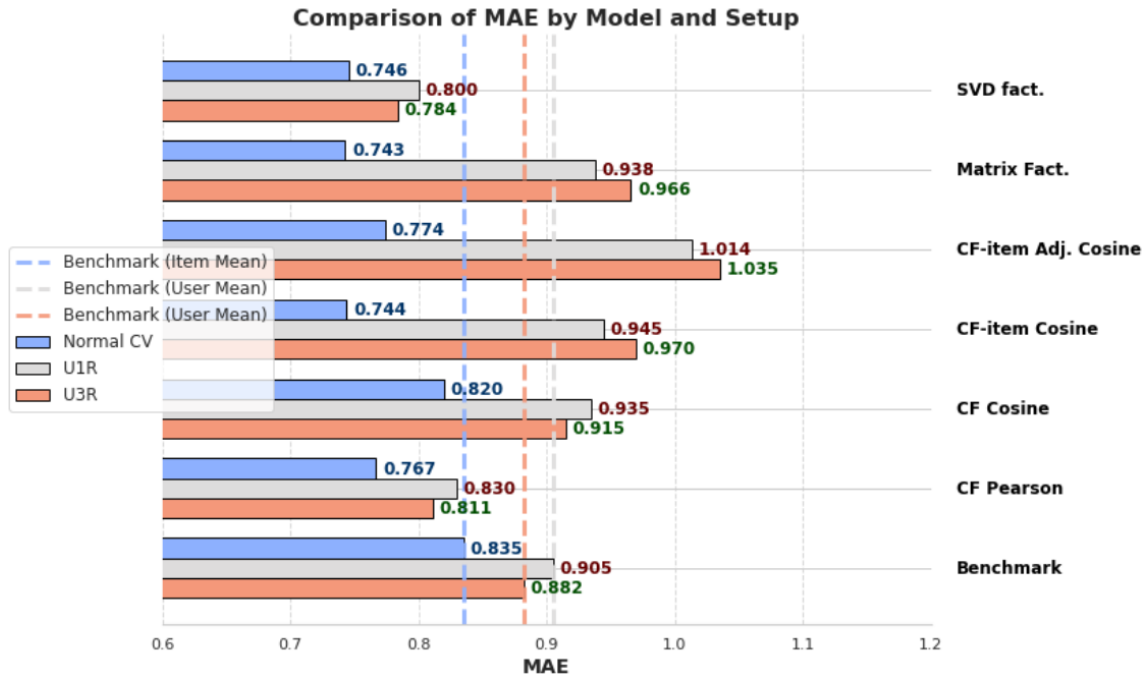


Figure 8.5
MAE results for all classical models under the different evaluation setups.

In contrast, under the U1R and U3R evaluation protocols, not all models are able to surpass the benchmark based on the user mean. This indicates that only some models are truly capable of generalizing and identifying meaningful patterns when the setup is designed to minimize the influence of popularity bias. In particular, item-based collaborative

filtering models perform worse in these setups. Their heavy reliance on item similarity makes them more susceptible to popularity effects, which negatively impacts their performance and generalization when those effects are reduced.

Finally, it is worth highlighting the effectiveness of SVD factorization. Due to its strong ability to generalize and capture relevant patterns, along with its low computational cost, SVD is the model that achieved the best results. It consistently outperforms the benchmark across all evaluation protocols, leading us to conclude that SVD is a highly effective model that should be seriously considered for future recommender systems.

9. CONCLUSIONS

As studied in the theoretical section, collaborative filtering models based on user and item similarity are the most basic approaches. They achieve solid results and offer a degree of explainability that helps interpret their predictions. However, computing similarities incurs additional computational costs, which can limit scalability. On the other hand, latent factor models based on matrix factorization generally demonstrate better performance in terms of accuracy and lower computational cost. Nonetheless, these models sacrifice explainability, as the latent factors representing hidden features are not easily interpretable. This contrast was clearly observed in the practical case with the MovieLens 100k dataset, where the SVD factorization model not only achieved the best accuracy but also proved to be computationally efficient.

This work contributes by presenting a comprehensive analysis of various collaborative filtering models used in recommender systems, including both traditional and more advanced techniques. Regarding fairness, rather than incorporating fairness constraints directly into the algorithms, our focus was on proposing a new training and evaluation framework aimed at mitigating popularity bias. The results were conclusive: by rigorously evaluating both popular and less popular items, we obtained a more balanced and realistic assessment of each algorithm. We evaluated these models under the U1R protocol proposed by Bellogín et al., 2011, which seeks to reduce popularity bias by considering a uniform distribution of items during evaluation. Beyond that, we introduced a new evaluation protocol, referred to as U3R, which extends the idea by requiring three ratings per item in the test set instead of just one. This approach provides a more robust and statistically reliable evaluation of model performance. However, it also introduces a trade-off: using more ratings for testing reduces the amount of data available for training, which may negatively affect the models' learning and generalization capabilities.

Nevertheless, this new methodology is not suitable for all cases. As observed, item-based models did not perform well under this evaluation framework, indicating that there is no one-size-fits-all solution. Therefore, we conclude that it is essential not only to study and optimize different recommender system models and their parameters but also to investigate and apply diverse training and evaluation frameworks. This holistic approach enables the selection of the most suitable configuration according to the specific goals and requirements of each system.

To ensure transparency and reproducibility, all source code and detailed experimental results are made publicly available. This includes model performance under standard 5-fold cross-validation, as well as evaluations following both the U1R fairness protocol and the newly proposed U3R extension. The complete repository can be accessed at:

<https://github.com/JorgeMoral1906/TFM-RecommenderSystems-Fairness>.

BIBLIOGRAPHY

- Aggarwal, C. C. (2016). *Recommender systems: The textbook*. Springer.
- Barocas, S., Hardt, M., & Narayanan, A. (2023). *Fairness and machine learning: Limitations and opportunities*. MIT Press.
- Bellogín, A., Castells, P., & Cantador, I. (2011). Precision-oriented evaluation of recommender systems: An algorithmic comparison. *Proceedings of the ACM Conference on Recommender Systems (RecSys)*, 333–336. <https://doi.org/10.1145/2043932.2043996>
- Ekstrand, M. D., Tian, M., Azpiazu, I. M., Ekstrand, J. D., Anuyah, O., McNeill, D., & Pera, M. S. (2018). All the cool kids, how do they fit in?: Popularity and demographic biases in recommender evaluation and effectiveness. In S. A. Friedler & C. Wilson (Eds.), *Proceedings of the 1st conference on fairness, accountability and transparency* (pp. 172–186, Vol. 81). PMLR. <https://proceedings.mlr.press/v81/ekstrand18b.html>
- Jin, D., Wang, L., Zhang, H., Zheng, Y., Ding, W., Xia, F., & Pan, S. (2023). A survey on fairness-aware recommender systems. *Information Fusion*, 100, 101906. <https://doi.org/10.1016/j.inffus.2023.101906>
- Pérez-Peralta, A. (2024). Progressing towards fairness in machine learning [Master’s Thesis, Universidad Carlos III].
- Wasserman, L. (2010). *All of statistics: A concise course in statistical inference*. Springer.

A. PROOFS OF THEOREMS AND PROPOSITIONS

Proposition 5.1.1

Let Y be a binary target variable. Consider the simplest case where we assume a symmetric loss function such that

$$l(0, 0) = l(1, 1) = 0, \quad l(1, 0) = l(0, 1) = 1.$$

Then, the optimal classifier is given by

$$f(x) = \begin{cases} 1, & \text{if } \mathbb{P}(Y = 1 \mid X = x) > \frac{1}{2}, \\ 0, & \text{otherwise.} \end{cases}$$

Proof of Proposition 5.1.1 The proof will be presented in two parts. In the first part, we will explain the reasoning behind the development and formulation of the classification function. In the second part, we will demonstrate that this classification rule is optimal for the naive case, where the goal is to minimize the expected loss function under the assumption of a symmetric loss.

Part 1. We aim to find the function f that minimizes $\mathbb{E}[l(f(X), Y)]$. The optimal function f should take the following form.

$$f(x) = \begin{cases} 1, & \text{if a certain condition holds,} \\ 0, & \text{otherwise.} \end{cases}$$

For $f(x) = 0$.

$$\begin{aligned} \mathbb{E}[l(f(X) = 0, Y)] &= \mathbb{P}(Y = 1 \mid X = x) \cdot L(0, 1) + \mathbb{P}(Y = 0 \mid X = x) \cdot L(0, 0) \\ &= \mathbb{P}(Y = 1 \mid X = x). \end{aligned}$$

For $f(x) = 1$.

$$\begin{aligned} \mathbb{E}[l(f(X) = 1, Y)] &= \mathbb{P}(Y = 0 \mid X = x) \cdot L(1, 0) + \mathbb{P}(Y = 1 \mid X = x) \cdot L(1, 1) \\ &= \mathbb{P}(Y = 0 \mid X = x). \end{aligned}$$

So, in order to minimize the expected loss, the condition for predicting the value 0 ($f(X) = 0$) is that

$$\mathbb{E}[l(f(X) = 0, Y)] < \mathbb{E}[l(f(X) = 1, Y)]$$

which is in fact the following condition

$$\mathbb{P}(Y = 1 \mid X = x) < \mathbb{P}(Y = 0 \mid X = x).$$

For predicting the value 1 ($f(X) = 1$), the condition follows the same structure, but with the inequality reversed. Using this, and the fact that the probabilities sum to 1, we can simplify our condition and obtain the resulting function f as follows.

$$f(X) = \begin{cases} 1, & \text{if } \mathbb{P}(Y = 1 | X = x) \geq \frac{1}{2}, \\ 0, & \text{otherwise.} \end{cases}$$

Part 2. Now that we have the expression for the function f , we expect it to be the optimal classifier. To demonstrate this, we will show that for any other function g , the expected value of the loss function with g will always be greater than the expected loss of the function f .

Let g be a general classifier, then the expected value of the loss function can be expressed as follows

$$\mathbb{E}[l(g(X), Y)] = \int_X L(0, 1) \cdot P(Y = 1 | X = x) \mathbb{I}_{g(x)=0} + L(1, 0) \cdot P(Y = 0 | X = x) \mathbb{I}_{g(x)=1} dP_X(x)$$

For our initial classifier f we can use a trick to better visualize why the expected loss is minimal. Specifically, we can rewrite the integrand as the minimum between certain functions.

$$\mathbb{E}[l(f(X), Y)] = \int_X \min(\mathbb{P}(Y = 1 | X = x), \mathbb{P}(Y = 0 | X = x)) dP_X(x).$$

By doing this, we can show that for any given classifier g , the difference in the expected value of the loss function between the general classifier and our previously defined function f is positive, proving that f is the optimal classifier.

$$\mathbb{E}[l(g(X), Y)] - \mathbb{E}[l(f(X), Y)] = \int_X \mathbb{I}_{g(x) \neq f(x)} |\mathbb{P}(Y = 1 | X = x) - \mathbb{P}(Y = 0 | X = x)| dP_X(x).$$

Since the term inside the integral is always non-negative and the indicator function $\mathbb{I}_{g(x) \neq f(x)}$ ensures that we only sum positive contributions when $g(x) \neq f(x)$, it follows that

$$\mathbb{E}[l(g(X), Y)] - \mathbb{E}[l(f(X), Y)] \geq 0.$$

□

Theorem 5.1.1

The risk score minimizes the square loss $\mathbb{E}[(Y - r(X))^2]$.

Proof of Theorem 5.1.1 Let $g(x)$ be a general function. Our goal is to demonstrate that the minimum of the expected squared loss occurs when $g(x) = r(x) = \mathbb{E}[Y|X = x]$, which is the risk score.

The expected squared loss for any function $g(x)$ is given by

$$\mathbb{E}[(Y - g(X))^2].$$

Expanding this expression, we get the following

$$\mathbb{E}[(Y - g(X))^2] = \mathbb{E}[Y^2 - 2Yg(X) + g(X)^2] = \mathbb{E}[Y^2] - 2g(X)\mathbb{E}[Y|X] + g(X)^2.$$

To minimize the expected loss, we compute the derivative of the loss function with respect to $g(x)$ and obtain the following

$$\frac{\partial}{\partial g(x)} (\mathbb{E}[Y^2] - 2g(x)\mathbb{E}[Y|X] + g(x)^2) = -2\mathbb{E}[Y|X] + 2g(x).$$

Setting the derivative equal to zero to find the critical points and solving for $g(x)$, we obtain

$$g(x) = \mathbb{E}[Y|X].$$

Therefore, the minimum is achieved when $g(x) = r(x) = \mathbb{E}[Y|X = x]$, which is the risk score. $\square$

Proposition 5.2.1

Let Y be a binary variable. If R is a score that satisfies independence with the sensitive variable A , then any classifier $\hat{Y} = \mathbb{I}(R > t)$, where t is a threshold, also satisfies independence, provided that the threshold t is independent of the group A .

Proof of Proposition 5.2.1 We consider the classifier $\hat{Y} = \mathbb{I}(R > t)$. This classifier outputs 1 if $R > t$ and 0 otherwise. Therefore, the value of $\hat{Y}$ depends solely on R and the threshold t .

We now want to show that $\hat{Y}$ satisfies independence with respect to A . This is

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b) \quad \forall a, b \in A.$$

Since $\hat{Y} = \mathbb{I}(R > t)$, we have that

$$P(\hat{Y} = 1 \mid A = a) = P(R > t \mid A = a),$$

and similarly with b

$$P(\hat{Y} = 1 \mid A = b) = P(R > t \mid A = b).$$

Using the assumption that A is independent of R , we know that the distribution of R does not depend on A . This implies that

$$P(R > t \mid A = a) = P(R > t \mid A = b) = P(R > t).$$

Therefore, we can show that

$$P(\hat{Y} = 1 \mid A = a) = P(\hat{Y} = 1 \mid A = b).$$

Applying these same ideas to $\hat{Y} = 0$ we can say that $\hat{Y} \perp A$.

Finally, for this independence to hold, we require that the threshold t does not depend on A . If t were dependent on A , this would introduce a correlation between $\hat{Y}$ and A , violating the independence. Hence, it is crucial that t is independent of A .

□

Proposition 5.2.2

Let $\hat{Y} = F(R, \theta)$ be a classifier, where R is the risk score and θ represents the parameters or thresholds, both independent of A . Then, if R and θ are independent of A , the classifier $\hat{Y}$ satisfies independence with respect to A . In other words, if the score R is independent of the sensitive attribute A , the decision-making function of the classifier will not depend on A , ensuring fairness in the prediction.

Proof of Proposition 5.2.2 We aim to show that if R is independent of A and θ is independent of A , the classifier $\hat{Y}$ defined as $\hat{Y} = F(R, \theta)$ satisfies the independence condition, which is

$$P(\hat{Y} | A) = P(\hat{Y}),$$

for all groups of A . That is, we want to show that the distribution of $\hat{Y}$ does not depend on the sensitive attribute A . The first step is to express the probability $P(\hat{Y} = y | A = a)$ as follows

$$P(\hat{Y} = y | A = a) = \int_{\Omega} P(\hat{Y} = y | R = r, \theta = \vartheta) P(R = r, \theta = \vartheta | A = a) d\omega,$$

where Ω represents the entire sample space of (R, θ) , and $d\omega$ is shorthand for the appropriate measure of probability. The second step is to rewrite $P(R = r, \theta = \vartheta | A = a)$ as

$$P(R = r, \theta = \vartheta | A = a) = P(R = r | A = a) P(\theta = \vartheta | R = r, A = a).$$

Since R and θ are both separately independent of A , we have

$$P(R = r | A = a) = P(R = r) \quad \& \quad P(\theta = \vartheta | A = a) = P(\theta = \vartheta).$$

Using this fact, we can say $P(\theta = \vartheta | R = r, A = a) = P(\theta = \vartheta | R = r)$.

Therefore,

$$P(R = r, \theta = \vartheta | A = a) = P(R = r) P(\theta = \vartheta | R = r).$$

Substituting this into the equation, we get

$$P(\hat{Y} = y | A = a) = \int_{\Omega} P(\hat{Y} = y | R = r, \theta = \vartheta) P(R = r) P(\theta = \vartheta | R = r) d\omega.$$

This expression is independent of A . So finally, we have the desired result

$$P(\hat{Y} = y | A = a) = P(\hat{Y} = y).$$

This proves that the classifier $\hat{Y}$ satisfies the independence condition with respect to A . $\square$

Theorem 5.2.1

If a score R satisfies calibration by groups, then it also satisfies sufficiency.

Proof of Theorem 5.2.1 We aim to show that R satisfies sufficiency, which requires proving that

$$P(Y = y \mid R = r, A = a) = P(Y = y \mid R = r), \quad \forall a \in A.$$

Since we assume calibration by group, we have the property

$$P(Y = 1 \mid R = r, A = a) = r.$$

Applying the law of total probability, we express $P(Y = y \mid R = r)$ as

$$P(Y = y \mid R = r) = \sum_{a \in A} P(Y = y \mid R = r, A = a) \cdot P(A = a \mid R = r).$$

Since Y is a binary variable, we use its standard representation:

$$P(Y = y \mid R = r, A = a) = r^y(1 - r)^{1-y}.$$

Substituting this expression into the previous equation, we obtain

$$P(Y = y \mid R = r) = \sum_{a \in A} r^y(1 - r)^{1-y} P(A = a \mid R = r).$$

Observing that the term $r^y(1 - r)^{1-y}$ does not depend on A , we factor it out:

$$P(Y = y \mid R = r) = r^y(1 - r)^{1-y} \sum_{a \in A} P(A = a \mid R = r).$$

Since the probabilities sum to one,

$$\sum_{a \in A} P(A = a \mid R = r) = 1,$$

we conclude that

$$P(Y = y \mid R = r) = r^y(1 - r)^{1-y}.$$

Since we previously established that

$$P(Y = y \mid R = r, A = a) = r^y(1 - r)^{1-y},$$

it follows that

$$P(Y = y \mid R = r, A = a) = P(Y = y \mid R = r), \quad \forall a \in A.$$

Thus, R satisfies sufficiency. □

Proposition 5.2.4

If a score R satisfies sufficiency, then there exists a function $\ell : [0, 1] \rightarrow [0, 1]$ such that $\ell(R)$ satisfies calibration by group.

Proof of Proposition 5.2.4 Fix any group $a \in A$ and define the function ℓ as follows

$$\ell(r) = P(Y = 1 \mid R = r, A = a).$$

Since R satisfies sufficiency, we know that

$$P(Y = 1 \mid R = r, A = a) = P(Y = 1 \mid R = r, A = b), \quad \forall a, b \in A.$$

This means that the function $\ell(r)$ is independent of the choice of a , ensuring that the transformation $\ell(R)$ is well-defined for all groups.

Now, consider any two groups a, b . By definition of ℓ , we have

$$r = P(Y = 1 \mid \ell(R) = r, A = a).$$

Substituting $\ell(R) = r$ in terms of R , we obtain

$$P(Y = 1 \mid R = \ell^{-1}(r), A = a).$$

By the definition of ℓ , we know that

$$P(Y = 1 \mid R = \ell^{-1}(r), A = a) = P(Y = 1 \mid R = \ell^{-1}(r), A = b).$$

Thus, we conclude that

$$P(Y = 1 \mid \ell(R) = r, A = a) = P(Y = 1 \mid \ell(R) = r, A = b).$$

This shows that $\ell(R)$ satisfies calibration by group, completing the proof. □

Proposition 5.2.5

Let $R(X) = \mathbb{E}[Y | X]$ be the optimal score function obtained from an unconstrained supervised learning model. Then $R(X)$ satisfies calibration by group. Specifically, for any group attribute A and any measurable subset S of the feature space, we have

$$P(Y = 1 | R = r, X \in S) = r.$$

Proof of Proposition 5.2.5 We want to show that the following holds for any set S

$$P(Y = 1 | R = r, X \in S) = r.$$

Expressing the conditional probability $P(Y = 1 | R = r, X \in S)$ as a ratio of integrals, we have

$$P(Y = 1 | R = r, X \in S) = \frac{P(Y = 1, R = r, X \in S)}{P(R = r, X \in S)} = \frac{\int_S P(Y = 1, R = r | X = x) p(x) dx}{\int_S P(R = r | X = x) p(x) dx},$$

where $p(x)$ is the probability density function of X . The integrals are taken over the set S .

We simplify this using conditional probability

$$P(Y = 1 | R = r, X \in S) = \frac{\int_S P(Y = 1 | R = r, X = x) P(R = r | X = x) p(x) dx}{\int_S P(R = r | X = x) p(x) dx}.$$

Since $R = r(X)$, the value of R is fully determined by the function r applied to X . This means that

$$P(R = r | X = x) = \begin{cases} 1 & \text{if } r(x) = r, \\ 0 & \text{if } r(x) \neq r. \end{cases}$$

As a result, any integral involving this conditional probability is only non-zero for those x where $r(x) = r$.

Additionally, we have that

$$P(Y = 1 | R = r, X = x) = P(Y = 1 | X = x) = r(x).$$

Putting this together, we obtain the following expression

$$P(Y = 1 | R = r, X \in S) = \frac{\int_{S \cap \{x: r(x)=r\}} r(x) p(x) dx}{\int_{S \cap \{x: r(x)=r\}} p(x) dx}.$$

This shows that the conditional probability is computed only over the region where the score function $r(x)$ matches the given value r .

Since we are integrating over the set where $r(x) = r$, we can replace $r(x)$ with r

$$P(Y = 1 | R = r, X \in S) = \frac{r \int_{S \cap \{x: r(x)=r\}} p(x) dx}{\int_{S \cap \{x: r(x)=r\}} p(x) dx}.$$

Finally, we can cancel the integrals in the numerator and denominator

$$P(Y = 1 \mid R = r, X \in S) = r.$$

Therefore, the optimal score $r(x) = E[Y \mid X = x]$ satisfies group calibration for any set S .

□

Proposition 5.3.1

Assume that A and Y are not independent. Then, sufficiency and independence cannot both hold.

Proof of Proposition 5.3.1 We proceed by contradiction. Suppose that both independence and sufficiency hold simultaneously:

$$A \perp R \quad \text{and} \quad A \perp Y \mid R.$$

By the contraction rule for conditional independence, this implies that

$$A \perp (Y, R),$$

which means that A is independent of the joint distribution of Y and R . In particular, marginalizing over R preserves independence, leading to

$$A \perp Y.$$

However, this contradicts our initial assumption that A and Y are not independent. Therefore, our assumption that both independence and sufficiency can hold simultaneously must be false.

Taking the contrapositive, we conclude that:

$$A \not\perp Y \implies A \not\perp R \quad \text{or} \quad A \not\perp Y \mid R.$$

This completes the proof. □

Proposition 5.3.2

Assume that Y is binary, A is not independent of Y , and R is not independent of Y . Then, independence and separation cannot both hold.

Proof of Proposition 5.3.2 proceed by contradiction. Suppose that both independence and separation hold simultaneously:

$$A \perp R \quad \text{and} \quad A \perp Y \mid R.$$

Taking the contrapositive, we aim to show that

$$A \not\perp Y \implies A \not\perp R \quad \text{or} \quad R \not\perp Y.$$

Applying the law of total probability we obtain

$$P(R = r \mid A = a) = \sum_y P(R = r \mid A = a, Y = y)P(Y = y \mid A = a).$$

Using the assumptions $A \perp R$ and $A \perp Y \mid R$, this simplifies to

$$P(R = r) = \sum_y P(R = r \mid Y = y)P(Y = y \mid A = a).$$

Applying the law of total probability again, we also have

$$P(R = r) = \sum_y P(R = r \mid Y = y)P(Y = y).$$

Equating both expressions we obtain the following expression.

$$\sum_y P(R = r \mid Y = y)P(Y = y) = \sum_y P(R = r \mid Y = y)P(Y = y \mid A = a).$$

Since Y is binary, we define the following probabilities:

- $p = P(Y = 0)$
- $p_a = P(Y = 0 \mid A = a)$
- $r_0 = P(R = r \mid Y = 0)$
- $r_1 = P(R = r \mid Y = 1)$

With these definitions, the original equation becomes

$$p \cdot r_0 + (1 - p) \cdot r_1 = p_a \cdot r_0 + (1 - p_a) \cdot r_1.$$

Rearranging terms, we get:

$$p(r_0 - r_1) = p_a(r_0 - r_1).$$

This equation can only hold in one of two cases:

- If $r_0 = r_1$, which implies $R \perp Y$ (i.e., R is independent of Y).
- If $p = p_a$ for all a , which implies $Y \perp A$ (i.e., Y is independent of the sensitive attribute A).

Since we assumed that $A \not\perp Y$ and $R \not\perp Y$, at least one of these conditions must be violated, leading to a contradiction. Thus, independence and separation cannot hold simultaneously. $\square$

Proposition 5.3.3

Assume that all events in the joint distribution of (A, R, Y) have positive probability, and assume $A \perp Y$. Then, separation and sufficiency cannot both hold at the same time.

Proof of Proposition 5.3.3 As in the previous demonstration, we apply a result on conditional independence that will be used to ease the proof, which can be found in Theorem 17.2 of Wasserman's text (Wasserman, 2010). This result shows that

$$A \perp R \mid Y \quad \text{and} \quad A \perp Y \mid R \implies A \perp (R, Y).$$

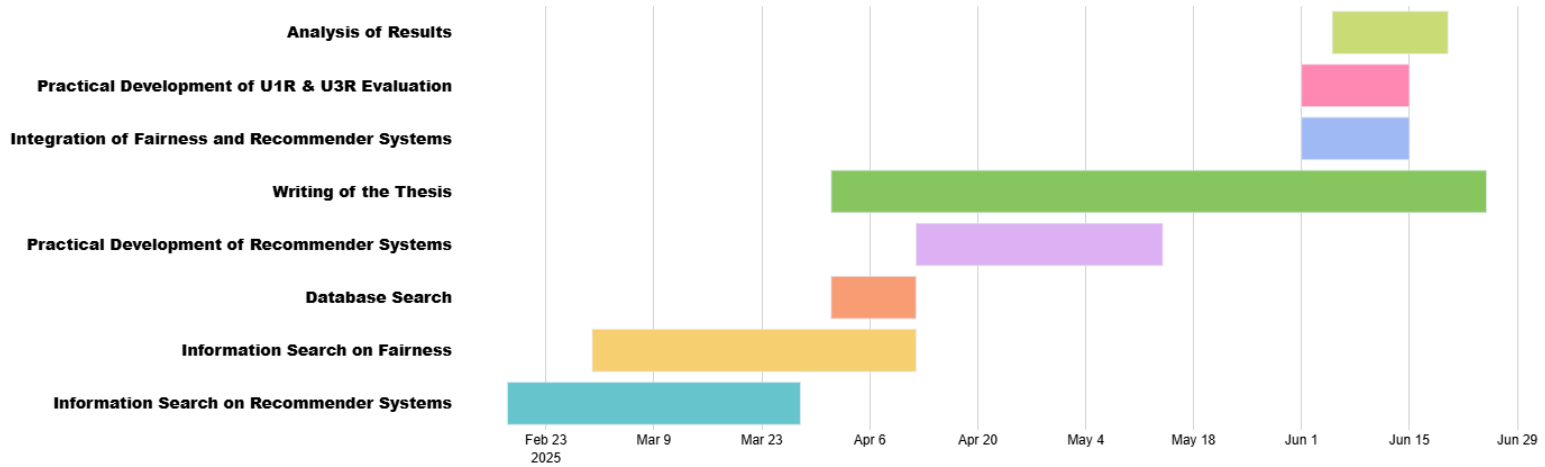
Moreover, we have

$$A \perp (R, Y) \implies A \perp R \quad \text{and} \quad A \perp Y.$$

Taking the contrapositive completes the proof. □

B. THESIS PROJECT TIMELINE

Master's Thesis Timeline – Recommender Systems & Fairness



DECLARACIÓN DE USO DE IA GENERATIVA EN EL TRABAJO DE FIN DE GRADO

He usado IA Generativa en este trabajo

Marca lo que corresponda:

☒ SI ☐ NO

Si has marcado SI, completa las siguientes 3 partes de este documento:

Parte 1: reflexión sobre comportamiento ético y responsable

Ten presente que el uso de IA Generativa conlleva unos riesgos y puede generar una serie de consecuencias que afecten a la integridad moral de tu actuación con ella. Por eso, te pedimos que contestes con honestidad a las siguientes preguntas (*Marca lo que corresponda*):

Pregunta		
1. En mi interacción con herramientas de IA Generativa he remitido datos de carácter sensible con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter sensible
2. En mi interacción con herramientas de IA Generativa he remitido materiales protegidos por derechos de autor con la debida autorización de los interesados.		
SÍ, he usado estos materiales con autorización	NO, he usado estos materiales sin autorización	NO, no he usado materiales protegidos
3. En mi interacción con herramientas de IA Generativa he remitido datos de carácter personal con la debida autorización de los interesados.		
SÍ, he usado estos datos con autorización	NO, he usado estos datos sin autorización	NO, no he usado datos de carácter personal

4. Mi utilización de la herramienta de IA Generativa ha respetado sus términos de uso, así como los principios éticos esenciales, no orientándola de manera maliciosa a obtener un resultado inapropiado para el trabajo presentado, es decir, que produzca una impresión o conocimiento contrario a la realidad de los resultados obtenidos, que suplante mi propio trabajo o que pueda resultar en un perjuicio para las personas.	
SI	NO

Si **NO** has contado con la autorización de los interesados en alguna de las preguntas 1, 2 ó 3, explica brevemente el motivo (*por ejemplo, "los materiales estaban protegidos pero permitían su uso para este fin" o "los términos de uso, que se pueden encontrar en esta dirección (...), impiden el uso que he hecho, pero era imprescindible dada la naturaleza del trabajo"*).

Parte 2: declaración de uso técnico

Utiliza el siguiente modelo de declaración tantas veces como sea necesario, a fin de reflejar todos los tipos de iteración que has tenido con herramientas de IA Generativa. Incluye un ejemplo por cada tipo de uso realizado donde se indique: *[Añade un ejemplo]*.

Declaro haber hecho uso del sistema de IA Generativa (Nombre del sistema/herramienta IA y versión: ChatGPT, Gemini, Copilot...) **para:**

Documentación y redacción:

- *Soporte a la reflexión en relación con el desarrollo del trabajo: proceso iterativo de análisis de alternativas y enfoques utilizando la IA*

Pedí una comparación sencilla entre los distintos tipos de sistemas de recomendación (como los basados en contenido, colaborativos o híbridos), con ejemplos que ayudaran a entender mejor cómo funcionan, sus puntos fuertes, limitaciones y en qué casos se usan más.

Se pidió una explicación sencilla con ejemplos claros para entender las distintas nociones de equidad (independence, separation y sufficiency) en sistemas de recomendación, y cómo se aplican en situaciones reales con grupos sensibles.

- *Revisión o reescritura de párrafos redactados previamente*

He pedido ayuda para reorganizar y dejar más claros algunos párrafos que ya había escrito, corregir errores gramaticales en inglés y ajustar ciertos detalles para que el texto quedara más limpio y bien redactado.

He pedido sugerencias de ideas para estructurar mejor el trabajo, incluyendo posibles títulos de secciones y cómo organizar los contenidos de forma coherente.

- *Búsqueda de información o respuesta a preguntas concretas*

Las preguntas realizadas se orientaban a resolver dudas concretas sobre cómo escribir ciertos comandos en LaTeX, así como a obtener explicaciones y ejemplos específicos relacionados con el funcionamiento y aplicación de sistemas de recomendación y técnicas de evaluación de la equidad (fairness)

- *Búsqueda de bibliografía*
- *Resumen de bibliografía consultada*

Se ha pedido ayuda para resumir y explicar ciertas ideas que no estaban del todo claras, añadiendo algún ejemplo sencillo que facilitara la comprensión del concepto o técnica explicada.

- *Traducción de texto consultados*

Solicité la traducción de párrafos o ideas técnicas que resultaban difíciles de entender en inglés, así como la aclaración de términos técnicos cuyo significado no conocía.

Desarrollar contenido específico

Se ha hecho uso de IA Generativa como herramienta de soporte para el desarrollo del contenido específico del TFG, incluyendo:

- *Asistencia en el desarrollo de líneas de código (programación)*

Pedí ayuda para corregir errores en el código, desarrollar ideas complejas de programación, verificar la corrección del código, recomendar librerías que optimicen el trabajo y sugerir otras formas de mejorar la eficiencia del código.

- *Generación de esquemas, imágenes, audios o vídeos*

- *Procesos de optimización*

Solicité apoyo para optimizar el rendimiento del código, incluyendo técnicas para mejorar la velocidad de ejecución, reducir el uso de memoria y simplificar funciones complejas. También pedí recomendaciones sobre librerías y herramientas que facilitan la optimización y la limpieza del código.

- *Tratamiento de datos: recogida, análisis, cruce de datos...*

Pedí ayuda para reorganizar y preparar los datos correctamente, adaptándolos al formato requerido por los distintos algoritmos de sistemas de recomendación, asegurando que las matrices de usuarios, ítems y ratings fueran compatibles y optimizadas para el entrenamiento y evaluación.

- *Inspiración de ideas en el proceso creativo*

Pedí ideas sobre variables sensibles comunes en fairness (género, edad, ubicación) y cómo abordar el popularity bias para evitar que solo se recomienden los ítems más populares.

- *Otros usos vinculados a la generación de puntos concretos del desarrollo específico del trabajo*

Si crees necesario incluirlo, añade, en cada uno de los casos:

empleando el *prompt* ...

(escribe la petición realizada a la IAG)

teniendo como interacción ...

(describe qué interacción realizaste con la IAG tras la respuesta del prompt)

Ejemplo: ***Declaro haber hecho uso del sistema de IA Generativa Chat GPT 3.5 para buscar información empleando el prompt: “Dime un ejemplo concreto que ilustre la aplicación de la propuesta urbanística de la Ciudad de los 15 minutos en España” teniendo como interacción la proposición del ejemplo concreto del distrito de Chamartín que se ha reflejado en la memoria.***

Parte 3: reflexión sobre utilidad

Por favor, aporta una valoración personal (formato libre) sobre las fortalezas y debilidades que has identificado en el uso de herramientas de IA Generativa en el desarrollo de tu trabajo. Menciona si te ha servido en el proceso de aprendizaje, o en el desarrollo o en la extracción de conclusiones de tu trabajo.

La IA generativa me ha facilitado mucho la parte de programar, sobre todo para encontrar y corregir errores en el código, lo que ha hecho que todo el proceso sea más rápido y menos complicado. También me ayudó a optimizar un poco la carga de trabajo y a escribir tablas más rápido, pero su función principal fue sin duda hacer la programación mucho más sencilla.